Java aktuell

iJUG member
java-aktuell.eu

Java & Architektur

OpenJDK 26/27, Spring Boot Best Practices, HTMX und moderne Frontend-Strategien

KI & Verantwortung

LLMs und APIs, Bias in neuronalen Netzen, Architektur im KI-Zeitalter

Menschen & Software

Psychologie in Projekten, Wirksamkeit von Entwickler:innen, Informatik-Nachwuchs fördern



Ausgabe 04/2026

Java aktuell

Praxis. Wissen. Networking. Das Magazin für Entwicklerinnen und Entwickler

D: 7,95 EUR | CH: 11,99 CHF | A: 9,00 EUR | Benelux: 8,50 EUR

4 191978 307959 04

Liebe Leserinnen und Leser,

in dieser Ausgabe erwartet euch erneut eine abwechslungsreiche Mischung aus Technologie, Methodik, Architektur und den Auswirkungen von KI auf unseren Berufsalltag. Den Auftakt gibt wieder einmal Andreas Badelt mit seinem Java-Tagebuch, das euch auf den neuesten Stand rund um alle Geschehnisse innerhalb der Java-Community bringt. Im Anschluss folgt Andreas Monschau, der seine satirische Reihe über das zuverlässige Scheitern von Softwareprojekten fortsetzt und zeigt, warum ungeprüfte Biases und Denkfehler in Projekten oft ein unterschätztes Risiko darstellen.

Anschließend wirft Falk Sippach ab Seite 14 einen Blick auf die Neuerungen von OpenJDK 26 und 27. Ihr erfahrt, welche Entwicklungen die Java-Plattform in den Bereichen Performance, Nebenläufigkeit, Sicherheit und moderne Schnittstellen voranbringen. Mit dem Zusammenspiel von KI und Schnittstellen beschäftigen sich Benny Burkert und Felix Medam, die erläutern, warum LLMs beim Konsumieren von HTTP-APIs mehr benötigen als technische Erreichbarkeit und eine OpenAPI-Beschreibung allein.

Für die tägliche Entwicklungspraxis liefert Julius Mischok ab Seite 32 zehn pragmatische Empfehlungen für langfristig stabile und wartbare Spring-Boot-Anwendungen. Danach zeigen Sebastian Sprenger und Felix Seidel, wie sich mit HTMX und einem backendzentrierten Ansatz moderne Anwendungen auch mit deutlich weniger JavaScript umsetzen lassen. Passend dazu untersucht Emil Pawlowski, wie viel Verantwortung ein Client überhaupt tragen sollte, und vergleicht verschiedene Ansätze vom klassischen HTML bis zur React-basierten Single-Page-Application.

Einen wichtigen Schwerpunkt bildet zudem das Thema Künstliche Intelligenz. Eldar Sultanow und Lars Santesson beleuchten in ihrem Artikel ab Seite 52, wie sich die Rolle von IT-Architektinnen und -Architekten im KI-Zeitalter verändert. Dr. Sandra C. Signore zeigt anschließend, wie neuronale Netze gesellschaftliche Verzerrungen übernehmen können und welche Verantwortung Entwicklerinnen und Entwickler entlang des gesamten KI-Lebenszyklus tragen.

Abgerundet wird diese Ausgabe durch zwei Beiträge, die den Menschen in den Mittelpunkt stellen: Christian Seifert geht der Frage nach, warum sich viele Entwicklerinnen und Entwickler im Projektalltag trotz hoher Kompetenz oft ohnmächtig fühlen und wie sie wieder mehr Wirksamkeit entfalten können. Zum Abschluss berichtet Fin Labusch von zehn Jahren IT-Workshops für Kinder und gibt Einblicke, wie ihr junge Menschen spielerisch für Informatik und Programmierung begeistern könnt.

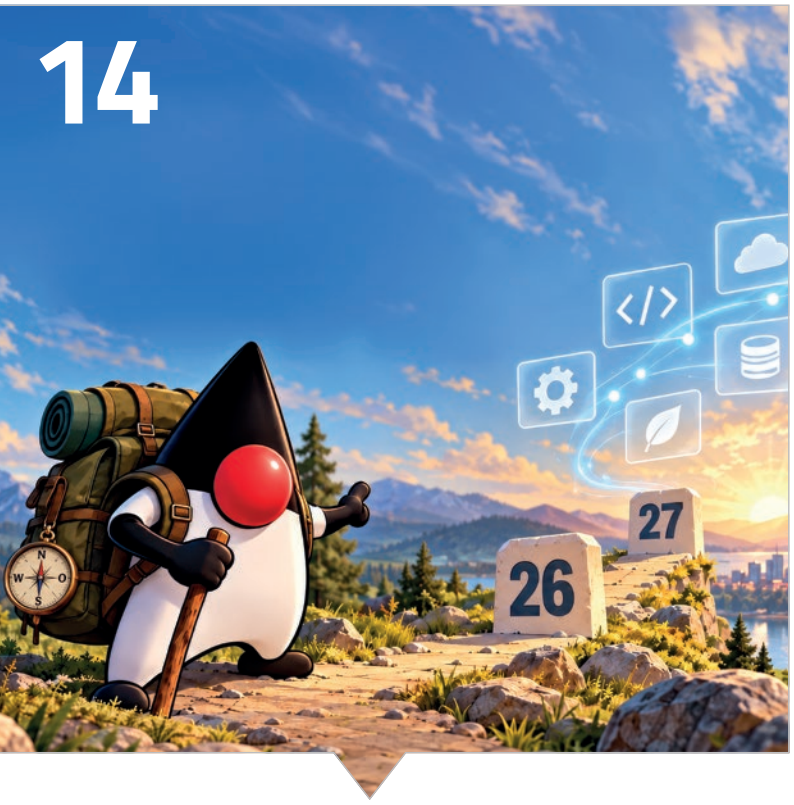
Wir wünschen euch viel Freude beim Lesen und viele Impulse, die euch inspirieren, motivieren, weiterbringen und zum Nachdenken anregen.





Lisa Damerow
Redaktionsleitung Java aktuell

INHALT



Java 26 und 27:
Die Releases im Detail



LLMs und APIs:
Warum OpenAPI allein nicht reicht

3 Editorial

6 Java-Tagebuch

10 Wie du deine Softwareprojekte zuverlässig ruinierst – Teil 3: Nehmt jedes Bias mit (ohne es zu hinterfragen)
Andreas Monschau

14 OpenJDK 26 und 27:
Die Neuerungen bei Java in 2026
Falk Sippach

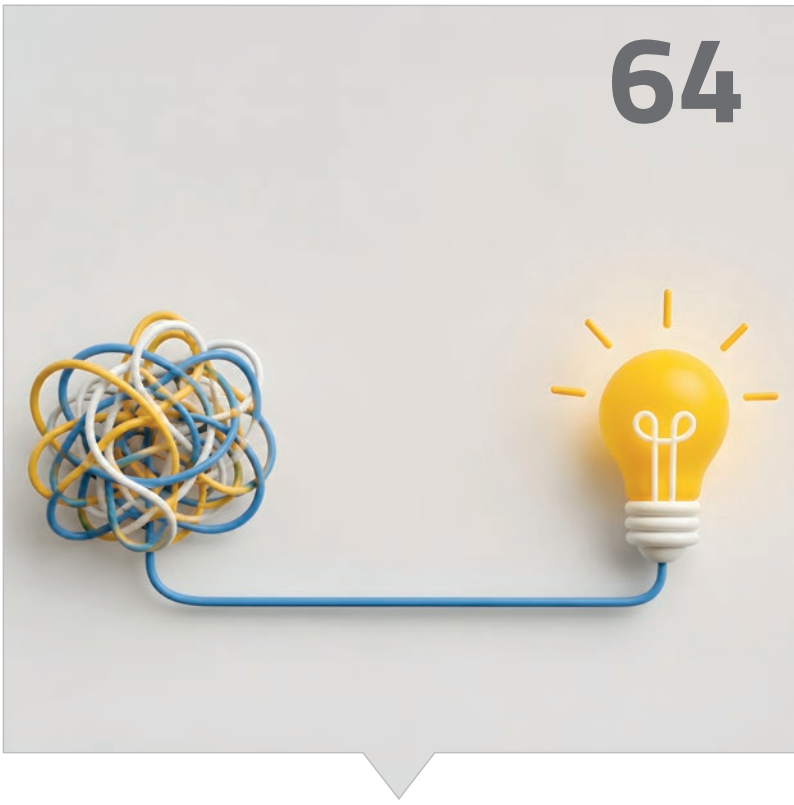
22 Wenn LLMs HTTP-APIs konsumieren: Qualität, Kontext und ausführbare Capabilities
Benny Burkert & Felix Medam

32 10 Tipps für Spring Boot:
Weniger Magie, mehr Pragmatismus
Julius Mischok

38 Dank HTMX zu einem Backend-lastigen Frontend
Sebastian Sprenger & Felix Seidel



Bias in KI-Systemen:
Wenn Algorithmen Vorurteile lernen



Vom Frust zur Wirksamkeit: Wie gute Entwickler:innen wieder mehr Einfluss gewinnen

44 Mehr als nur Single-Page-Applications: Wie viel Verantwortung braucht der Client wirklich?
Emil Pawlowski

52 Die neue Rolle des Architekten im KI-Zeitalter
Eldar Sultanow & Lars Santesson

58 Fairness not found: Wie neuronale Netze gesellschaftliche Verzerrungen lernen
Dr. Sandra C. Signore

64 Zwischen Frust und Verantwortung: Warum gute Entwickler sich ohnmächtig fühlen – und wie sie Wirksamkeit zurückgewinnen
Christian Seifert

70 Was macht ein Java-Entwickler in seiner Freizeit? Oder: 10 Jahre IT-Workshops für Kinder
Fin Labusch

78 Impressum/Inserenten

OpenJDK 26 und 27: Die Neuerungen bei Java in 2026

Falk Sippach, embarc Software Consulting GmbH




26
27

Mit Java 26 und Java 27 sind in diesem Jahr wieder zwei neue Versionen der Java-Plattform erschienen. Das OpenJDK 26 wurde im März veröffentlicht, das OpenJDK 27 folgte im September. Viele Änderungen zahlen auf langfristige Themen ein: bessere Start- und Laufzeitperformance, reifere Nebenläufigkeit, mehr Plattformintegrität und modernere Schnittstellen für Web und Security. In diesem Artikel werfen wir wieder einen Blick auf die Neuerungen.

Beide Releases enthalten zusammen 19 JEPs – wobei ein erheblicher Teil davon bereits bekannte Preview- und Inkubator-Features weiterentwickelt. Für einen ersten Überblick ist wie immer die Projektseite des OpenJDK *[1]* ein guter Startpunkt. Zehn JEPs (JDK Enhancement Proposals) sind dort bei Java 26 *[2]* gelistet:

- 500: Prepare to Make Final Mean Final
- 504: Remove the Applet API
- 516: Ahead-of-Time Object Caching with Any GC
- 517: HTTP/3 for the HTTP Client API
- 522: G1 GC: Improve Throughput by Reducing Synchronization
- 524: PEM Encodings of Cryptographic Objects (Second Preview)
- 525: Structured Concurrency (Sixth Preview)
- 526: Lazy Constants (Second Preview)
- 529: Vector API (Eleventh Incubator)
- 530: Primitive Types in Patterns, instanceof, and switch (Fourth Preview)

Java 27 *[3]* bringt neun weitere beziehungsweise erneut aufgenommene JEPs:

- 523: Make G1 the Default Garbage Collector in All Environments
- 527: Post-Quantum Hybrid Key Exchange for TLS 1.3
- 531: Lazy Constants (Third Preview)
- 532: Primitive Types in Patterns, instanceof, and switch (Fifth Preview)
- 533: Structured Concurrency (Seventh Preview)
- 534: Compact Object Headers by Default
- 536: JFR In-Process Data Redaction
- 537: Vector API (Twelfth Incubator)
- 538: PEM Encodings of Cryptographic Objects (Third Preview)

Das sind diesmal deutlich weniger JEPs als in den letzten Releases. Es setzt sich jedoch der Trend fort, Java durch gezielte Verbesserungen moderner, effizienter und leistungsfähiger zu machen. Neben Änderungen an der Sprache gibt es auch Optimierungen in der JVM, neue APIs sowie die Unterstützung neuer Protokollversionen. Besonders auffällig ist, dass mehrere Preview-Features weiter geschärft, statt vorschnell finalisiert zu werden. Die Java Releases im Jahr 2026 erscheinen damit zwar weniger spektakulär, legen aber wichtige Grundlagen für die nächsten Jahre.

Pattern Matching nähert sich auch den primitiven Typen

Pattern Matching gehört seit mehreren Jahren zu den sichtbarsten Sprachentwicklungen in Java. Nach Type, Record sowie Unnamed Patterns und Pattern Matching für switch soll das Konzept auch auf primitive Typen ausgedehnt werden. Primitive Werte sollen künftig nach denselben Regeln behandelt werden wie Referenztypen. So lässt sich natürlich schon immer ein long-Wert direkt in einem switch verwenden *(siehe Listing 1)*.

```
static String classify(long value) {
     return switch (value) {
           case 0L -> "zero";
           case 1L -> "one";
           default -> "many";
     };
}
```

Listing 1: Constant Patterns mit primitiven Werten im switch

Interessanter ist jedoch die Verbindung von Pattern Matching und primitiven Konvertierungen. So können jetzt neben Referenztypen auch primitive Typen als Patterns eingesetzt werden oder bei Record Patterns auch kombiniert werden *(siehe Listing 2)*. Dadurch werden Konvertierungen, instanceof und switch in ein konsistenteres Typsystem integriert. Java 26 hat mit JEP 530 die vierte Preview gebracht. Unter anderem wurden die Regeln für unbedingt exakte Konvertierungen und die Dominanz von case-Labeln weiter präzisiert. Java 27 hat das Feature mit JEP 532 in der fünften Preview unverändert erneut vorgelegt. Dass nur noch wenige Änderungen notwendig sind, deutet darauf hin, dass sich das Feature seiner endgültigen Form nähert. Wie im Beispiel in *Listing 3* zu sehen ist, können Entwickler künftig auch einfach prüfen, ob ein ganzzahliger Wert in den Wertebereich eines Byte-Werts passt.

```
record Measurement(String sensor, long value) {}
var m = new Measurement("temperature", 25);
if (m instanceof Measurement(String sensor, int value)) {
     IO.println(sensor + ": " + value);
}
```

Listing 2: Record Patterns mit primitiven Typen

```
String checkByte(int value) {
  if (value instanceof byte b) {
     return "byte b = " + b;
  }
  return "kein byte: " + value;
}
IO.println(checkByte(127)); // b = 127
IO.println(checkByte(128)); // kein byte: 128
```

Listing 3: Prüfung auf Wertebereiche von primitiven Datentypen

Structured Concurrency: Die API wird weiter geschärft

Mit Virtual Threads hat Java ein leichtgewichtigeres Thread-Modell bekommen. Dabei wird aber nur die Frage beantwortet, wie güns-

tig viele nebenläufige Tasks ausgeführt werden können. Es beantwortet jedoch nicht automatisch die Frage, wie deren Lebenszyklus, Fehler und Ergebnisse strukturiert werden. Hier setzt Structured Concurrency an. Mehrere fachlich zusammengehörige Tasks werden in einem `StructuredTaskScope` zusammengefasst *(siehe Listing 4)*.

```
try (var scope = StructuredTaskScope.open()) {
     var user = scope.fork(this::loadUser);
     var orders = scope.fork(this::loadOrders);
     scope.join();
     return new Result(user.get(), orders.get());
}
```

Listing 4: StructuredTaskScope startet zwei Aufgaben

Die gestarteten Tasks können den Scope nicht einfach unkontrolliert überleben. Abbruch, Fehlerbehandlung und Lebensdauer werden Teil der Kontrollstruktur. Im Jahr 2026 ging Structured Concurrency in zwei weitere Preview-Runden. Java 26 bringt die sechste, Java 27 die siebte Preview. Dabei hat sich vor allem die Joiner-API verändert. Ein Joiner beschreibt, wann eine Gruppe von Tasks als beendet gilt und welches der Ergebnisse relevant ist. Beispielsweise liefert `Joiner.allSuccessfulOrThrow()` die Ergebnisse nur dann, wenn alle Tasks erfolgreich waren. Für ein Rennen mehrerer Tasks liefert `Joiner.anySuccessfulOrThrow()` das erste erfolgreiche Ergebnis zurück. In Java 27 wurde die Fehlerbehandlung weiter ausgebaut. Joiner können nun expliziter bestimmen, welche Art von Exception geworfen wird. Neue überladene Methoden wie `Joiner.allSuccessfulOrThrow(exceptionMapper)` erlauben zudem eine anwendungsbezogene Übersetzung von Fehlern. Auch die Timeout-Behandlung wurde überarbeitet. Java 27 hat unter anderem `CancelledByTimeoutException` eingeführt und `onTimeout()` zu `timeout()` umbenannt.

Die ungewöhnlich hohe Zahl der Preview-Runden sollte nicht darüber hinwegtäuschen, dass die Grundidee inzwischen stabil ist. Die Arbeit konzentriert sich inzwischen vor allem darauf, eine API zu finden, die typische Fälle einfach ausdrückt und trotzdem genügend Flexibilität für Fehlerbehandlung, Abbruch und Timeouts bietet.

Lazy Constants: verzögert initialisieren, anschließend unveränderlich

Eine weitere Lücke im Java-Programmiermodell betrifft die verzögerte Initialisierung von final-Attributen. Da ein final-Feld unveränderlich ist, muss der Wert unmittelbar initialisiert werden. Ist diese Initialisierung teuer und soll erst beim ersten tatsächlichen Zugriff geladen werden, benötigte man bisher eigene, nicht trivial zu implementierende Lazy-Initialization-Mechanismen. Hier setzen Lazy Constants an. Sie sind eine Art Wrapper, deren Inhalt beim ersten Zugriff einmalig und threadsicher berechnet und anschließend lesend zur Verfügung steht *(siehe Listing 5)*.

```
static final LazyConstant<Configuration> CONFIG =
     LazyConstant.of(Configuration::load);
[..]
// Initialisierung erfolgt verzögert beim ersten Zugriff
CONFIG.get();
```

Listing 5: Erzeugung einer LayzConstant und spätere Initialisierung

Das Feature begann bereits in Java 25 unter dem Namen Stable Values. Mit Java 26 wurde die API deutlich umgebaut und das Konzept in Lazy Constants umbenannt. Die ursprünglichen Möglichkeiten, einen Wert explizit über Methoden wie `trySet()` oder `setOrThrow()` zu setzen, sind verschwunden. Stattdessen wird die Initialisierungsfunktion bereits beim Anlegen des Lazy Constants angegeben. Das macht die Abstraktion wesentlich klarer: Es handelt sich nicht um eine einmal beschreibbare Variable, sondern um eine Konstante, deren Erzeugung aufgeschoben wird.

Es lassen sich auch Lazy Collections (Listen und Maps) auf diese Art definieren. Die einzelnen Felder bzw. Elemente werden erst erzeugt,

```
enum Option { VERBOSE, DRY_RUN, STRICT }

// Return true when the given Option is enabled
boolean isEnabled(Option option) {
     IO.println(String.format("isEnabled(%s) ...", option.name()));
     // could be read from a configuration file (expensive)
     if (option == Option.DRY_RUN) {
         return false; // DRY_RUN is disabled
     }
     return true;
}

// Lazily initialized Set of Options
final Set<Option> OPTIONS =
     Set.ofLazy(EnumSet.allOf(Option.class), this::isEnabled);

void main() {
     IO.println("started ...");
     if (OPTIONS.contains(Option.DRY_RUN)) {
           IO.println("dry run mode enabled");
     }
     if (OPTIONS.contains(Option.STRICT)) {
           IO.println("strict mode enabled");
     }
     // Actual processing logic
}
```

Listing 6: Lazy Set mit verzögerter Initialisierung der Werte

wenn tatsächlich darauf zugegriffen wird. Java 27 ergänzt schließlich `Set.ofLazy(...)` *(siehe Listing 6)*. Gleichzeitig sind `isInitialized()` und `orElse()` verschwunden. Denn der Initialisierungszustand soll bewusst nicht Teil der sichtbaren Abstraktion sein. Der Benutzer soll eine Lazy Constant wie einen normalen unveränderlichen Wert behandeln. Damit geht es bei Lazy Constants um mehr als nur syntaktischen Zucker. Die JVM weiß nach erfolgreicher Initialisierung, dass sich der Wert nicht mehr ändern wird und kann diese Information für Optimierungen nutzen.

HTTP/3 erreicht den Java HTTP Client

Der mit Java 11 eingeführte HTTP-Client wurde in Java 26 um die Enum-Konstante `HttpClient.Version.HTTP_3` ergänzt. Ein Client kann damit HTTP/3 bevorzugen *(siehe Listing 7)*. Am grundsätzlichen Programmiermodell ändert sich nichts. Requests werden weiterhin wie gewohnt erzeugt und versendet.

```
var client = HttpClient.newBuilder()
            .version(HttpClient.Version.HTTP_3)
            .build();
```

Listing 7: Aktivierung von HTTP/3

HTTP/3 verwendet im Gegensatz zu HTTP/1.1 und HTTP/2 nicht TCP, sondern QUIC über UDP. Einer der Vorteile besteht darin, dass Paketverluste nicht mehr sämtliche parallel laufende HTTP-Streams einer Verbindung blockieren. HTTP/3 wird allerdings nicht zum neuen Default. Der HTTP-Client bevorzugt weiterhin HTTP/2. Wer HTTP/3 verwendet, muss es bewusst auswählen. Unterstützt der Server das Protokoll nicht, kann der Client auf HTTP/2 oder HTTP/1.1 zurückfallen. Das OpenJDK beschränkt sich dabei bewusst auf HTTP/3. Eine allgemeine öffentliche QUIC-API gehört nicht zum JEP.

G1 wird schneller – und schließlich überall Standard

Auch bei der Garbage Collection gibt es wieder kleine Änderungen. Der G1 ist bereits seit Java 9 der Standard-Garbage-Collector, wird aber weiterhin intensiv optimiert. JEP 522 reduziert in Java 26 die Synchronisation zwischen Anwendungs- und GC-Threads. Bestimmte Barrier-Operationen mussten bislang globale Datenstrukturen aktualisieren, was bei vielen Threads zu Konflikten führen konnte. Die neue Implementierung verwendet stärker thread-lokale Strukturen und soll dadurch insbesondere den Durchsatz von Anwendungen mit vielen aktiven Threads verbessern.

Und Java 27 geht noch einen Schritt weiter. Denn bislang war die Aussage „G1 ist der Default-GC" nicht ganz richtig. Auf sehr kleinen Systemen, insbesondere bei nur einer verfügbaren CPU oder sehr wenig Arbeitsspeicher, konnte die HotSpot-JVM stattdessen automatisch den Serial GC auswählen. JEP 523 entfernt diese Sonderregel und ab Java 27 ist der G1 unabhängig von der erkannten Umgebung der Standard. Der Serial GC verschwindet nicht und kann weiterhin explizit gewählt werden *(siehe Listing 8)*. Relevant ist die

```
java ... -XX:+UseSerialGC
```

Listing 8: Serial GC beim Start aktivieren

Änderung vor allem bei kleinen (Docker-) Containern. Bisher konnte schon eine Änderung der CPU- oder Memory-Limits indirekt dazu führen, dass eine Anwendung mit einem anderen Garbage Collector startete. Dieses Verhalten wird nun vorhersehbarer.

Kleinere Objekt Header werden Standard

Auch Compact Object Headers sind keine völlig neue Technik. Sie stammen aus Project Lilliput und wurden bereits in vorherigen Java-Versionen eingeführt und erprobt. Ein Java-Objekt benötigt auf einer typischen 64-Bit-JVM zusätzlich zu seinen Nutzdaten einen Header. Mit komprimierten Class Pointern waren dies typischerweise zwölf Byte. Compact Object Headers reduzieren diesen Header auf acht Byte. Die Ersparnis von vier Byte erscheint zunächst wenig. Bei Anwendungen mit Millionen kleiner Objekte summiert sich der Unterschied jedoch erheblich. Zusätzlich passen mehr Objekte in die CPU-Caches und der Garbage Collector muss weniger Speicher verarbeiten. Erste Erfahrungen (zum Beispiel bei Anwendungen auf AWS) zeigen, dass sich der Heap-Verbrauch um zirka 10 bis 20 % reduziert und der Durchsatz um zirka 5 bis 10 % erhöht.

Java 25 machte Compact Object Headers zu einem regulären, aber noch optionalen Feature. Mit dem JEP 534 werden sie ab Java 27 standardmäßig aktiviert. Wer das alte Layout benötigt, kann vorerst zurückschalten *(siehe Listing 9)*. Für normalen Java-Code ist die Änderung transparent. Interessant wird sie vor allem für JVM-nahe Tools und Bibliotheken, die Annahmen über das konkrete Objektlayout treffen.

```
java ... -XX:-UseCompactObjectHeaders
```

Listing 9: Compact Object Headers deaktivieren

Project Leyden arbeitet weiter an Startzeit und Warmup

Ein weiteres Performance-Thema ist das Ahead-of-Time (AOT) Caching aus Project Leyden. Die Idee besteht darin, die Zeit, welche die JVM zum Warmwerden nach dem Start einer Anwendung braucht, in einen vorherigen Trainingslauf zu verlagern. Informationen über geladene Klassen, Objekte und Profilinformationen können anschließend in einem AOT-Cache gespeichert und bei späteren Starts wiederverwendet werden. Bislang gab es dabei ein Problem: Im Cache abgelegte Java-Objekte waren teilweise an das konkrete Speicherlayout des Garbage Collectors gebunden. Insbesondere ZGC verwendet intern andere Repräsentationen als G1, Serial oder Parallel GC. Der JEP 516 (Ahead-of-Time Object Caching with Any GC) beseitigt diese Einschränkung. Objekte können nun in einem GC-neutralen Format gespeichert und beim Start entsprechend dem tatsächlich verwendeten Garbage Collector materialisiert werden. Damit müssen sich Anwendungen nicht mehr zwischen den Startup-Vorteilen des AOT-Cache und einem Low-Latency-Collector wie ZGC entscheiden. Diese Neuerung ist vor allem ein weiterer Baustein von Project Leyden. Es wird darauf hingearbeitet, immer mehr Arbeit aus Startup und Warmup vorzuverlegen, ohne Java in ein statisches Native-Image-Modell wie bei GraalVM zu verwandeln.

Security: von PEM bis Post Quantum

Java hat seit jeher einen starken Fokus auf Security-Funktionen. Im Moment wird die Sprache insbesondere auf die Zukunft der

Quantencomputer vorbereitet, denn dann sind heutige Verschlüsselungsstandards nicht mehr ausreichend. Auch für im Sicherheitsumfeld relevante Formate werden APIs entwickelt. Gerade mit PEM-Dateien kommen Entwickler häufig in Berührung *(Listing 10)*. Trotzdem hatte Java lange Zeit keine einheitliche API dafür. Daher mussten Entwickler selbst Header entfernen, Base64 dekodieren und anschließend mit verschiedenen Klassen wie CertificateFactory, KeyFactory und KeySpec kombinieren.

```
-----BEGIN CERTIFICATE-----
...
-----END CERTIFICATE-----
```

Listing 10: PEM File

Im OpenJDK 25 wurde die neue PEM API mit den Klassen PEMEncoder und PEMDecoder als Preview vorgestellt. In Java 26 und 27 erschienen die zweite und dritte Preview. Dabei wurde die API nochmals strukturell überarbeitet, unter anderem wurde die bisherige Abstraktion DEREncodable durch die allgemeinere BinaryEncodable ersetzt. Zertifikate, Key Pairs und verschiedene Key Specs implementieren anschließend diese Schnittstelle. Auch PEMEncoder wurde entsprechend angepasst. Der Java Almanac *[4]* zeigt die Veränderungen der dritten Preview detailliert. Die Grundidee bleibt aber stabil: Das JDK soll PEM als eigenes kryptografisches Format verstehen, statt Anwendungen dazu zu zwingen, PEM lediglich als Base64-Text zu behandeln.

Noch grundlegender ist JEP 527 (Post-Quantum Hybrid Key Exchange for TLS 1.3). Ein ausreichend leistungsfähiger Quantencomputer könnte viele heute verbreitete Public-Key-Verfahren angreifen. Das Problem beginnt jedoch potenziell schon vor der Existenz eines solchen Computers. Denn verschlüsselter Netzwerkverkehr kann heute aufgezeichnet und möglicherweise Jahre später entschlüsselt werden („Harvest now, decrypt later"). Java 27 unterstützt deshalb drei hybride TLS-1.3-Gruppen, wie zum Beispiel X25519MLKEM768. Dabei werden ein klassischer elliptischer Schlüsselaustausch und das quantenresistente ML-KEM kombiniert. Der hybride Ansatz ist wichtig. Denn Java setzt nicht einfach alles auf den vergleichsweise jungen Post-Quantum-Algorithmus. Stattdessen ist die Verbindung sicher, solange mindestens einer der beiden beteiligten Algorithmen sicher bleibt. Für Anwendungen ist das weitgehend transparent. Bestehender JSSE-, HTTPS- oder HTTP-Client-Code muss nicht auf eine neue API umgestellt werden. Unterstützen beide Seiten eine hybride Gruppe, kann sie während des TLS-Handshakes ausgehandelt werden. Damit gehört Post-Quantum TLS zu den relevanten Neuerungen des Jahres, obwohl die meisten Java-Entwickler dafür keine einzige neue Codezeile schreiben werden.

Ebenfalls unter das Thema Sicherheit fällt der JEP 536 (JFR In-Process Data Redaction). Durch den Java Flight Recorder (JFR) können auch System Properties, Umgebungsvariablen und Kommandozeilenargumente aufgezeichnet werden. Darunter befinden sich in realen Anwendungen nicht selten Passwörter, Tokens oder andere Secrets. Bisher konnten JFR-Dateien nur nachträglich bereinigt werden. Dann waren die sensiblen Daten allerdings bereits im Umlauf. Ab Java 27 werden solche Informationen deshalb schon innerhalb der JVM geschwärzt, bevor sie in das Recording gelangen. Potenziell sensible Informationen werden durch einen Platzhalter ersetzt. Gerade weil JFR-Dateien häufig zwischen Produktion, Entwicklung und externem Support ausgetauscht werden, ist das ein sinnvoller Schritt hin zu sichereren Defaults.

Standardmäßig werden Inhalte geschwärzt, die vordefinierten Patterns (*secret*, *pwd*, *password*, *api*key*, …) entsprechen. Beim Starten der Anwendung können diese Muster erweitert oder durch eigene ersetzt werden. Dabei wird nach Mustern für Umgebungsvariablen und System Properties sowie für Programmargumente unterschieden. Zudem ist es auch möglich, die Schwärzung komplett zu deaktivieren *(siehe Listing 11)*.

```
java -XX:FlightRecorderOptions:'redact-
key=none,redactargument:none' ...
```

Listing 11: Schwärzung der Geheimnisse durch JFR deaktivieren

`final` soll künftig wirklich final bedeuten

Auch außerhalb klassischer Kryptografie verfolgt Java das Thema Security mit Integrity by Default weiter. Eigentlich sollte ein final-Feld nach seiner Initialisierung nicht mehr verändert werden können. Deep Reflection kann diese Garantie aber in bestimmten Situationen umgehen. Und insbesondere Bibliotheken und Java Tools machen davon viel Gebrauch. Auch Entwickler könnten sich diese Tricks jederzeit zunutze machen. Der JVM entgehen durch diese Optionen aber Möglichkeiten der tiefgreifenden Optimierung. Der JEP 500 (Prepare to Make Final Mean Final) ändert ab Java 26 zwar noch nicht viel. Aber es werden durch die JVM Warnungen erzeugt, wenn Reflection ein final-Feld nachträglich verändert.

Für die Übergangszeit existieren Optionen wie `--enable-final-field-mutation`. Wichtig dabei ist jedoch die Richtung. Nicht eine Bibliothek soll sich selbst diese Fähigkeit geben können, sondern die Anwendung beziehungsweise deren Betreiber muss sie bewusst erlauben. Das verbessert nicht nur die Integrität von Objekten. Wenn die JVM sich tatsächlich darauf verlassen kann, dass ein final-Feld unveränderlich bleibt, eröffnet dies auch zusätzliche Optimierungsmöglichkeiten.

Applets sind endgültig Geschichte

Ein Stück Java-Geschichte endet im Jahr 2026. Der JEP 504 hat die Applet API vollständig entfernt. Das betrifft insbesondere das Package java.applet sowie die damit verbundenen APIs in AWT, Swing und JavaBeans. Browser-Plug-ins für Java spielen schon seit vielen Jahren keine praktische Rolle mehr. Die API war seit Java 9 deprecated und seit Java 17 zur Entfernung vorgesehen. Java 26 vollzieht nun den letzten Schritt. Der API-Vergleich zwischen Java 25 und 27 *[4]* zeigt entsprechend die Löschung von JApplet und AppletInitializer. Für moderne Anwendungen dürfte das kaum Konsequenzen haben. Die Entfernung ist vor allem ein gutes Beispiel dafür, dass Java trotz seiner ausgeprägten Rückwärtskompatibilität historische APIs nicht unbegrenzt mitführt.

Vector API: Warten auf Valhalla

Eine fast schon traditionelle Position auf der JEP-Liste nimmt die Vector API ein. Java 26 bringt die elfte, Java 27 die zwölfte Incubator-Ver-

sion. Die API ermöglicht explizite SIMD-Berechnungen *(siehe Listing 12)*. SIMD steht für Single Instruction, Multiple Data und die JVM kann solche Operationen auf Prozessor-Instruktionsbefehle wie AVX oder NEON abbilden und damit die Berechnung deutlich beschleunigen.

```
var va = FloatVector.fromArray(SPECIES, a, offset);
var vb = FloatVector.fromArray(SPECIES, b, offset);
va.add(vb).intoArray(result, offset);
```

Listing 12: Beispiel der Vector API

In beiden JEPs gibt es kaum inhaltliche Veränderungen. Der Grund für die lange Inkubation liegt vor allem bei Project Valhalla und der Einführung von Value Classes. Denn Vektoren sind typische Value Objects. Sie brauchen keine Identität und sollten möglichst ohne klassischen Objekt-Overhead verarbeitet werden. OpenJDK möchte deshalb zunächst die entsprechenden Value-Class-Funktionen aus Valhalla abwarten und die Vector API anschließend darauf aufbauen, statt heute eine API zu standardisieren, die kurze Zeit später grundlegend geändert werden müsste. Im März 2027 sollen Value Classes in Java 28 übrigens als Preview Einzug halten. Ab dann werden wir vermutlich auch Previews und hoffentlich bald auch eine finale Version der Vector API sehen.

Kleine Verbesserungen in den APIs

Nicht jede interessante Änderung bekommt einen JEP. Die Release Notes *([5]* und *[6])* sowie der API-Vergleich des Java Almanac *[4]* zeigen aber noch viele kleinere Ergänzungen, die im Entwickleralltag durchaus nützlich sind. So ergänzt Java 26 die Klasse String um `text.equalsFoldCase(other)` und `text.compareToFoldCase(other)` sowie den Comparator um `String.UNICODE_CASEFOLD_ORDER`. Case Folding ist für Unicode-definierte Vergleiche gedacht, bei denen Groß- und Kleinschreibung ignoriert werden sollen. Es ist robuster als beide Strings zunächst mit `toLowerCase()` zu transformieren. Auch in Java 27 wird die String-Klasse ergänzt. Mit `text.encodedLength(StandardCharsets.UTF_8)` lässt sich feststellen, wie viele Bytes ein String in einer bestimmten Kodierung benötigt, ohne zunächst über `text.getBytes(StandardCharsets.UTF_8)` ein byte-Array erzeugen zu müssen. Passend dazu hat auch die Foreign Function & Memory API neue String-Operationen auf MemorySegment und SegmentAllocator erhalten.

Bereits in Java 26 wird die Klasse Process von AutoCloseable und Closeable abgeleitet. In einem Try-with-Resources-Block wird beim Schließen sichergestellt, dass der Prozess beendet ist und die verbundenen Streams und Ressourcen freigegeben werden *(siehe Listing 13)*.

```
try (Process process =
        new ProcessBuilder("some-command").start()) {
    // ...
}
```

Listing 13: Process in Try-with-Resources

Ebenfalls in Java 26 wurden der Klasse Comparator zwei Convenience-Methoden (`comparator.min(a, b)` und `comparator.max(a, b)`) hinzugefügt. Damit muss für die Auswahl des kleineren oder größeren Elements nicht mehr selbst `compare()` ausgewertet werden.

Die Java Database Connectivity API (JDBC) wurde in Java 26 auf die Version 4.5 aktualisiert. Neu sind unter anderem die SQL-Typen `JDBCType.JSON` und `JDBCType.DECFLOAT`. Außerdem implementieren Array, Blob, Clob, NClob und SQLXML nun AutoCloseable. Damit kann beispielsweise ein Blob direkt mit Try-with-Resources verwendet werden, wodurch beim Verlassen des Blocks das Freigeben der Ressourcen sichergestellt wird.

In Java 27 wurden die Klassen Math und `StrictMath` um die inversen hyperbolischen Funktionen für Sinus, Cosinus und Tangens (`Math.asinh(x)`, `Math.acosh(x)`, `Math.atanh(x)`) erweitert. BigDecimal bekommt zusätzlich die Methode `rootn(n, mathContext)` für die Berechnung einer n-ten Wurzel. Und die vordefinierten DateTimeFormatter akzeptieren nun auch kurze ISO-8601-Offsets wie „2026-09-01T10:00+02" und nicht mehr nur Varianten wie 2026-09-01T10:00+02:00. Das ist vor allem beim Einlesen externer ISO-8601-Daten praktisch.

Außerdem räumt Java weiter auf. Neben der Applet API verschwinden weitere alte Mechanismen. Dazu gehören `Thread.stop()` sowie mehrere alte Netzwerkmethoden und außerdem `ThreadPoolExecutor.finalize()`. Auch beim Java Launcher wurden alte Optionen (`-noverify`, `-Xverify:none`, `-verifyremote`, `-noclassgc`) entfernt. Insbesondere die Möglichkeit, die Bytecode-Verifikation vollständig abzuschalten, verschwindet damit. Solche Änderungen zeigen eine Entwicklung, die sich durch viele aktuelle Java-Releases zieht. Java behält seine starke Rückwärtskompatibilität, trägt aber nicht mehr jede historische Ausnahme unbegrenzt weiter.

Fazit und Ausblick

Die Neuerungen von Java 26 und 27 wirken auf den ersten Blick weniger spektakulär als Releases, die beispielsweise Records, Sealed Classes oder Virtual Threads eingeführt haben. Betrachtet man beide Versionen gemeinsam, wird jedoch eine klare Entwicklung sichtbar. Java verbessert weiter sein Programmiermodell. Pattern Matching wird auf primitive Typen ausgeweitet, Structured Concurrency ergänzt Virtual Threads um eine strukturierte Behandlung zusammengehöriger Tasks und Lazy Constants schließen die Lücke zwischen verzögerter Initialisierung und Unveränderlichkeit.

Gleichzeitig entwickelt sich die Laufzeitumgebung weiter. G1 wird schneller und zum einheitlichen Default, Compact Object Headers reduzieren den Speicherverbrauch und Project Leyden arbeitet weiter daran, Start- und Warmup-Zeiten zu verkürzen. Ein besonders wichtiges Thema ist Security. Eine standardisierte PEM API beseitigt eine lange bestehende Lücke, TLS bereitet sich mit hybriden Verfahren auf Post-Quantum-Kryptografie vor, der Java Flight Recorder schützt Secrets besser und Java schränkt mit „Make Final Mean Final" schrittweise Mechanismen ein, die zentrale Garantien der Sprache umgehen. Mit HTTP/3 und zahlreichen kleineren Verbesserungen an Strings, JDBC, Prozessen, Mathematik und anderen APIs kommen außerdem Funktionen hinzu, die sich unmittelbar im Anwendungscode nutzen lassen.

Java 26 und 27 stehen damit weniger für einzelne große Neuerungen, sondern für die konsequente Weiterentwicklung der Plattform

auf vielen Ebenen gleichzeitig: Sprache, Nebenläufigkeit, Performance, Startup, Netzwerk, Sicherheit und Klassenbibliothek. Während neue Möglichkeiten hinzukommen, verschwinden gleichzeitig historische Altlasten und Sonderwege. Und richtig spannend wird es im nächsten Jahr, wenn mit dem JEP 401 nach mehr als 10 Jahren Entwicklung endlich Value Classes/Objects Einzug ins OpenJDK erhalten. Seit dem Feature Freeze vom OpenJDK 27 (Rampdown Phase One am 04.06.2027) sind die ersten Veränderungen im Git Repo zu sehen. Die Ankündigung auf der Mailingliste *[7]* zeigt einige Details. Value Classes sind die größte Veränderung des Java-Typsystems seit Generics. Die darauf aufbauenden JEPs (Enhanced Primitive Boxing, Null-Restricted and Nullable Types und Null-Restricted Value Class Types) und weitere Ideen können auf dem JEP-Index *[8]* nachgelesen werden. Nach einigen ruhigeren Releases wird es mit dem OpenJDK 28 wieder spektakuläre neue Features geben. Uns wird als Java-Entwicklern in den nächsten Jahren also nicht langweilig werden.

Referenzen:

[1] *https://openjdk.org/projects/jdk/*
[2] *https://openjdk.java.net/projects/jdk/26/*
[3] *https://openjdk.java.net/projects/jdk/27/*
[4] *https://javaalmanac.io/jdk/27/apidiff/25/*
[5] *https://jdk.java.net/26/release-notes*
[6] *https://jdk.java.net/27/release-notes*
[7] *https://mail.openjdk.org/archives/list/jdk-dev@openjdk.org/thread/AIA3O3LHFZ6T7TIPH7KZT4WS4B6U72U5/*
[8] *https://openjdk.org/jeps/0#Draft-and-submitted-JEPs*



Falk Sippach
falk@jug-da.de
https://twitter.com/sippsack

Falk Sippach ist bei der embarc Software Consulting GmbH als Softwarearchitekt, Berater und Trainer stets auf der Suche nach dem Funken Leidenschaft, den er bei seinen Teilnehmern, Kunden und Kollegen entfachen kann. Bereits seit über 20 Jahren unterstützt er in meist agilen Softwareentwicklungsprojekten im Java-Umfeld. Als aktiver Bestandteil der Community (Mitorganisator der JUG Darmstadt) teilt er zudem sein Wissen gern in Artikeln, Blog-Beiträgen, sowie bei Vorträgen auf Konferenzen oder User Group Treffen und unterstützt bei der Organisation diverser Fachveranstaltungen. Falk twittert unter @sippsack.

MITMACHEN UND AUTORIN ODER AUTOR WERDEN!



Du kennst dich in einem bestimmten Gebiet aus dem Java-Themenbereich aus und du möchtest als Autorin oder Autor für die Java aktuell dein Wissen mit der Community teilen?

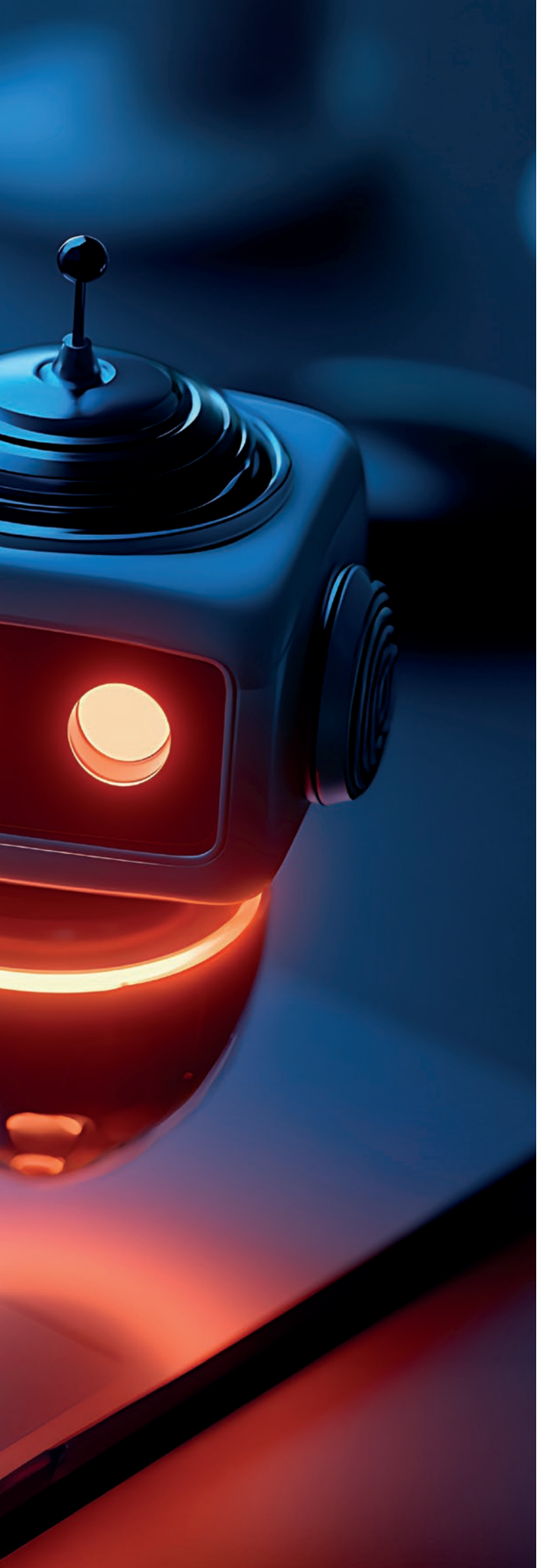
Nimm Kontakt zu uns aufund sende deinen Artikelvorschlag an:
redaktion@java-aktuell.eu
Wir freuen uns, von dir zu hören!

Weitere Informationen

Wenn LLMs HTTP-APIs konsumieren: Qualität, Kontext und ausführbare Capabilities

Benny Burkert, Felix Medam, codecentric AG





Ein Agent ruft die API auf, die wir betreiben, bekommt HTTP 201 zurück und meldet die Aufgabe als erledigt, obwohl fachlich nichts abgeschlossen ist. Wir zeigen, warum technische Erreichbarkeit und eine OpenAPI-Spec dafür nicht genügen, welche zwei Problemklassen dahinterstecken und was API-Provider konkret tun können: von Governance und Linting über Kontextbereitstellung mit Markdown und Arazzo bis zu ausführbaren Capabilities.

1. Eine valide API ist noch keine verständliche API

Eine Hochschule verleiht Geräte: Kameras, Messtechnik, Notebooks. Der Bestand wird über eine HTTP-API verwaltet, die Ausleihen, Ausgaben, Rückgaben und Prüfungen abbildet. Ein Teammitglied in der Poolverwaltung promptet seinen Agenten mit: „Das Objektiv aus Ausleihe A-4711 liegt hier auf dem Tresen, bitte einbuchen."

Der Agent hat alles, was er nach gängiger Vorstellung braucht: einen Zugangstoken, die OpenAPI-Spec der API (im Folgenden „Spec" genannt), und HTTP kennt er so gut, wie man es aus Millionen Beispielen lernen kann. Was er tut, sieht vernünftig aus: Er sucht in der Spec nach einer passenden Operation, findet `POST /returns`, füllt die Pflichtfelder aus, schickt den Request ab und bekommt `201 Created` zurück. Er meldet: „Rückgabe gebucht, Vorgang abgeschlossen."

Zwei Tage später steht jemand am Tresen und will genau dieses Objektiv abholen. Es ist gesperrt. Objektive dieser Kategorie müssen nach jeder Rückgabe zur Sichtprüfung: Die Rückgabe war angenommen, das zeigt der `201`, aber das Exemplar steht seitdem im Zustand *'in Prüfung'*, und über die Freigabe entscheidet eine dritte Person. Dass eine zusätzliche Meldung an die Poolverwaltung nötig ist, um diese Prüfung anzustoßen, war die entscheidende Wissenslücke des Agenten.

1.1 Warum ein erfolgreicher HTTP-Aufruf nicht genügt

Der Aufruf war technisch fehlerfrei: gültiges Token, korrektes Schema, erwarteter Statuscode. Aber fachlich unvollständig, und zwar aus zwei sehr verschiedenen Gründen.

Die erste Ursache liegt in der Spec selbst. Die Operation heißt dort `createRtn`, ihre `summary` lautet „Create rtn", und das Feld `condition` im Request Body ist als `string` ohne weitere Angabe deklariert – die zulässigen Werte stehen in einem Confluence-Artikel, den vor zwei Jahren jemand geschrieben hat. Zwei weitere Operationen, die es in der Spec gibt, wurden im letzten Release entfernt. Der Agent hat mit einer Spec gearbeitet, die nicht ausreichend beschreibt, was die API tut.

Die zweite Ursache steht in keiner OpenAPI-Spec und ist dort auch nicht vorgesehen: Rückgaben laufen je nach Gerätetyp, Nutzungsart und gemeldetem Zustand unterschiedlich weiter, dass „ange-

nommen" nicht „abgeschlossen" heißt und dass ein offener Prüfauftrag eine Sperre und keine Freigabe ist, sprich fachliches Wissen über die Domäne. Mitarbeiter*innen lernen es in der Einarbeitung, Entwickler*innen erfragen es im Fachbereich und gießen es in Code. Der Agent hat weder Einarbeitung noch Fachbereich. Er hat nur den Prompt und die OpenAPI-Spec.

1.2 Was der Artikel leistet und was nicht

Wir betrachten in diesem Artikel, was API-Provider über Erreichbarkeit und eine formale Spec hinaus bereitstellen können, damit ein Agent einen fachlichen Auftrag korrekt bearbeitet. Wir ordnen dafür zwei Lösungsdimensionen ein: die Qualität der API und ihrer OpenAPI-Spec, und die Bereitstellung des fachlichen, situativen und prozeduralen Kontexts. Für jede Form, die wir vorstellen, benennen wir das adressierte Problem und die Grenzen. Wir sprechen dabei von HTTP-APIs mit OpenAPI-Spec, der Gedankengang überträgt sich bedingt auch auf andere web-basierte APIs.

Zwei Themen, die in unserer Geschichte vorkommen, vertiefen wir bewusst nicht. Das erste ist der Betrieb: Zugangsdaten, Rechte, Rate Limits, Nachvollziehbarkeit der Agentenaktivität. Das ist ein großes Feld für sich und würde den Rahmen sprengen. Das zweite ist das Auffinden der richtigen API und ihrer Operationen, kurz API-Discovery. Dafür gibt es etablierte Ansätze, wie den Code Mode von Cloudflare *[1]*, die wir hier nicht vertiefen. Wir entwerfen keine Zielarchitektur und liefern keinen Implementierungsleitfaden, sondern geben einen Überblick über Auswirkungen des API-Designs auf KI-Consumer und wie man ihnen helfen kann.

Das ist auch für Leser*innen relevant, die keine API betreiben, da dieselben Methoden helfen, wenn Menschen mit oder ohne KI-Werkzeugen einen Client für eine API entwickeln, zum Beispiel ein Frontend, eine CLI, ein SDK oder einen MCP-Server (Model Context Protocol).

2 Zwei Problemklassen statt eines diffusen Kontextproblems

Was unserem Agenten passiert ist, wird in Diskussionen gern als „Kontextproblem" zusammengefasst. Das ist zu grob, um daraus Handlungen abzuleiten. Genauer trifft es die Frage, welches Wissen dem Agenten gefehlt hat. Die beiden Ursachen aus Abschnitt 1.1 betreffen verschiedene Arten von Wissen und haben deshalb unterschiedliche Lösungsansätze. Dabei kann nicht nur die entwickelnde Partei die Spec verbessern, sondern auch der Fachbereich muss Wissen bereitstellen, das für den KI-Agenten sonst nicht zugänglich ist. Vielleicht auch, weil es nie aufgeschrieben wurde.

2.1 Problemklasse 1: die Spec taugt nicht

Die erste Klasse betrifft die API und ihre Spec als Artefakt. Sie ist unvollständig, veraltet, uneinheitlich oder nicht standardkonform, sprich von minderer Qualität, auf deren Grundlage der Agent falsche Annahmen trifft. Vier Ausprägungen von Qualität in OpenAPI-Specs:

- **Aktualität** ist die schwerwiegendste Ausprägung. Wenn die Spec Operationen beschreibt, die es nicht mehr gibt, oder fehlen Felder, die längst bereitgestellt werden, sind Probleme nahezu sicher.
- **Vollständigkeit** meint, dass die gesamte API an einer Stelle steht und der Consumer nicht an drei Orten suchen, aber auch inhaltlich keine Annahmen treffen muss.
- **Standardkonformität** betrifft die erwartungsgemäße Verwendung von HTTP. Ein POST, das eine reine Abfrage ist, oder ein Fehler im Body einer `200`-Antwort brechen jede Annahme, die ein LLM aus Millionen anderer APIs gelernt hat.
- **Verständlichkeit** entscheidet, ob aus der Spec hervorgeht, was eine Operation fachlich tut. `createRtn` mit der Zusammenfassung „Create rtn" enthält keine relevante Information.

Der letzte Punkt ist für Agenten besonders relevant. Ein Modell kompensiert Lücken aus seinem Training. Bei einer verbreiteten Domäne mit gängigen Begriffen funktioniert das erstaunlich gut. Bei einer Fachdomäne mit hauseigenen Begriffen, Abkürzungen und Sonderregeln leider nicht. Dann halluziniert das Modell die fehlende Bedeutung, statt nachzufragen *[2]*.

2.2 Problemklasse 2: das Wissen ist nicht formalisiert

Die zweite Klasse betrifft Wissen, das in einer noch so guten OpenAPI-Spec nicht steht, weil das Format dafür nicht gemacht ist *[2]*. Wir unterscheiden drei Ausprägungen.

- **Fachlicher und situativer Kontext** beantwortet, was ein Begriff in dieser Domäne bedeutet und was im konkreten Fall gilt.
- **Reihenfolgen und Zustandsübergänge** beantworten, welcher Schritt auf welchen folgen darf. Nicht jeder Übergang ist zu jedem Zeitpunkt zulässig, und die Zulässigkeit hängt vom aktuellen Zustand ab. In der Spec stehen die Operationen einzeln nebeneinander, ohne Ordnung. REST ist ressourcenorientiert konzipiert, der Ablauf war immer Aufgabe des Clients [3, Kap. 5.1.3]. Bisher hat ihn ein Mensch implementiert. Jetzt soll ihn ein Agent zur Laufzeit erschließen.
- **Abschlusskriterien und Fehlersemantik** beantworten, woran der Agent erkennt, dass er fertig ist, und was er tun soll, wenn er es nicht ist. An beidem ist unser Agent gescheitert. 201 Created war die richtige Antwort auf "Rückgabe angenommen" und die falsche auf "Auftrag erledigt?". Ein `400 Bad Request` mit dem Text `invalid request` wiederum sagt nicht, ob das Format falsch war, der Übergang unzulässig oder nur der vorherige Schritt noch offen.

Für eine systematische Liste prüfbarer Kriterien eignet sich das Jentic API AI-Readiness Framework (JAIRF), das statische Signale einer OpenAPI-Spec in sechs Dimensionen bündelt *[4]*.

Hinter beiden Problemklassen stehen dieselben drei Arten von Wissen *(siehe Abbildung 1)*. *Schnittstellen- und Protokollwissen* beantwortet, welche Operationen es gibt und wie man sie aufruft. *Domänenwissen* beantwortet, was eine Operation fachlich bedeutet, und welche Regeln gelten. *Interaktionswissen* beantwortet, in welcher Reihenfolge Aufrufe erfolgen müssen. Der Fokus liegt hier auf Wissen, das beim Entwurf einer API entsteht - Sicherheit, Betrieb und Lebenszyklus lassen wir bewusst außen vor.

Die Problemklassen lassen sich dabei nicht eindeutig einer Wissensart zuordnen, eher lässt sich sagen, dass Problemklasse 1 Wissen betrifft, für das in der OpenAPI-Spec ein Feld vorgesehen ist. Problemklasse 2 betrifft hingegen Wissen, für das kein entsprechendes Feld existiert. Deshalb gehört zum Beispiel „Verständlichkeit" zu Problemklasse 1, obwohl fachliche Bedeutung dahintersteht.

Die drei Arten überschneiden sich zudem, und das Wissen, an dem ein Auftrag scheitert, liegt meist in einer Schnittmenge *(siehe Abbildung 1)*.

2.3 Warum das Menschen genauso betrifft

Kein einziger dieser Punkte ist ein ausschließliches KI-Problem. Entwickler*innen, die zum ersten Mal gegen diese API arbeiten, stolpern über dieselbe veraltete Beschreibung, dasselbe undokumentierte Statusfeld, denselben nichtssagenden 400er. Der Unterschied ist nicht die Art des Problems, sondern was danach passiert. Sie fragen im Fachbereich nach, lesen den Quellcode oder probieren es im Testsystem, und sie merken, wenn ihnen etwas komisch vorkommt. Agenten verfügen weder über diese Möglichkeiten noch über dieses Gespür.

KI-Agenten sind damit weniger ein neues Problem als ein sehr unbestechlicher Prüfstand für ein altes Problem. Die gute Nachricht lautet: Fast alles, was jetzt folgt, ist Arbeit, die sich ohnehin lohnt.

3 Problemklasse 1: API-Qualität und Governance

Der wirksamste Schritt ist es, die eigene OpenAPI-Spec in Ordnung zu bringen, indem wir die vier Dimensionen aus der ersten Problemklasse als Anforderungen an uns selbst stellen. Das Ergebnis ist eine qualitativ hochwertige Spec, welche die gesam-

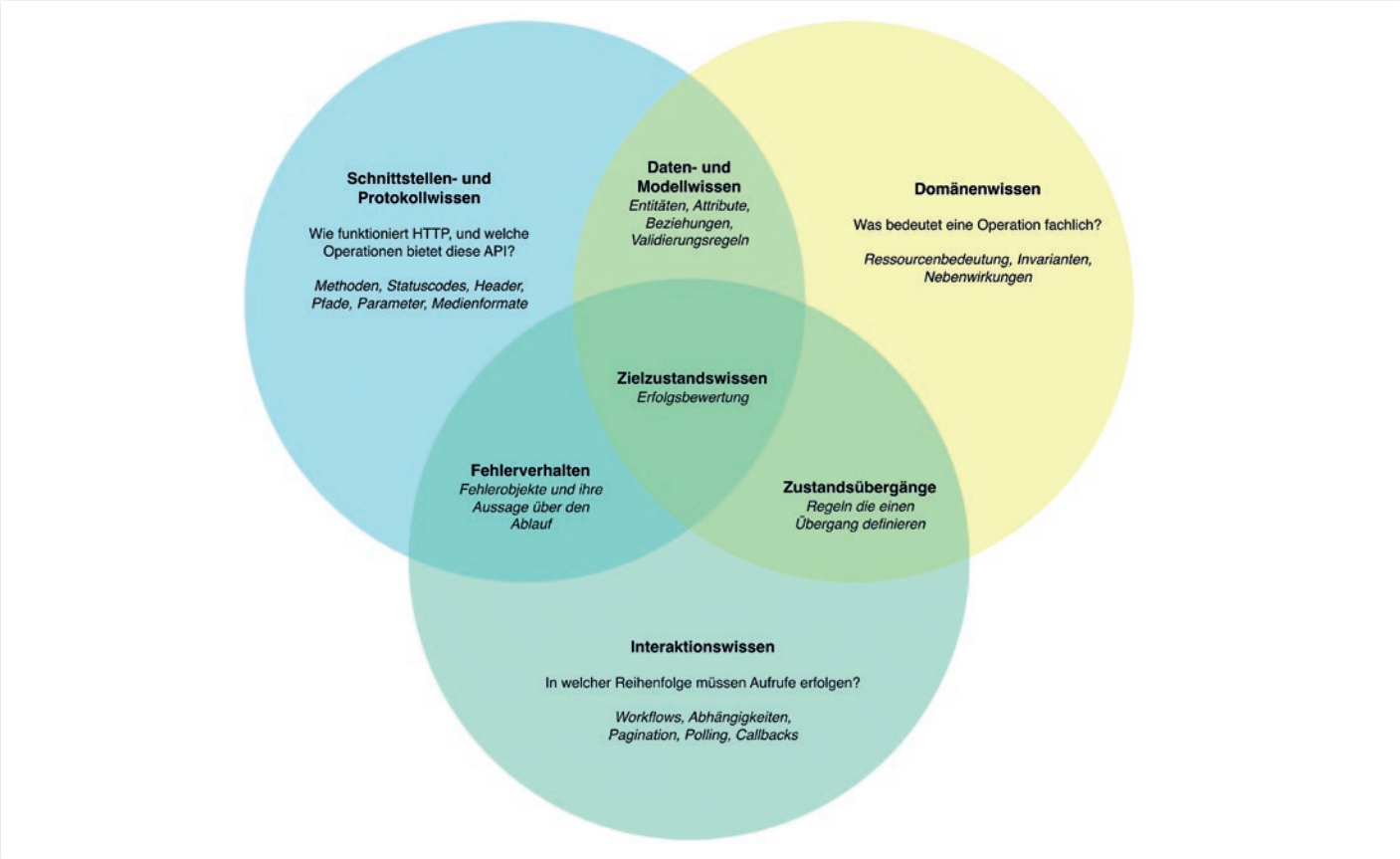


Abbildung 1: Arten von Wissen im API-Design-Kontext (© codecentric AG)

```yaml
# vorher
operationId: createRtn
summary: Create rtn
# ... und im Request Body:
condition:
  type: string

# nachher
operationId: accept-equipment-return
summary: Nimmt die Rückgabe eines ausgegebenen Exemplars an
description: >
  Nimmt ein ausgegebenes Exemplar zurück und entscheidet anhand des Rückgabeprofils des Gerätetyps, der Nutzungsart und des gemeldeten Zustands über den Folgezustand. Das Exemplar wird entweder direkt wieder verfügbar, geht in Prüfung (dann entsteht ein eigener Prüfauftrag, über den ein Prüfer separat entscheidet) oder wird gesperrt. Eine angenommene Rückgabe bedeutet nicht, dass das Exemplar wieder ausgegeben werden darf.
# ... und im Request Body:
condition:
  type: string
  enum: [unauffaellig, auffaellig, sicherheitskritisch]
```

Listing 1: Dieselbe Operation ohne und mit fachlicher Beschreibung und Wertebereich

te API abbildet, den HTTP-Standard einhält und die Operationen fachlich erklärt.

3.1 Vollständig, aktuell, verständlich, standardkonform

Eine der vier Dimensionen, nämlich die **Aktualität**, fällt dabei aus dem Rahmen dieses Kapitels: Ob die Beschreibung dem entspricht, was deployed ist, kann kein Linter beantworten, denn er liest die Spec, nicht die laufende API. Stattdessen kann man Aktualität, etwa durch Generierung aus dem Code oder durch Contract-Tests *[5]*, die die Beschreibung mit einer laufenden Instanz abgleichen, sicherstellen. Mit beiden Ansätzen wird zugleich auch die **Vollständigkeit** verbessert.

Die **Verständlichkeit** kostet am meisten Disziplin, bringt aber auch am meisten. *Listing 1* zeigt denselben Endpunkt zweimal.

Die zweite Fassung transportiert das Rückgabeprofil, die drei möglichen Ausgänge und im letzten Satz die Warnung, dass "angenommen" nicht "freigegeben" heißt. Sie tut das an der Stelle, an der ein Consumer ohnehin nachsieht. Und die zulässigen Werte für `condition` stehen jetzt im Schema statt in einem Wiki-Artikel. Damit weiß unser Agent, dass er noch nicht fertig ist. Was er stattdessen tun muss, steht hier nicht, aber dazu kommen wir später noch.

3.2 Aus Ansprüchen werden Regeln: Governance, JAIRF und Linting

Anforderungen, die nur in einem Wiki dokumentiert sind, geraten schnell in Vergessenheit. Die Lösung: Anforderungen benennen und sie automatisiert prüfen.

Als externer Maßstab prüft das JAIRF aus Abschnitt 2.2 etwa das Vorhandensein von Beschreibungen, dokumentierte Fehlerantworten, wiederverwendete Komponenten und aggregiert sie zu einem Score *[4]*. Dieser bewertet dabei nur die Dokumentenqualität und -konsistenz, nicht die Interaktionsinformationen der API. Ein hoher Wert belegt weder, dass die API tut, was sie beschreibt, noch dass fachliche Regeln serverseitig durchgesetzt werden, noch dass ein Agent seinen Auftrag erledigen würde. Er ist eine Orientierung, keine Reifestufe.

Mindestens so wertvoll wie der Score ist die grundlegende, frei verfügbare Liste typischer Versäumnisse.

Im zweiten Schritt formulieren wir Anforderungen aus dieser Liste als Linting-Regeln und integrieren sie in die Pipeline. Zum Beispiel mit Redocly CLI, das OpenAPI-Specs gegen solche Regeln prüft und in der CI sowie IDE läuft *[6]*. *Listing 2* zeigt die Regel für unsere Anforderung aus 3.1.

Eine Mindestlänge ist nur ein grober Proxy für Verständlichkeit, da sie nur das Vorhandensein, aber nicht den Inhalt bewertet. Das ist der typische Zuschnitt solcher Regeln: Sie fangen das Vergessen ab, nicht die Nachlässigkeit. Für alles darüber hinaus bleibt das Review unerlässlich.

3.3 Serverseitige Invarianten und semantisch präzise Fehler

Bisher ging es um die Beschreibung als Dokument. Zur Laufzeit entscheiden zwei weitere Aspekte darüber, ob die API einen unzulässigen Aufruf überhaupt bemerkt, und was sie dem Consumer darüber sagt. Das Bemerken ist die ältere Pflicht, und sie ändert sich durch Agenten nicht. Was fachlich nicht passieren darf, muss der Server verhindern. Denn Kontext beeinflusst, was ein Consumer versucht, nicht, was die API zulässt. Erst diese Prüfungen erzeugen die Fehler, um die es jetzt geht.

Denn die Bedeutung der Fehlermeldung verschiebt sich. Bei Webanwendungen wird ein ausgegrauter Button oft vom Frontend erklärt, während die API knapp antwortet. Agenten sehen nur die Fehlerantwort und können keine Rückschlüsse vom Frontend ziehen. *Listing 3* zeigt beide Varianten.

Die zweite Antwort nennt den aktuellen Zustand, den Grund und den nächsten sinnvollen Schritt. Sie folgt mit `application/problem+json` einem etablierten Format *[7]*, das Agenten wie Menschen maschinell auswerten können. Die Information, warum wir ablehnen, hat der Server in dem Moment ohnehin vorliegen, was den Umsetzungsaufwand überschaubar hält.

Wie weit man das treibt, ist eine Abwägung. Den aktuellen Zustand mitzugeben, kostet fast nichts. Eine gepflegte Liste erlaubter Folgeschritte kostet an jeder Fehlerstelle echte Arbeit und kann still veralten.

4 Problemklasse 2: Kontext und Ablaufwissen zum Agenten bringen

Angenommen, unsere Beschreibung ist jetzt vollständig, aktuell und verständlich, die API erzwingt ihre Regeln und antwortet mit präzisen Fehlern. Unser Agent weiß trotzdem nicht, dass er nach der Rückgabe den Prüfauftrag nachverfolgen muss, bevor er „erledigt" meldet. Dieses Wissen ist nicht Teil der Schnittstelle. Es ist Teil der Domäne.

```
# redocly.yaml
extends:
  - recommended
rules:
  # eigene Regel: die vorhandene Beschreibung muss auch etwas erklären
  rule/operation-description-explains-outcome:
     subject:
        type: Operation
        property: description
        filterInParentKeys: [post, put, patch, delete]
     assertions:
        minLength: 80
     severity: error
     message: description ist zu kurz für eine fachliche Erklärung
```

Listing 2: Eine Governance-Anforderung als automatisiert prüfbarer Regel

```
HTTP/1.1 400 Bad Request
Content-Type: application/json

{ "error": "invalid request" }

------------------------------

HTTP/1.1 409 Conflict
Content-Type: application/problem+json

{
  "type": "https://api.example.org/problems/invalid-state-transition",
  "title": "Unzulässiger Zustandsübergang",
  "status": 409,
  "detail": "EX-0815 steht in Prüfung. Eine angenommene Rückgabe gibt das Exemplar nicht frei.",
  "instance": "/items/EX-0815",
  "currentState": "in_pruefung",
  "allowedNextSteps": ["GET /inspection-orders?item=EX-0815"]
}
```

Listing 3: Generische Fehlerantwort gegenüber einer semantisch präzisen Antwort nach RFC 9457

Damit sind wir bei der Frage, welches Wissen ein Agent für einen konkreten Auftrag zusätzlich braucht und in welcher Form er es bekommt. Für diese Frage hat sich der Begriff Context-Engineering etabliert. Ansätze dafür gibt es viele. Wir greifen zwei heraus, die die Spannweite markieren: den formlosen Fließtext und die schemagebundene Spezifikation. Jede Form hat ihren Nutzen und ihre Grenzen, die wir zu jeder einzeln benennen und in Kapitel 6 nebeneinanderlegen.

4.1 Markdown: der formlose Allrounder

Der einfachste Weg ist ein Fließtext, der erklärt, was die OpenAPI-Spec der API nicht erklärt. Markdown steht hier für jede formlose, flexible Kontextbereitstellung zum Beispiel als Datei im Repository, Doku im Entwicklerportal oder gezielt anforderbare Ressource für Agenten.

Die Stärke ist die Formlosigkeit. Begriffe, Sonderfälle, Gepflogenheiten, Abschlusskriterien: In den Fließtext gehört, wofür es kein Schema gibt. Und Sprachmodelle sind auf genau diese Textsorte trainiert. Besonders wertvoll wird es dort, wo eine Domäne eigene Begriffe hat. Wenn in unserem Gerätepool eine Ausgabe intern „AGA" heißt, dann steht dieses Kürzel in keinen Trainingsdaten. Ein Modell, das ihm begegnet, hat drei Möglichkeiten: nachfragen, ignorieren oder raten. Ein Glossareintrag kostet zwei Zeilen und nimmt die Frage aus dem Spiel.

Die Grenze ist dieselbe Formlosigkeit. Ein Fließtext ist nicht deterministisch validierbar, nicht ausführbar und nicht statisch auf Aktualität prüfbar. Er kann veralten, ohne dass es jemand merkt und dann wirkt er schlimmer als sein Fehlen, weil er Vertrauen genießt.

4.2 Arazzo: die API-nahe Spezifikation für Abläufe

Gegenüber steht Arazzo als ein schematisiertes Format der OpenAPI Initiative, die beschreibt, wie mehrere API-Aufrufe zu einem Workflow zusammengehören *[8]*. Also welche Schritte in welcher Reihenfolge, wie Informationen übertragen werden, und woran man erkennt, dass ein Schritt erfolgreich war. Wenn die OpenAPI-Spec das Wörterbuch ist, ist Arazzo die Grammatik.

Arazzo adressiert damit genau eine Ausprägung aus Problemklasse 2: Reihenfolgen und Zustandsübergänge. Für unsere Geräterückgabe bedeutet das drei Schritte: Rückgabe annehmen, Folgezustand auswerten und bei Bedarf den Prüfauftrag nachverfolgen. Sie stehen als ein benanntes, maschinenlesbares Artefakt beieinander, samt der Erfolgsbedingung, die aus „angenommen" erst „abgeschlossen" macht.

Mit einem Markdown-Dokument hat Arazzo einiges gemeinsam, denn beides liegt versioniert im Repository, wird im Review gelesen und ist an kein bestimmtes Agentenwerkzeug gebunden. Der Unterschied, auf den es ankommt, ist die Prüfbarkeit. Arazzo hat ein Schema und lässt sich deshalb, wie die OpenAPI-Beschreibung, automatisiert gegen dieselben Governance-Regeln prüfen, zum Beispiel mit Redocly CLI aus Kapitel 3. Bei Markdown endet die maschinelle Prüfung meist auf der Ebene des Frontmatters: Titel, Version und Gültigkeitsdatum lassen sich erzwingen, der Inhalt darunter nicht.

Die Grenze liegt dort, wo Fachlichkeit anfängt. Arazzo beschreibt Workflows zwischen API-Aufrufen, nicht die Regeln dahinter. Warum ein bestimmter Gerätetyp geprüft werden muss, steht nicht darin und gehört auch nicht hinein. Und das Format ist bewusst deklarativ: Es kennt kein dediziertes Schleifenkonstrukt und keinen mutierbaren Zustand. Ein bedingter Rücksprung lässt sich bauen, cursorgetriebene Paginierung damit auch — aber ohne Zähler kein gezähltes `forEach`, ohne akkumulierenden Zustand keine Aggregation über mehrere Aufrufe, und Werte wie Zeitstempel oder UUIDs entstehen nicht im Workflow, sondern kommen über Eingabeparameter. Daran ändert auch die neue Version 1.1 nichts, weil sie vor allem ereignisgetriebene APIs, Abhängigkeiten zwischen Workflows und feinere Datenauswahl hinzufügt.

Das größte Versprechen von Arazzo ist jedoch noch offen. Ein Arazzo-Dokument ist nicht nur lesbar, es ist potenziell ausführbar: Ein Runner nimmt die Datei und arbeitet die Schritte selbst ab, statt sie einem LLM oder Menschen zum Nachbauen vorzulegen. Der Workflow wäre dann kein Kontext mehr, den ein Agent interpretiert, sondern deterministisches Verhalten, das er aufruft. Verlässlich verfügbar ist das nach unseren Erfahrungen noch nicht, denn die verfügbaren Runner führen 1.0 aus, nicht 1.1, und hatten in Praxistests einige Kinderkrankheiten. Auch wenn die Toollandschaft wächst, ist sie derzeit nicht mit dem Umfang von OpenAPI zu vergleichen.

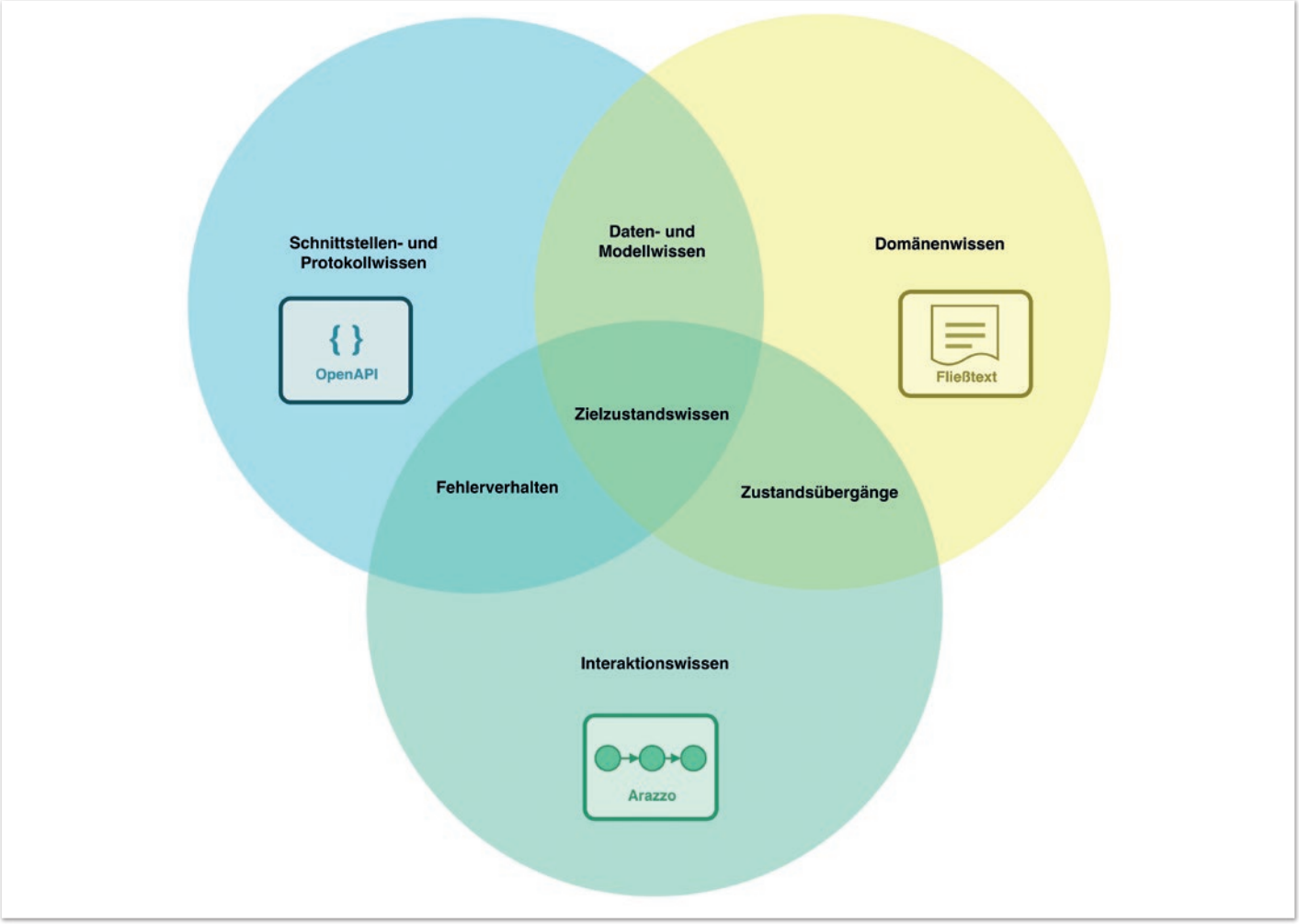


Abbildung 2: Arten von Wissen im API-Design-Kontext mit Beschreibungsformatzuordnung (© codecentric AG)

Damit liegen drei Formate (inklusive OpenAPI) vor, und jedes trägt einen anderen Teil der Wissensarten aus Kapitel 2. Je zwei von ihnen überschneiden sich dort, wo es auch die Wissensarten es tun. Die Mitte erschließt sich dabei aus allen dreien und hat, wie in unserem Fall, das größte Fehlerrisiko *(siehe Abbildung 2)*.

5 Von Kontext zu ausführbaren Capabilities

Beschreibender Kontext leitet Handeln an, garantiert es aber nicht. Eine Capability ist eine aufrufbare Einheit, die einen fachlichen Vorgang ausführt statt einzelner Ressourcenzugriffe [2,9]. Sie kennt die nötigen Schritte, ihre Reihenfolge und den Zustand, in dem der Vorgang endet. In unserem Gerätepool bucht eine Capability "Rückgabe verarbeiten" die Rückgabe, stößt die Prüfung an, wenn der Gerätetyp sie verlangt, und meldet zurück, dass das Exemplar bis zur Freigabe gesperrt bleibt. Der Agent muss dafür weder den zweiten Aufruf kennen noch wissen, dass „angenommen" nicht „abgeschlossen" heißt.

Die Zuständigkeit für Capabilities ist nicht symmetrisch verteilt. Der Provider kennt die Regeln und Invarianten, etwa dass ein Prüfauftrag den Zugriff sperrt. Was er nicht kennt, ist der konkrete Anwendungsfall. Der Consumer weiß, dass Prüfaufträge einmal wöchentlich gesammelt bearbeitet werden und dass die Meldung an die Poolverwaltung wichtiger ist als Geschwindigkeit. Um die Flexibilität zu halten, sollten Capabilities somit nicht ersetzend zu REST-Operationen gesehen werden.

5.1 Wer sollte die Capability bauen?

Beide Seiten können übersetzen, und keine ist pauschal richtig. Providerseitig gebaut, profitiert jeder Consumer davon und die Logik bleibt an einem Ort, erfordert aber, dass der Provider die Anwendungsfälle seiner Consumer vorhersehen muss. Consumerseitig gebaut, passt es genau zum Anwendungsfall; dafür dupliziert es Logik, die schleichend veralten kann. Am Ende ist es eine Architekturentscheidung des Einzelfalls. Für stabile oder kritische Abläufe halten wir eine providerseitige Capability für eine sinnvolle Option, aber keine allgemeine Regel.

5.2 Bereitstellungsform: MCP-Server, Skript, SDK oder CLI

Ist entschieden, was gekapselt wird, ist die Form die kleinere Frage. Dieselbe Capability kann als MCP-Server, als Skript, als SDK oder als CLI angeboten werden *(siehe Abbildung 3)*. Ein MCP-Server ist für Agenten unmittelbar konsumierbar, eine CLI ist es auch, wenn der Agent eine Shell hat, und ein SDK ist die richtige Antwort, wenn am Ende doch Menschen Code schreiben. Wissen, Kapselung und Form liegen damit an drei verschiedenen Stellen, und nur die letzte ist frei wählbar.

5.3 Was Kapselung nicht leistet

Sie ersetzt keinen Kontext, sondern verringert ihn. Eine Capability kann vier Aufrufe samt Reihenfolge kapseln, doch der Agent muss sie korrekt auswählen und ihre fachlichen Eingaben kennen. Weniger Aufrufe bedeuten potenziell vielfältigere Parameter, semanti-

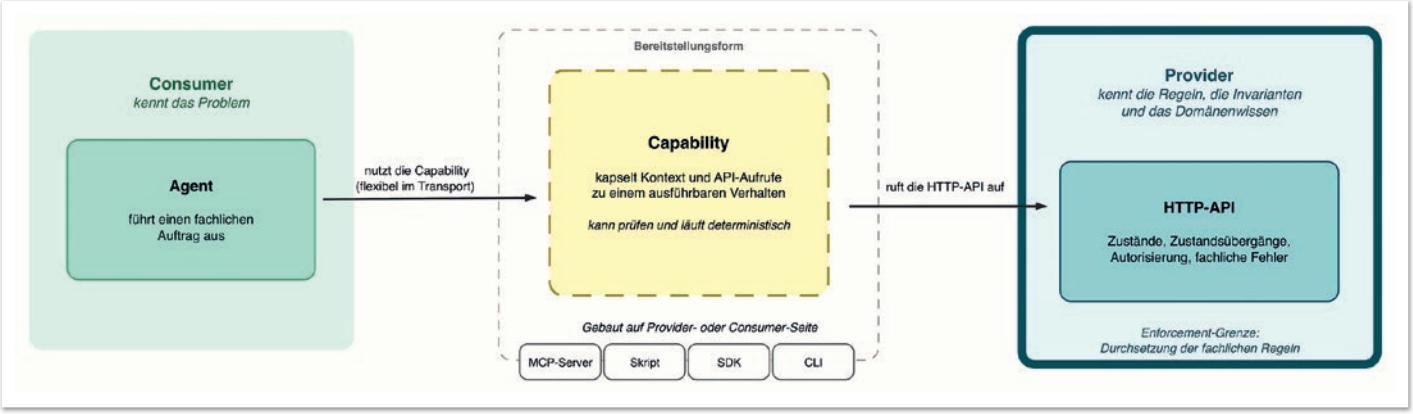


Abbildung 3: Einordnung von Capabilities im Kontext eines LLM-API-Konsumenten (© codecentric AG)

sches Verständnis für die Auswahl bleibt in jedem Fall notwendig. Und Kapselung ersetzt kein Enforcement. Das ist die bekannte Arbeitsteilung aus Abschnitt 3.3: Ein Frontend kann Eingaben prüfen und Schaltflächen sperren, die fachlichen Regeln werden trotzdem im Backend durchgesetzt. Für eine Capability gilt dasselbe. Sie kann Zustände vorab prüfen und Abläufe erzwingen, doch solange die HTTP-API daneben erreichbar ist, bleibt sie eine Bequemlichkeit für kooperative Consumer und keine Grenze.

6 Fazit: erst Problem, Kontext und Zuständigkeit, dann das Tooling

Vier Prinzipien überdauern die Werkzeuge, die wir genannt haben:

1. Hochwertige API und Beschreibung bilden die Basis. Sie helfen allen Consumern und können durch kein Kontextartefakt ersetzt werden.
2. Ein Auftrag braucht zusätzlich fachlichen, situativen und prozeduralen Kontext, der nicht in der Schnittstellenbeschreibung zu finden ist.
3. Form und Zuständigkeit folgen dem Anwendungsfall, nicht umgekehrt.
4. Was fachlich nicht passieren darf, verhindert die API, nicht die Spec, nicht der Prompt, nicht der MCP-Server.

Wer direkt etwas tun möchte, beginn am besten damit, die eigene OpenAPI-Spec gegen die Signale des JAIRF zu halten *[4]*, die drei bis fünf relevantesten Befunde als Governance-Anforderungen zu formulieren und mechanisch prüfbare als Lint-Regeln in die Pipeline zu hängen. Das ist an einem Nachmittag machbar und hilft allen Consumern.

6.1 Eine Landkarte statt eines Siegers

Die nachfolgende Tabelle ordnet die besprochenen Ansätze den Problemen zu: Problem identifizieren, möglichen Beitrag und verbleibende Grenze ablesen. Sie ist keine Rangfolge und kein Entscheidungsbaum.

6.2 Grenzen und Ausblick

Zum Artikel gehört auch, was wir nicht zeigen. Ansätze wurden eingeordnet und mit Beispielen verdeutlicht, aber nicht empirisch verglichen. Ein Teil der Aussagen braucht das auch nicht: Dass eine Arazzo-Beschreibung ein Schema hat und sich damit gegen Regeln prüfen lässt, ein Fließtext dagegen nicht, folgt aus den Formaten selbst. Dünn wird es dort, wo es um die Wirkung auf ein Modell geht. Wie viel ein Arazzo-Dokument gegenüber einem gut geschriebenen Markdown-Abschnitt bringt, können wir nicht beziffern. Messungen mit Sprachmodellen sind nichtdeterministisch, kostenintensiv und veralten schnell. Allein während der Arbeit an diesem Artikel hat Arazzo eine neue Version bekommen, die im Tooling noch ungleichmäßig unterstützt wird.

Was sehr große OpenAPI-Specs für Tokenaufwand und Trefferquote bedeuten, fällt unter den Betrieb, den wir in Abschnitt 1.2 ausgeklammert haben.

Nicht ein bestimmtes Tooling macht eine API agententauglich, sondern eine hochwertige API sowie der für den Auftrag nötige Kontext in einer Form, die zum Anwendungsfall passt.

Quellen

[1] Carey, M. (2026): Code Mode: give agents an entire API in 1,000 tokens. Cloudflare Blog, 20.02.2026.

Problemfeld	Möglicher Beitrag	Wesentliche Grenze
Beschreibungsqualität	Governance-Regeln, statische Diagnose und Linting machen Anforderungen sichtbar und prüfbar	Kein Nachweis für Laufzeitverhalten oder fachlich korrekte Abläufe
Fachlicher, situativer Kontext	Formlose Dokumente transportieren Wissen flexibel und modellnah	Nicht deterministisch validierbar, nicht ausführbar, veraltet unbemerkt
Reihenfolgen, Zustandsübergänge	Arazzo beschreibt Workflows formal, versioniert und prüfbar	Fachliche Regeln, gezählte Iteration und Aggregation bleiben außen vor
Stabile oder kritische Abläufe	Kapselung als Capability, bereitgestellt als MCP-Server, Skript, SDK oder CLI	Zuständigkeit und Eignung bleiben anwendungsfallspezifisch
Konsistenz, Fehler, Abschluss	Serverseitige Invarianten und präzise Fehlerantworten	Nicht ersetzbar und bleibt Provider-Verantwortung

https://blog.cloudflare.com/code-mode-mcp (Zugriff 2026-08-13)
[2] Tupe, V., & Thube, S. (2025): AI Agentic Workflows and Enterprise APIs: Adapting API Architectures for the Age of AI Agents. arXiv Preprint arXiv:2502.17443, *https://doi.org/10.48550/arXiv.2502.17443* (Zugriff 2026-08-13)
[3] Fielding, R. T. (2000): *Architectural Styles and the Design of Network-based Software Architectures*. Dissertation, University of California, Irvine, *https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm*
[4] Jentic: *Jentic API AI-Readiness Framework (JAIRF) Specification*, v1.0.0. *https://docs.jentic.com/reference/api-readiness-framework/specification* · Repository: *https://github.com/jentic/api-ai-readiness-framework* (Zugriff 2026-08-10)
[5] Brunelli, P. (2024): Charge your APIs Volume 25: Contract Testing codecentric Knowledge Hub, 02.04.2024. *https://www.codecentric.de/en/knowledge-hub/blog/charge-your-apis-volume-25-contract-testing* (Zugriff 2026-08-05)
[6] Redocly: *Redocly CLI – Rules to describe API standards*. *https://redocly.com/docs/cli/rules* (Zugriff 2026-08-15)
[7] Nottingham, M.; Wilde, E.; Dalal, S. (2023): RFC 9457 — Problem Details for HTTP APIs. IETF, *https://www.rfc-editor.org/rfc/rfc9457.html*
[8] OpenAPI Initiative (2026): *Arazzo Specification 1.1.0*. *https://spec.openapis.org/arazzo/v1.1.0.html* (Zugriff 2026-08-12)
[9] Kocot, D. (2025): From APIs to Capabilities. *https://architecturalbytes.substack.com/p/from-apis-to-capabilities* (Zugriff 2026-08-03)



Benny Burkert
benny.burkert@codecentric.de
https://www.linkedin.com/in/bennyburkert

Benny Burkert ist IT-Consultant bei der codecentric AG mit Hintergrund in der Softwareentwicklung. Sein fachlicher Schwerpunkt liegt seit jeher auf Technologien und Methodiken rund um den API-Lifecycle – von Domain-Driven Design über API-Testing bis hin zu Governance-Themen. In Kundenprojekten hat er unter anderem an einem Entwicklerportal mitgewirkt und ein BDD-Testkonzept für Blackbox-API-Tests entworfen. Im Rahmen seines Masterstudiums hat er neue Standards wie MCP und Arazzo untersucht. Aktuell beschäftigt ihn besonders die Frage, wie KI-Lösungen den API-Lifecycle verändern.



Felix Medam
felix.medam@codecentric.de
https://www.linkedin.com/in/felix-medam

Felix Medam ist seit vier Jahren IT-Consultant und Developer bei der codecentric AG. Ursprünglich im Frontend zu Hause, hat er sich nach mehreren Jahren Ensemble Programming als Full-Stack Allrounder etabliert. Mit besonderer Begeisterung für Infrastruktur- und Backend-Themen rund um APIs. Zuletzt trieb er den Bau und Betrieb eines Developer-Portals voran, das als zentrale Vertriebsplattform von APIs dient.

Java aktuell

NEWSLETTER

Anmeldung

Der Java aktuell Newsletter informiert dich alle vier Wochen über das aktuelle Geschehen in der Java-Welt.

Abonniere ihn kostenfrei und bleibe immer auf dem Laufenden!



https://www.java-aktuell.eu/go/nlabonnieren

oder nach dem Login mit euren Zugangsdaten direkt über euer Profil abonnieren.

10 Tipps für Spring Boot: Weniger Magie, mehr Pragmatismus

Julius Mischok, Mischok GmbH



Spring Boot hat die Geschwindigkeit revolutioniert, mit der Java-Projekte entstehen. Starter bringen passende Kombinationen von Dependencies mit, Autokonfiguration verdrahtet die Anwendungskomponenten und der Embedded Server startet die Anwendung ohne lokal installierten Application Server. Aus wenig eigenem Code entsteht im Handumdrehen ein lauffähiges System. Aber wie gelingen langfristig stabile und wartbare Systeme auf Basis von Spring Boot?

Die Stärke des Spring-Boot-Ökosystems zeigt sich im langfristigen Projektverlauf. Das Framework liefert für zentrale Herausforderungen fertige Bausteine: Konfiguration, Datenzugriff, Security, Tests, Monitoring, externe Schnittstellen. Teams müssen diese Lösungen nicht selbst bauen. Sie müssen sie kennen, richtig kombinieren und integrieren.

Genau darin liegt die Herausforderung: Die Flexibilität von Spring Boot bietet selten nur einen möglichen Weg. Die gewählte Lösung kann heute schnell funktionieren und morgen Wartung, Tests oder Betrieb erschweren. Eine andere Lösung kostet in der Erstellung mehr Nachdenken, spart dann später Debugging, Sonderfälle und eigenen Infrastrukturcode.

Dieser Artikel zeigt zehn Spring-Boot-Tipps aus inzwischen zwölf Jahren Projektpraxis. Der rote Faden ist testgestützte Entwicklung: stabile Tests, klare Grenzen, weniger Zufall. Keine Magie, sondern pragmatisches Software Engineering.

Tipp 1: @Transactional in Tests verwenden

Ein typisches Szenario: Ein neuer Test erweitert eine bestehende Testsuite, plötzlich schlägt ein komplett anderer Testfall fehl.

In datenbanknahen Tests entscheidet nicht nur der Produktionscode über grün oder rot. Auch der Zustand der Testdatenbank wird Teil des Testergebnisses. Genau dort entstehen schwer nachvollziehbare Fehler: Ein Test legt Daten an, der nächste Test findet sie vor, obwohl sie nicht zu diesem Szenario gehören. Ändert sich zum Beispiel durch Hinzufügen oder Umbenennen einer Testklasse die Ausführungsreihenfolge, bleiben plötzlich Daten über, die einen komplett anderen Test ins Straucheln bringen.

`@Transactional` setzt an dieser Stelle an. Wird eine Testklasse entsprechend annotiert, führt Spring jede Testmethode in einer eigenen Transaktion aus. Nach dem Test rollt Spring diese Transaktion zurück. Inserts, Updates und Deletes verschwinden wieder aus der Testdatenbank. Unabhängig davon, ob der Test erfolgreich war oder fehlschlug.

Damit bleibt der Datenbankzustand zwischen den Tests kontrolliert. Die Testmethoden führen echte Datenbankzugriffe aus, ohne nachfolgende Tests zu beeinflussen. Das spart Cleanup-Code und stabilisiert die CI/CD-Pipeline.

Warnung

Der offene Transaktionskontext kann Fehler verdecken, die erst zur Laufzeit auftreten. Das klassische Beispiel ist die `LazyInitializationException`. Im Test funktioniert der Zugriff auf lazy geladene Beziehungen, weil die Transaktion aktiv ist. Im echten Aufrufpfad kann derselbe Zugriff bereits außerhalb einer Transaktion liegen.

Einige bewusst nicht transaktionale Tests prüfen dieses Verhalten und transaktionale Anforderungen explizit. Der Großteil der datenbanknahen Tests erhält aber die `@Transactional`-Annotation und gewährleistet dadurch eine vollständige Isolation der Testfälle.

Tipp 2: Flyway und `@Sql` kombinieren

Mit `@Transactional` verschwinden Datenänderungen nach jedem Test. Das löst aber nicht die Frage, welche Daten zu Beginn eines Tests vorhanden sein sollen.

Hier lohnt sich eine klare Trennung: Flyway oder ein anderes Datenbankmigrationstool erstellt das Schema und die Basiskonfiguration der Anwendung. Dazu gehören Tabellen, Constraints, Rollen oder technische Startwerte, ohne die die Anwendung nicht sinnvoll läuft. Diese Skripte laufen beim Start des Spring Contexts und damit außerhalb der Testtransaktion.

`@Sql` bereitet dagegen das konkrete Testszenario vor. Skripte erstellen Kundendaten, einen Warenkorb, offene Rechnungen oder einen Benutzer mit bestimmten Rechten unmittelbar am Testfall. Diese Daten werden zusammen mit der Testtransaktion zurückgerollt. *Listing 1* zeigt die Verwendung der Annotation.

```
@SpringBootTest
@Transactional
@SqlMergeMode(MERGE)
@Sql("classpath:/data-init.sql")
public class PersonRepositoryTest {

      @Test
      @Sql("classpath:/scenarios/001-active-users.sql")
      void findsActivePersons() {
            // ...
      }
}
```

Listing 1: Dateninitialisierung mit der @Sql Annotation

Der Gewinn liegt in der Zuständigkeit: Flyway beschreibt den stabilen Zustand der Anwendung. `@Sql` beschreibt das Szenario des Tests. Dadurch bleibt nachvollziehbar, welche Daten fachlich zur Anwendung gehören und welche Daten nur einen Test vorbereiten.

Das Repository unter *[1]* enthält im Branch „strategy/flyway-and-testscripts“ ein vollständiges Beispiel.

Warnung

Je weiter Testdaten vom Test entfernt liegen, desto teurer wird die Fehlersuche. Große SQL-Skripte schaffen Ordnung im Repository,

aber nicht automatisch Klarheit im Test. Die Kombination aus testglobalen Skripten auf Klassenebene und auf Methodenebene annotierten Skripten ist hilfreich. Sie wird, wie in *Listing 1* gezeigt, durch die Annotation `@SqlMergeMode` aktiviert.

Tipp 3: Testcontainers verwenden

Embedded Databases machen die lokale Entwicklung schlank und schnell. Eine zusätzliche Dependency reicht, und die Tests laufen ohne lokal installierte Datenbank. Für einfache Tests ist das praktisch. Für belastbare Integrationstests reicht es nicht immer.

Der Grund ist simpel: H2 ist nicht PostgreSQL, MySQL oder Oracle. SQL-Dialekte, Datentypen, Constraints und Transaktionsverhalten unterscheiden sich. Damit entsteht eine Lücke zwischen Testumgebung und späterem Betrieb.

Testcontainers schließt diese Lücke. Der Test startet die benötigte Infrastruktur als Docker-Container durch den Code aus *Listing 2*. Die Anwendung spricht im Test mit derselben Datenbanktechnologie wie später im Betrieb.

`@DynamicPropertySource` überträgt die dynamisch erzeugten Verbindungsdaten des Containers in die Spring-Konfiguration. Genau dort liegt der eigentliche Trick: Der Test startet nicht nur PostgreSQL, sondern verbindet die Anwendung auch mit dieser Testinstanz. Aktuelle Spring-Boot-Versionen bieten mit `@ServiceConnection` eine noch kompaktere Möglichkeit. Die Containerinstanz überträgt ihre Konfiguration damit direkt in den Spring Kontext. Dafür muss die Dependency `spring-boot-testcontainers` als Testabhängigkeit eingebunden sein.

Warnung

Spring Boot benötigt nur ein Java Development Kit auf der lokalen Maschine. Testcontainers brauchen Docker. Das betrifft lokale Entwicklungsmaschinen und CI Runner.

Tipp 4: Mockito für Innen-Mocking verwenden

Ein automatisierter Test darf sein Verhalten nur ändern, wenn sich der getestete Produktionscode ändert. Schlägt ein Test wegen Netzwerkproblemen, geänderten Testdaten einer Partner-API oder einem nicht erreichbaren Fremdsystem fehl, prüft er nicht mehr isoliert die Anwendung.

Wie *Abbildung 1* zeigt, löst Mockito diese Problemstellung innerhalb des Spring Contexts. Die Fachlogik läuft weiter durch die Anwendung. Die Adapter-Bean für den externen Zugriff ersetzt der Test durch einen kontrollierten Mock.

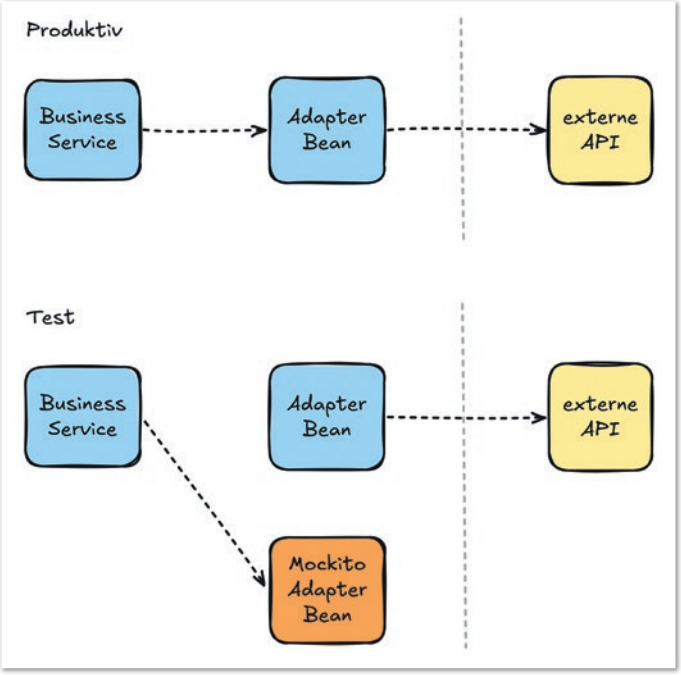


Abbildung 1: Mockito ersetzt die Adapter-Bean innerhalb der Anwendung (© Julius Mischok)

Spring ersetzt eine per `@MockitoBean` injizierte Bean im Testkontext durch eine leere Hülle. Alle verwendeten Methoden des Mocks müssen als expliziter Stub angelegt werden, wie in *Listing 3*. Ein ausführliches Beispiel befindet sich im Repository *[2]* im Branch „feature/mockito".

Einen anderen Ansatz verfolgt @MockitoSpyBean: Damit injizierte Beans verwenden standardmäßig die Implementierung der Bean, erlauben aber das gezielte Stubben einzelner Methoden.

Warnung

Mockito koppelt den Test an Methodensignaturen des Produktionscodes. Die Stubs müssen fortlaufend gepflegt werden.

```
@SpringBootTest
@Testcontainers
class PostgresIntegrationTest {

      @Container
      static PostgreSQLContainer<?> postgres =
                  new PostgreSQLContainer<>("postgres:16.2")
                              .withDatabaseName("testdb")
                              .withUsername("testuser")
                              .withPassword("testpass");

      @DynamicPropertySource
      static void configureProperties(DynamicPropertyRegistry registry) {
            registry.add("spring.datasource.url", postgres::getJdbcUrl);
            registry.add("spring.datasource.username", postgres::getUsername);
            registry.add("spring.datasource.password", postgres::getPassword);
      }
}
```

Listing 2

```
@SpringBootTest
@AutoConfigureMockMvc
class CityServiceTest {

  @MockitoBean
  OpenMeteoService openMeteoService;

  @BeforeEach
  void setup() {
     when(openMeteoService.getWeatherForLocation(48.3705, 10.8960))
     .thenReturn(weatherResponse());
  }
}
```

Listing 3

In früheren Versionen hießen die Annotationen @MockBean und @SpyBean, die Funktionsweise war aber gleich.

Tipp 5: WireMock für Außen-Mocking verwenden

Mockito löst das Problem innerhalb der Anwendung. Warum nicht noch einen Schritt nach außen gehen? Der eigene Code soll den echten HTTP-Client verwenden, die echte URL bauen, Header setzen und die Antwort serialisieren. Nur der externe Server soll nicht Teil des Tests sein.

WireMock setzt genau an dieser Stelle an. Der Test startet einen lokalen HTTP-Server und definiert dessen Antworten. Wie *Abbildung 2* zeigt, ruft die Anwendung weiterhin eine HTTP-Schnittstelle auf, landet aber nicht beim echten Fremdsystem, sondern bei WireMock.

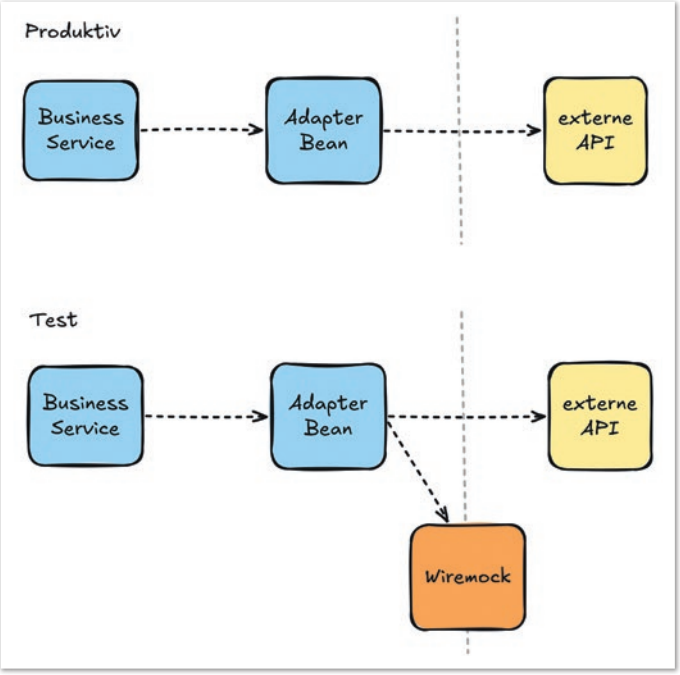


Abbildung 2: WireMock simuliert die externe HTTP-Schnittstelle außerhalb der Anwendung (© Julius Mischok)

Der Unterschied zu Mockito ist fundamental: WireMock ersetzt keine Spring Bean. Der Adapter-Code wird im Test mit ausgeführt. Dadurch prüft der Test auch URLs, Request-Parameter, Header, Statuscodes und JSON-Payloads. Das bringt den Test näher an die echte Kommunikation. *Listing 4* zeigt die konkrete Umsetzung des Stubbings, das Repository *[2]* enthält im Branch „feature/wiremock" ein ausführliches Beispiel.

Warnung

Wie bei Mockito: WireMock-Stubs erfordern fortlaufende Wartung. Ändert die externe API ihren Vertrag, bleiben alte Stubs trotzdem grün. Integrationstests oder ein Abgleich gegen eine OpenAPI-Spezifikation sichern die Korrektheit der Stubs ab.

Tipp 6: Konfiguration auslagern

Konfiguration gehört nicht in den Produktionscode. Keine hart codierte API-URL, kein Passwort im Repository, keine `if`-Abfrage auf den Servernamen. Was sich zwischen lokalem Start, Testsystem und Produktion unterscheidet, gehört in die Umgebungskonfiguration.

Spring Boot liefert dafür das Konzept der Externalized Configuration. Ein Wert kann aus Properties, YAML, Umgebungsvariablen, Kommandozeilenparametern oder Test-Overrides kommen. Die Reihenfolge ist definiert; spätere Quellen überschreiben frühere Quellen. Tatsächlich stellt das Framework 15 Möglichkeiten zur Verfügung, Konfigurationswerte in die Anwendung zu bringen. Eine gute Übersicht über die Hierarchie bietet *[3]*.

Die Anbindung von WireMock ist dadurch sehr elegant. *Listing 5* zeigt die Konfiguration des `RestClients` im Produktionscode. Bei der Initialisierung der Bean injiziert das Framework den Konfigurationsparameter in die Variable baseUrl. *Listing 6* enthält dann die Überschreibung im konkreten WireMock-Test durch die Annotation auf Klassenebene. Die Quelle `@TestPropertySource` hat eine höhere Priorität als die `application.properties` und überschreibt deren Wert.

```
@BeforeEach
void setup() {
  stubFor(get(urlEqualTo(
     "/v1/forecast?latitude=48.3705&longitude=10.8960"
     )).willReturn(okJson("""
        {
           "latitude": 48.3705,
           "longitude": 10.8960,
           "current_weather": { "temperature": 15.0 }
        }
     """)));
}
```

Listing 4

Warnung

Die Flexibilität hat einen Preis: Ein falscher Wert kann aus mehreren Quellen stammen. Raten kostet Zeit. Actuator-Endpunkte wie `env` und `configprops` zeigen, welche Werte im laufenden System angekommen sind.

Tipp 7: Spring Profile einsetzen

Konfiguration auszulagern löst nicht jedes Problem. Mit wachsender Anwendung entstehen sinnvolle Konfigurationsvarianten: lokale Entwicklung, automatisierte Tests, Staging, Produktion. Spring Profile gruppieren diese Varianten und machen sie explizit.

Der bekannteste Weg führt über profilabhängige Dateien. `application.properties` enthält Defaults. `application-test.properties` oder `application-prod.properties` überschreiben gezielt die Werte für das aktive Profil. In Tests setzt `@ActiveProfiles` das gewünschte Profil direkt an der Testklasse. Profile steuern aber nicht nur Properties. Mit `@Profile` lädt Spring Beans abhängig vom aktiven Profil in den Application Context.

Das ist praktisch für Entwicklungsdaten, lokale Adapter oder bewusst abweichende Infrastruktur. Spring Boot unterstützt auch die Negierung von Profilen. Das Framework lädt beispielsweise eine mit `@Profile("!dev")` annotierte Bean nur dann in den Kontext, wenn das Profil `dev` nicht aktiv ist.

Warnung

Spring erlaubt die gleichzeitige Aktivierung mehrerer Profile. Damit entstehen schnell Kombinationen, die niemand getestet hat. Jeder Profilwechsel kann außerdem einen neuen Spring Context im Test erzwingen und die Laufzeit der Suite erhöhen.

Tipp 8: Spring Security und @WithMockUser kombinieren

Security sollte nicht selbst implementiert werden. Login, Passwortprüfung, Rollen, Rechte und Session-Verhalten wirken am Anfang überschaubar. Früher oder später rächt sich diese Abkürzung jedoch fast immer.

Spring Security liefert die etablierte Basis für Authentifizierung und Autorisierung in Spring-Anwendungen. Der eigentliche Gewinn liegt nicht nur im Betrieb, sondern auch im Test: Security-Regeln lassen sich prüfen, ohne für jeden Test einen vollständigen Login-Flow aufzubauen.

So setzt `@WithMockUser` für einen Test einen Benutzer in den Security Context. Damit prüft der Test, ob ein Controller, Service oder Endpunkt mit bestimmten Rollen erreichbar ist *(Listing 7)*.

Der Test beschreibt damit präzise die fachliche Frage: Darf ein Benutzer mit dieser Rolle diese Funktion ausführen? Passwortvalidierung, Login-Formular oder OAuth-Flow gehören nicht in jeden Autorisierungstest.

Warnung

`@WithMockUser` testet innerhalb der Spring-Security-Abstraktion. Das ist gewollt und effizient. Einige Integrationstests müssen trotzdem den echten Authentifizierungsweg prüfen. Sonst bleibt unklar, ob Login, Logout, Token-Verarbeitung oder Session-Handling korrekt zusammenspielen.

Tipp 9: Spring Data JPA verwenden

Datenzugriff erzeugt schnell mehr Code als fachlich nötig. Repository-Klassen, Standardqueries, Pagination und Sortierung landen dann als eigene Infrastruktur im Projekt. Der Code funktioniert, aber er löst Probleme, die Spring Data JPA längst gelöst hat.

Die JPA-Adaption im Spring-Ökosystem bietet einen zunächst schlanken und langfristig mächtigen Ansatz. So reicht ein Repository-Interface für CRUD-Zugriffe, Sortierung und Paging. Besonders sticht das Konzept der Query-Methods hervor: Allein aus einem Methodennamen im Repository-Interface generiert das Framework die Implementierung der Datenbankabfrage.

```
@Bean
RestClient restClient(
            RestClient.Builder builder,
            @Value("${meteo.service.baseurl}") String baseUrl) {

     return builder.baseUrl(baseUrl).build();
}
```

Listing 5

```
@TestPropertySource(properties =
            "meteo.service.baseurl=http://localhost:${wiremock.server.port}")
```

Listing 6

```
@Test
@WithMockUser(username = "mail@example.com", roles = "FINANCEVIEW")
void callPage_authorizedUserView() throws Exception {
     // ...
}
```

Listing 7

```
public interface UserRepository extends JpaRepository<User, Long> {

     @EntityGraph(attributePaths = {"roles", "department"})
     Page<User> findByActiveTrue(Pageable pageable);
}
```

Listing 8

Dieses Beispiel liefert aktive Benutzer, unterstützt Paginierung über `Pageable` und lädt Rollen sowie Abteilung per Join direkt mit. Alles ohne eigene Implementierung des Repository-Interfaces.

Ein vollständiges Beispiel liegt unter *[4]*, insbesondere enthält es Beispiele für komplexere Szenarien, die nicht mehr per Query-Method abgehandelt werden können.

Warnung

Spring Data JPA spart Code, aber nicht das Verständnis für Datenbankzugriffe. Unbedacht eingesetzte Beziehungen erzeugen schnell das N+1-Queries-Problem oder zu große Objektgraphen. Wer sich der Gefahren bewusst ist, löst sie mit den Mitteln von JPA auch sehr einfach.

Tipp 10: Caching mit @Cacheable einsetzen

Performanceprobleme entstehen gern an bekannten Stellen: Datenbankzugriffe, externe APIs, aufwendige Berechnungen. Spring Cache bietet auch dafür eine leichtgewichtige Lösung. `@Cacheable` speichert das Ergebnis einer Methode, `@CacheEvict` räumt passende Einträge wieder auf. *Listing 9* verdeutlicht, wie schlank die Einbindung ist. Aktiviert wird das Feature durch `@EnableCaching` an der Application-Klasse.

```
@Service
class EmployeeService {

     @Cacheable("employees")
     Optional<Employee> getEmployee(Long id) {
          return repository.findById(id);
     }

     @CacheEvict(value = "employees", key = "#employee.id")
     Employee saveEmployee(Employee employee) {
          return repository.save(employee);
     }
}
```

Listing 9

Das Repository unter *[4]* enthält das gesamte Codebeispiel, inklusive eines umfangreichen Tests, der die Caching-Funktionalität sicherstellt.

Warnung

Wie `@Transactional` und `@Async` funktioniert `@Cacheable` ohne weitere Konfiguration nur an public Methoden, die aus einer anderen Spring Bean aufgerufen werden. Hier ist beim Refactoring Vorsicht geboten, ein einfacher Caching-Test sichert das Verhalten ab.

Fazit

Spring Boot bietet einen leichten Einstieg. Gute Spring-Boot-Projekte entstehen aber nicht auf dem scheinbar schnellsten Weg, sondern durch die richtige Abkürzung an der richtigen Stelle.

Genau das zieht sich durch die zehn Tipps: Das Framework bietet für zentrale Projektprobleme fertige Mechanismen. Der entscheidende Schritt besteht darin, sie nicht zufällig einzusetzen, sondern bewusst zu kombinieren. Dann reduziert Spring Boot nicht nur Boilerplate, sondern auch Reibung im Projekt: weniger Sonderlösungen, stabilere Tests, klarere Verantwortung.

Der Unterschied zeigt sich im langfristigen Projektverlauf. Eine schnelle Lösung spart heute möglicherweise einige Minuten – eine tragfähige Lösung spart später Debugging, Abstimmung und Betriebsschmerzen. Genau dort entsteht der eigentliche Wert von pragmatischem Software Engineering.

Spring Boot liefert die Werkzeuge. Erfolgreiche Teams setzen sie nachhaltig ein.

Quellen

[1] *https://gitlab.mischok.de/open/persons*
[2] *https://gitlab.mischok.de/open/cityservice*
[3] *https://docs.spring.io/spring-boot/reference/features/external-config.html*
[4] *https://gitlab.mischok.de/open/company-database*



Julius Mischok
Mischok GmbH
julius.mischok@mischok.de

Julius Mischok setzt mit Projektpartnern unverzichtbare Softwareprojekte um. Egal ob Greenfield oder Legacy, Skalierung des Umfangs oder straffer Zeitplan: Das Team der Mischok GmbH versteht das große Ganze, übernimmt Verantwortung und führt das Projekt schrittweise zum Erfolg.

Dank HTMX zu einem Backend-lastigen Frontend

Sebastian Sprenger, Breuninger & Felix Seidel, andrena objects





Wir betreiben unsere Microservices ohne nodejs und ohne Frontend-Build. Unser Backend bringt alles mit, was das Frontend braucht. Der Stack besteht aus Gradle, Java, Spring, Thymeleaf, HTMX, Vanilla CSS und Vanilla JS. Die Schlüsseltechnologie HTMX reduziert die Menge an JavaScript signifikant und verlagert den Fokus auf das Java-Backend mit Spring und Thymeleaf.

Die Multichannel-Plattform bei Breuninger umfasst all die Systeme, die unseren Onlineshop, unsere Backoffices und Apps ausmachen. Dabei handelt es sich um eine Vielzahl an Self-Contained Systems, die im Backend auf Datenreplikation und im Frontend auf Integration durch Transklusion setzen. Wir achten auf die fachlichen Schnitte der Entwicklungsteams, um organisatorisch skalieren zu können und so die Plattform effektiv weiterentwickeln zu können.

In diesem Artikel berichten wir von einem neu gegründeten Team, das mit einer Fachlichkeit startete und anschließend mehrere Microservices von **verschiedenen** Teams übernahm. Die Teams haben neben übergreifend geltenden Makro-Guidelines auch pro Team geltende Mikro-Guidelines, die gewisse Freiheiten beim Einsatz von Technologien gewähren.

Die Frontends der Microservices waren im Kern sehr ähnlich aufgebaut: TypeScript sprach JSON-APIs an, verarbeitete das Ergebnis, entschied was anzuzeigen ist, validierte Inputs, manipulierte den DOM und sendete gegebenenfalls weitere Netzwerkanfragen. Die Entscheidungs- und Validierungslogik wurde mit Jest getestet. Technologisch fanden wir aber auch Unterschiede: Neben TypeScript fanden wir auch reines JavaScript sowie die CSS-Preprocessoren Stylus und SASS und die Build-Tools Rollup, Gulp und Faucet

Microservices mit unterschiedlichen Technologien umzusetzen, kann eine Stärke sein. In unserem Fall erlebten wir dies jedoch als Bottleneck. Wir lösten mit den verschiedenen Technologien keine unterschiedlichen Spezialprobleme, sondern die gleichen Probleme anders. Dies erhöhte die Cognitive Load durch Kontextswitches und Mehrfachimplementierungen ohne Gegenwert. Außerdem schien sich eine Einarbeitung in mehrere Technologien nicht zu lohnen.

In der Praxis hatten wir zudem häufig Schwierigkeiten bei der lokalen Ausführung: Wurde der Frontend-Server gestartet, um die Assets zur Verfügung zu stellen? Wurde der Frontend-Build ausgeführt, bevor der Test gestartet worden ist? Warum ist überhaupt der Frontend-Build gescheitert? Und wenn der Code nicht wie erwartet funktioniert: Steckt der Fehler im Code oder im Vorgehen?

Während hinsichtlich Cognitive Load und Skillaufbau eine Harmonisierung auf **einen** Technologiestack als naheliegende Lösung erscheint, trieb uns die Frustration rund um die lokale Ausführung

weiter: Geht es auch ohne Frontend-Build?

Was steckt in einem Frontend-Build?

In den verschiedenen Builds haben wir im Grunde verschiedene Implementierungen von diesen Aufgaben gefunden:

- TypeScript zu JavaScript Transpilation sowie entsprechendes Testing
- CSS Preprocessing
- Fingerprinting der Static Assets (JavaScript- und CSS-Dateien) sowie die dazugehörige Link Resolution
- Herunterladen von Dependencies und die dazu passende Auflösung von Path Aliasen
- Minify und Bundling
- Linting
- Serving von Dateien

Allesamt valide Punkte. Um ohne Frontend-Build auszukommen, also mit einem Buildless Frontend zu arbeiten, brauchen wir alternative Lösungen.

HTMX als Schlüsseltechnologie

HTMX ist eine JavaScript-Bibliothek, die es erlaubt, ein beliebiges HTML-Element mit Attributen auszustatten, die dazu führen, dass der Browser einen AJAX-Call an das Backend sendet und das HTML-Fragment, das als Antwort kommt, an einer beliebigen Stelle in den DOM integriert.

In dem Beispiel in *Listing 1* sorgt ein Klick (`hx-trigger`) auf den Paragraph mit der ID „launcher" dafür, dass ein POST Request an den Endpunkt `/helloworld` (`hx-post`) gesendet wird. Der Endpunkt gibt ein HTML-Fragment zurück, das das div mit der ID "target" (`hx-target`) ersetzt, da als Swap-Strategie `OuterHTML` angegeben wurde (`hx-swap`).

```
<p id="launcher" hx-trigger="click" hx-post="/helloworld"
  hx-target="#target" hx-swap="OuterHTML">Click me!</
p>
<div id="target"></div>
```

Listing 1: Verwendung von HTMX

Es gibt weitere Werte und Attribute, um das Verhalten von HTMX im Detail zu steuern, die auf der Homepage*[1]* erklärt werden.

In einem Satz: Mit HTMX wird UI-Verhalten durch HTML-Attribute deklariert statt JavaScript imperativ zu schreiben.

Frontend ohne TypeScript führt zu einem Build ohne Transpilation

HTMX ermöglicht es uns, neben dem Rendern der HTML-Struktur auch die damit einhergehende Entscheidungs- und Validierungslogik in das Backend zu verlagern. Wenn der Controller über das passende Template entscheidet, dann können wir die Unit-Tests mit Jest stattdessen mit JUnit und @WebMvcTest schreiben.

Für ein Team mit Stärken im Backend eröffnete sich uns eine spannende Frage: Könnten wir auf selbstgeschriebenes JavaScript verzichten und trotzdem interaktive Webseiten gestalten?

HTMX hilft uns, einen großen Teil der Anzeigethematik in das Backend zu verlagern, aber manche Dinge müssen weiterhin im Frontend passieren: Event Listener für unser Web Tracking, Toggles von Modalen und Passwortsichtbarkeit, spezifischen Formatierungen von Eingabefeldern (zum Beispiel IBAN), sowie das bedingte Hinzufügen oder Entfernen von CSS-Klassen. Statt vieler verteilter Dateien würden wir für jede Seite **eine** korrespondierende JavaScript-Datei haben, welche den spezifischen Code enthalten würde. Einzelne wiederverwendbare Komponenten wie einen Spinner könnten wir per ESM-Imports einbinden.

Wir erwarteten dadurch eine signifikante Reduzierung der Komplexität unserer Codebase und stellten daher die Hypothese auf, dass wir zumindest auf TypeScript und die damit verbundene Transpilation verzichten und gleichzeitig den Code wartbar halten könnten. Vor diesem Hintergrund haben wir uns den Frontend-Build erneut angesehen und überlegt, worauf wir noch verzichten könnten.

Bei der Analyse des Einsatzes der verschiedenen CSS Preprocessors zeigte sich, dass wir keine Features der Preprocessors verwendeten, die nicht nativ in modernem CSS vorhanden waren. Wir haben entschieden, mit nativem CSS zu arbeiten und haben so den zweiten Transpilation-Schritt eliminiert.

Mit der Annahme, dass wir für jede Seite korrespondierende JavaScript- und CSS-Dateien mit spezifischem Code haben würden, erschien uns ein Bundling der Dateien nicht mehr notwendig. Das Tree-Shaking wäre implizit und die Anzahl der wiederverwendbaren Dateien überschaubar. Wir entschieden uns ohne Bundling weiterzugehen.

Eine Minifizierung von Static Assets ist an sich eine gute Idee, um verlustfrei die zu sendende Datenmenge zu reduzieren. Mit den neuen Rahmenbedingungen gingen wir davon aus, dass der Impact gering sein würde, und verschoben die Entscheidung auf später. Eine nachträgliche Messung zeigte, dass wir beim CSS wenige Bytes und beim JavaScript weniger als 4 kb gespart hätten. Das haben wir im Kontext unserer Fachlichkeit als vernachlässigbar bewertet und uns entschlossen, auch auf die Minifizierung zu verzichten.

Mit der reduzierten Menge an Code sahen wir auch wenig Bedarf, dass Linting im Build durchzusetzen. Stattdessen haben wir die Regeln in unseren IDEs definiert und formatieren die Dateien beim Speichern. Das funktioniert für uns gut genug.

Caching & Invalidierung

Wenn eine Webseite eine CSS-Datei namens `style.css` sowie eine JavaScript-Datei namens `feature.js` importiert, dann ist es sinnvoll, diese Dateien zu cachen, damit sie bei weiteren Besuchen der Seite nicht erneut heruntergeladen werden müssen. Gleichzeitig wollen wir durch ein Deployment Änderungen am Cache vorbei wirksam werden lassen. Die gängigste Lösung nennt sich Cache Busting *[2]*: Die URL weist ein Eindeutigkeitsmerkmal auf, das sich bei einer Änderung verändert, sodass der Browser die Datei nicht mehr in seinem lokalen Cache findet und die neue Datei anfordert.

Bei wiederverwendbaren Libraries wie HTMX wird meistens eine Versionsnummer verwendet (zum Beispiel `*https://example.com/*

```
@Configuration
public class MvcConfig implements WebMvcConfigurer {
     @Value("${static.assets.version}")
     private String version;

     @Override
     public void addResourceHandlers(ResourceHandlerRegistry registry) {
           registry.addResourceHandler("/de/public/**")
                 .addResourceLocations("classpath:/public/")
                 .setCacheControl(CacheControl.maxAge(365, DAYS))
                 .resourceChain(false)
                 .addResolver(new VersionResourceResolver().addFixedVersionStrategy(version, "/**"));
     }
}
```

Listing 2: Spring Konfiguration für versionierte Static Assets

npm/htmx.org@2.0.10/dist/htmx.min.js`). Bei selbstgeschriebenem JavaScript wird häufig ein Hash in den Dateinamen eingefügt, der aus dem Inhalt der Datei erzeugt worden ist. Die genannten Dateien könnten zum Beispiel `style.d375.css` und `feature.49fb.js` heißen. Sobald sich ihr Inhalt verändert, ändert sich auch der Name.

Um die Codebase wartbar zu halten, sollten Links zu den Dateien versionsagnostisch bleiben. In den Sourcen wollen wir also Libraries ohne Versionsnummer und eigene Assets ohne Hash referenzieren, zur Laufzeit aber die spezifischen Versionen und Hashes verwenden, damit der Browser korrekt funktioniert. Diese Auflösung leistet in der Regel ein Frontend-Build durch eine Transpilation der Sourcen. Auf diesen Schritt können wir nicht verzichten und brauchten eine Alternative.

Static Assets

Nachdem wir alle Transpilation-Schritte entfernt hatten, drehten sich die übrigen Aufgaben des Frontend-Builds um Static Assets:

1. Static Assets ausliefern, damit Browser sie herunterladen können
2. Dateien mit einem Uniqueness-Merkmal ausstatten, damit wir den Cache invalidieren können
3. Dateien in HTML-Templates so einbinden, dass der Code wartbar bleibt und der Browser die richtigen Dateien lädt

Schauen wir uns zuerst an, wie wir das für eigenes JavaScript und CSS umsetzen können.

Das Spring Framework bietet von Haus aus unterschiedliche Möglichkeiten, Static Assets auszuliefern *[3]*. Mit einer Implementierung des `WebMvcConfigurer`-Interfaces können wir verschiedene `ResourceResolver` konfigurieren und definieren, welche Ressourcen unter welchen URL-Pfaden verfügbar sind. Der `VersionResourceResolver` mit den beiden Versionierungsstrategien `ContentVersionStrategy` und `FixedVersionStrategy` erlaubt es uns, Static Assets samt änderbaren Uniqueness-Merkmal in der URL auszuliefern.

`ContentVersionStrategy` berechnet für jede Datei (zum Beispiel `feature.js`) zur Laufzeit einen Fingerprint (zum Beispiel `ae9d`) und liefert sie unter einer versionierten URL aus (zum Beispiel `/de/public/feature-ae9d.js`). Diese Strategie sorgt für sehr effiziente Cache-Invalidierung, ist aber ungeeignet für ESM-Imports: Wenn eine zweite JavaScript-Datei `feature.js` importiert, dann müsste sie in ihrem Import-Statement die Version berücksichtigen. Spring tauscht den Importpfad nicht zur Laufzeit aus.

`FixedVersionStrategy` hingegen ergänzt einen konfigurierbaren Versionsstring (zum Beispiel `v1`) als Pfadsegment in die URL aller auslieferbaren Dateien (zum Beispiel `/de/public/v1/feature.js`). Unterhalb des Versionsstrings könnte eine zweite JavaScript-Datei `feature.js` mit einem relativen Pfad importieren, so dass der Versionsstring keine Rolle bei dem Import spielt.

Listing 2 skizziert unser Setup. Die Variable `version` lässt sich dynamisch von außen befüllen. Wir berechnen in unserem Gradle-Build einen Fingerprint über **alle** Static Assets, so dass sich der Hash ändert, wenn sich mindestens **eine** Datei verändert. Dies ist effizienter als ein Commit-Hash (= Veränderung mit jedem Deployment), aber ineffizienter als eine reine Content-Strategie. Aufgrund der geringen Änderungsrate und der überschaubaren Anzahl an Static Assets akzeptierten wir diesen Trade-off.

Mit unserem Setup funktioniert die Standard URL Syntax von Thymeleaf *[4]* so, wie wir das wollen: Versionsagnostisch in den Templates *(siehe Listing 3)* und mit dem ersetzten Versionsstring im gerenderten HTML *(siehe Listing 4)*.

```
<script type="module" th:src="@{/de/public/feature.js}"></script>
```

Listing 3: Versionsagnostische Referenz im Thymeleaf-Template

```
<script type="module" src="/de/public/v1/feature.js"></script>
```

Listing 4: Versionsspezifisch gerendertes HTML

```
<script defer th:src="@{/webjars/htmx.org/dist/htmx.min.js}"></script>
```

Listing 5: Versionsagnostische Referenz im Thymeleaf-Template

```
<script defer src="/webjars/htmx.org/2.0.10/dist/htmx.min.js"></script>
```

Listing 6: Versionsspezifisch gerendertes HTML

```
<script type="importmap" th:inline="javascript">
{
  "imports": {
     "@breuni-ui/components": "[(@{/webjars/breuni-ui/dist/comp.js})]"
  }
}
</script>
```

Listing 7: Versionsagnostische Referenz im Thymeleaf-Template

Externe Dependencies

Mit dem WebJars-Projekt *[5]* kann man NPM-Libraries über Gradle managen. Die Abhängigkeit auf HTMX lässt sich dadurch in der `build.gradle` wie jede andere Gradle-Dependency definieren: `runtimeOnly "org.webjars.npm:htmx.org:2.0.10"`. Gradle lädt das passende Jar herunter und stellt es im Classpath bereit. Dieses Jar enthält die NPM-Library, wie sie auch von einem `npm install` im `node_modules`-Verzeichnis angelegt würde.

Das Paket `org.webjars:webjars-locator-lite` konfiguriert automatisch den `LiteWebJarsResourceResolver`. Dieser Resolver stellt die auf dem Classpath liegenden WebJar-Libraries bereit. Das funktioniert analog zu den Static Assets und ermöglicht WebJars versionsagnostisch zu verwenden. *Listing 5* zeigt die beispielhafte Einbindung von HTMX im Thymeleaf-Template, welche dann als HTML in *Listing 6* gerendert wird.

JavaScript aus externen Dependencies verwenden

Die interne Breuninger Component Library wird ebenfalls als WebJar zur Verfügung gestellt, via Gradle eingebunden und durch den Resolver mit einem versionierten Pfad bereitgestellt. Aus einem Thymeleaf-Template heraus können wir die Komponenten versionsagnostisch einbinden. Doch wenn wir sie in unserem selbstgeschriebenen JavaScript verwenden wollen, dann können wir das nicht mit einem relativen Import machen, sondern müssen die Versionsnummer irgendwie in die Import-Kette bekommen, ohne die Wartbarkeit des Codes aufzugeben.

Importmap*[6]* ist ein relativ neues Browserfeature, um Aliase für Imports zu definieren und dem Browser die Information zu geben, unter welcher URL welcher Import zu finden ist. Während die Importmap als Teil des HTMLs im Template die Versionsnummern dynamisch von Thymeleaf erhält *(Listings 7 und 8)*, können wir im JavaScript versionsagnostische Aliase verwenden *(Listing 9)*.

Vorteile von CDNs nutzen

Die Static Assets sind Teil des Java-Deployments und das Backend stellt diese selbst bereit. Das ist lokal ein Vorteil: Alles funktioniert out-of-the-box. In Produktion ist es aber ungünstig, dass der gesamte Traffic über unser Backend läuft. Um diesen Traffic über ein Content Delivery Network (CDN) zu routen, brauchen wir zwei Dinge:

1. Spring muss die Static Assets kennen, um die versionsagnostischen Referenzen auflösen zu können
2. Die URL-Struktur, die Spring/Thymeleaf referenziert, muss im CDN nachgebildet werden

Wenn diese Aspekte gegeben sind, unterscheiden sich die URLs zu den Static Assets auf dem CDN (zum Beispiel `assets.prod.breuni.de/de/public/v1/feature.js`) nur durch ein Präfix von den URLs zu unserer Anwendung (`localhost:8080/de/public/v1/feature.js`). Um die lokale out-of-the-box Experience zu erhalten und gleichzeitig remote ein CDN zu verwenden, können wir zum Beispiel den `StandardLinkBuilder` von Thymeleaf erweitern *(siehe Listing 10)*, um dieses Präfix abhängig vom Profile zu setzen.

Zusammenfassung & Fazit

Auf unserer Reise zu einem Buildless-Frontend setzen wir auf HTMX als Schlüsseltechnologie, um die Menge an JavaScript signi-

```
<script type="importmap">
{
     "imports": {
          "@breuni-ui/components": "/webjars/breuni-ui/1.2.3/dist/comp.js"
     }
}
</script>
```

Listing 8: Versionsspezifisch gerendertes HTML

```
import {BreuningerButton} from '@breuni-ui/components';
customElements.define('breuninger-button', BreuningerButton);
```

Listing 9: ESM-Import in JavaScript mittels Alias

```
public class ThymeleafLinkBuilder extends StandardLinkBuilder {
     private final String prefix;

     public ThymeleafLinkBuilder(String prefix) {
           super();
           this.prefix = prefix;
     }

     @Override
     protected String processLink(final IExpressionContext context, final String link) {
           var result = super.processLink(context, link);
           return prefix + result;
     }
}
```

Listing 10: Erweiterung des ThymeleafLinkBuilders

fikant zu reduzieren. Dieser Umstand führt dazu, dass wir auf diverse Transpilation-Schritte im Frontend-Build verzichten können. Wir nutzen Standard Spring Komponenten, um Static Assets versionsagnostisch pflegen und zur Laufzeit versionsspezifisch anfragen zu können. Die Static Assets liefern wir zwar immer gemeinsam mit dem Backend aus, benutzen sie in Produktion aber nur, damit Spring und Thymeleaf die Auflösung der Referenzen leisten können. Die Auslieferung im Browser erfolgt via CDN. Lokal führt ein Starten des Backends zu einer out-of-the-box Experience. Um externe JavaScript Libraries wie unsere interne Component Library einzubinden, setzen wir auf WebJars, welche sich über Gradle managen lassen. Wir nutzen das Browserfeature Importmap, um versionsagnostische Aliase für die Imports in den JavaScript-Dateien zu definieren.

Nach unserem Refactoring ist die Frustration hinsichtlich des Frontend-Builds einer echten Freude bei der Nutzung gewichen. Es bestätigt sich, dass wir nur noch sehr wenig JavaScript selbst schreiben. Alle Entwickler haben sich in HTMX eingearbeitet und genießen, dass der Ansatz gut zu unseren bestehenden Denkmustern passt. Die Cognitive Load wurde durch die Harmonisierung auf einen einzigen technologischen Ansatz deutlich reduziert.

Für uns ein voller Erfolg.

Quellen

[1] *https://htmx.org*

[2] *https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Caching#cache_busting*

[3] *https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-config/static-resources.html*

[4] *https://www.thymeleaf.org/doc/articles/standardurlsyntax.html*

[5] *https://www.webjars.org/*

[6] *https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/script/type/importmap*



Sebastian Sprenger

sebastian.sprenger@breuninger.de



Felix Seidel

felix.seidel@andrena.de

Sebastian Sprenger arbeitet als Staff Software Engineer für Breuninger und Felix Seidel arbeitet als Agile Software Developer für andrena objects. Beide arbeiteten zusammen an dem Refactoring-Projekt, um HTMX im Team einzuführen.

Mehr als nur Single-Page-Applications: Wie viel Verantwortung braucht der Client wirklich?

Emil Pawlowski, iteratec GmbH




JS

Wenn es um das Frontend einer neuen Anwendung geht, fällt schnell die Frage: React, Angular oder Vue? Davor steht eine wichtigere: Wie viel Verantwortung muss überhaupt in den Client? An einer Beispielanwendung vergleiche ich sechs Ansätze von statischem HTML bis zur React-SPA und zeige, welche zusätzliche Client-Verantwortung sie jeweils mitbringen.

Ein neues Projekt startet, die fachlichen Anforderungen sind grob geklärt, dann kommt die Technologiefrage. Auf der Backend-Seite ist die Antwort im Java-Umfeld meist schnell da: Java, dazu zum Beispiel Spring Boot oder Quarkus. Beim Frontend dagegen fällt eine Frage meist ganz aus: Braucht diese Anwendung überhaupt eine Single-Page-Application (SPA)? Stattdessen steht die Antwort oft schon fest, bevor die Frage überhaupt gestellt wurde.

Wer direkt eine SPA voraussetzt, hat damit bereits entschieden, wesentliche UI-Verantwortung in den Browser zu verlagern. Das kann richtig sein, ist aber nicht immer der sinnvollste Weg.

Wie viel eine Anwendung wirklich im Client braucht, ist eine Architekturentscheidung, keine Gewohnheitssache. Zwischen einer statisch ausgelieferten Seite und einer vollwertigen SPA liegt ein breites Spektrum, und viele Anwendungen gehören irgendwo dazwischen.

Sechs Ansätze, eine Anwendung

Wie Client und Server sich die Arbeit teilen, ist eine der grundlegenden Architekturentscheidungen der Webentwicklung, vergleichbar mit der Wahl zwischen Monolith und verteilten Diensten auf der Backend-Seite. Zwischen den Extremen liegt ein Kontinuum: Am einen Ende steht eine Seite, die der Server fertig ausliefert, am anderen eine Anwendung, die praktisch vollständig im Browser läuft. Die Frage ist, wie viel Verantwortung dazwischen in den Client wandert: Rendering, Routing, Navigation, Anwendungszustand.

Abbildung 1 stellt das vereinfacht als Spektrum dar: Von links nach rechts wandert zunehmend Verantwortung zum Client. Im Folgenden vergleiche ich sechs Stationen auf diesem Spektrum: statisches HTML, klassisches Server-Side-Rendering (SSR), mit JavaScript angereichertes SSR, HTMX, Inertia und die Single-Page-Application (SPA). Die Ansätze sind in der Abbildung nur als Punkt dargestellt, decken in Wirklichkeit aber einen Bereich ab, in dem sie sich besonders eignen: ihren Sweet Spot. Je weiter man sich davon entfernt, desto weniger der Vorteile lassen sich nutzen und desto mehr Nachteile muss man in Kauf nehmen.



Abbildung 1: Das Spektrum der Verantwortungsverteilung von Server zu Client mit sechs Beispiel-Ansätzen (© Emil Pawlowski)

Vergleichbar werden die sechs Ansätze an einer einzigen Anwendung: einem Chat nach dem Muster bekannter KI-Assistenten, zur Vereinfachung ohne Sprachmodell und Token-Streaming. Nachrichten senden, eine Antwort erhalten, frühere Konversationen öffnen, eine neue starten. Alle sechs Varianten teilen dieselbe Oberfläche bis hin zum identischen CSS *(siehe Abbildung 2)*; die fünf interaktiven Varianten teilen zusätzlich denselben Backend-Kern rund um Kon-

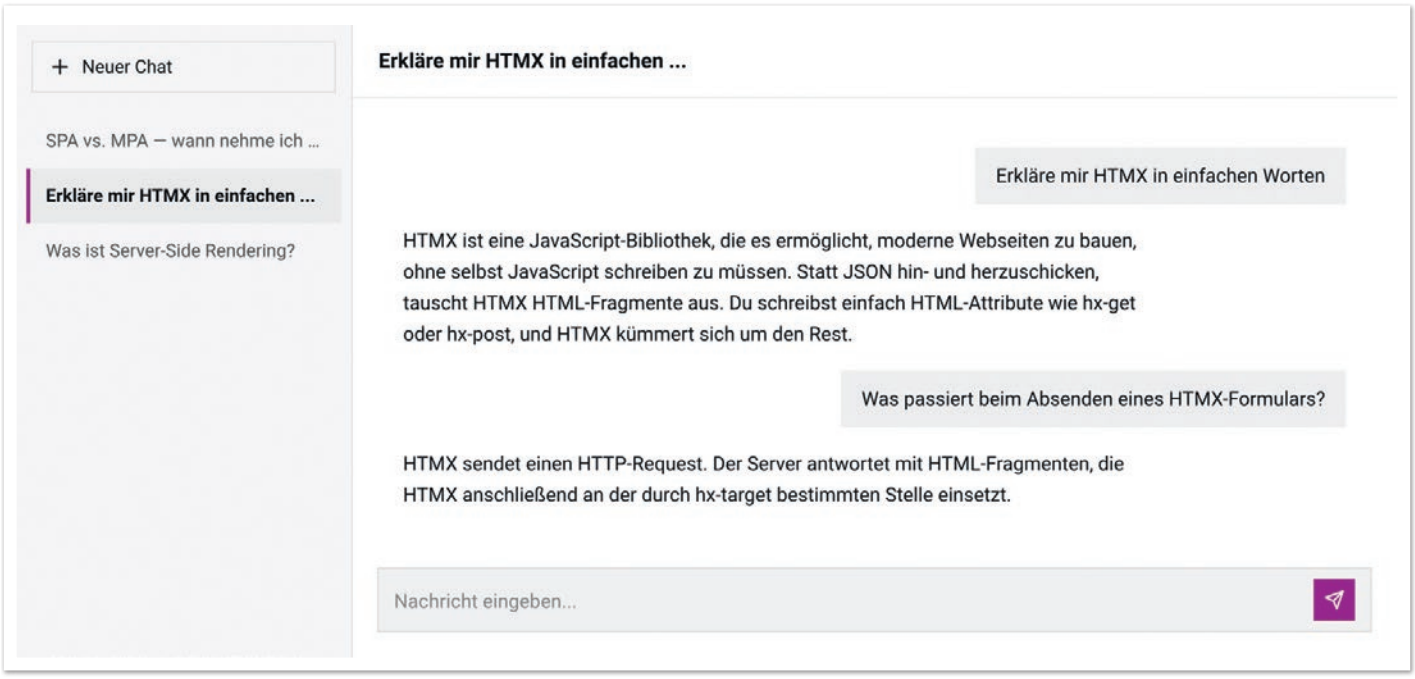


Abbildung 2: Die Chat-Oberfläche, identisch für alle sechs Varianten (© Emil Pawlowski)

```
// SsrConversationController.java
@PostMapping("/chat/{id}/messages")
public String sendMessage(@PathVariable UUID id, @RequestParam String content) {
     chatService.sendMessage(id, content);
     return "redirect:/ssr/chat/" + id;  // Post-Redirect-Get
}


<!-- ssr/chat.html -->
<form th:action="@{/ssr/chat/{id}/messages(id=${conversation.id})}" method="post">
     <textarea name="content" placeholder="Nachricht eingeben..." required></textarea>
     <button type="submit" class="send-button">…</button>
</form>
```

Listing 1: Klassisches SSR, beide Enden. Oben der Controller: Er nimmt den POST entgegen, legt die Nachricht ab und antwortet mit einem Redirect statt mit HTML (Post-Redirect-Get), sodass der Browser die Seite anschließend per GET neu holt. Unten das Thymeleaf-Template: ein gewöhnliches Formular ohne eine Zeile JavaScript, das der Browser mit seinem Standardverhalten abschickt. (© Emil Pawlowski)

versationen und Nachrichten. Dieser Kern läuft auf Spring Boot 3 unter Java 21. Zwischen den Varianten unterscheidet sich vor allem, wie viel Arbeit jeweils Client und Server übernehmen.

Statisch und SSR: die Baseline

Am äußersten Server-Ende steht die statische Seite: reines HTML und CSS, kein JavaScript, ein Server, der nur Dateien ausliefert. Der Browser rendert dabei ausschließlich das ausgelieferte Dokument. Für einen Blog oder eine Dokumentation kann das bereits reichen, für den Chat nicht: Die Seite kann nur einen statischen Zustand anzeigen.

Interaktiv wird es erst mit klassischem Server-Side-Rendering. Man tippt eine Nachricht und klickt auf den Senden-Button: Der Browser schickt das Formular per Standardverhalten ab. Der Spring-MVC-Controller nimmt den POST entgegen, legt die Nachricht ab und antwortet mit einem Redirect zurück auf die Chat-Seite. Dieses Post-Redirect-Get löst ein frisches GET aus, und die Template-Engine Thymeleaf rendert die komplette Seite neu, jetzt mit der neuen Nachricht darin. *Listing 1* zeigt beide Enden.

Beim Redirect lädt der Browser die Seite vollständig neu. Fokus oder Scrollposition können dabei verloren gehen, und ohne zusätzliche Client-Logik lassen sich eigene Ladezustände oder eine einlaufende Antwort nicht ohne Weiteres umsetzen. Bei langsamen Antworten wirkt die Anwendung dadurch schnell zäh.

Dafür bleibt die Technik einfach: Controller, Template und Browserstandardverhalten, ohne eigenes Client-Build. Im Kern dasselbe Modell wie bei PHP, JavaServer Pages (JSP) oder eben Thymeleaf. Native Formulare, Links und semantisches HTML nutzen viele

```
<!-- enhanced/chat.html -->
<form th:action="@{/enhanced/chat/{id}/messages(id=${conversation.id})}"
        method="post" id="chat-form">
     <textarea name="content" required></textarea>
     <button type="submit" class="send-button">…</button>
</form>


// chat-enhanced.js
form.addEventListener('submit', function (e) {
     e.preventDefault();
     var content = input.value.trim();
     if (!content) return;
     input.value = '';
     sendButton.disabled = true;
     appendMessage(content, 'user');              // optimistisch sofort anzeigen
     showProgressBar();
     var thinkingEl = showThinkingIndicator();
     fetch(asyncUrl, {
           method: 'POST',
           headers: { 'Content-Type': 'text/plain' },
           body: content
     }).then(function (r) { return r.json(); })
        .then(function (data) {
           removeThinkingIndicator(thinkingEl);
           hideProgressBar();
           appendMessage(data.content, 'assistant');  // Antwort nachladen
           sendButton.disabled = false;
     });
});
```

Listing 2: Progressive Enhancement. Oben das Formular mit regulärem `action` und `method` als Fallback ohne JavaScript, unten das Skript, das denselben Submit abfängt und die Nachricht per `fetch()` schickt: Die eigene Nachricht erscheint optimistisch sofort, danach wird die Antwort nachgeladen. Der Ausschnitt ist gekürzt: Fortschrittsbalken, Konversationsübersichtsliste und Fehlerbehandlung sind nicht dargestellt. (© Emil Pawlowski)

Browsermechanismen ohne zusätzliche Client-Logik; Inhalte stehen zudem ohne clientseitiges Rendering bereit.

SSR+JS und HTMX: Interaktivität ohne Frontend-Framework

Den vollen Reload kann man vermeiden, ohne das Server-Modell aufzugeben. Statt dem Browser das Abschicken des Formulars zu überlassen, schreibt man dafür selbst ein wenig JavaScript, bindet es in die Seite ein und versendet die Nachricht per `fetch()`. Damit lässt sich im Client eigener Code auslösen: Wie *Listing 2* zeigt, erscheint die eigene Nachricht dann optimistisch, also sofort, der Senden-Button ist deaktiviert und ein eigener Ladeindikator eingeblendet, während die Antwort vom Server noch aussteht.

Diese Basis funktioniert weiter, wenn JavaScript ausfällt, und braucht kein Build-Tooling: ein paar Zeilen unminifiziertes JavaScript, direkt ausgeliefert. Darin steckt neben dem Senden auch der Ladeindikator, der Fortschrittsbalken und das Nachladen der Seitenleiste. Neu entsteht dafür ein Skalierungsproblem: Vanilla JavaScript gibt weder ein UI-Komponentenmodell noch eine Projektstruktur vor. Ohne eigene Konventionen können so schnell schwer wartbare DOM-Manipulationen und Event-Handler entstehen.

HTMX verfolgt denselben Grundgedanken, ersetzt das handgeschriebene JavaScript aber durch HTML-Attribute. Die Grundidee: das serverseitig gerenderte Modell um eine Möglichkeit erweitern, bei einem abgeschickten Formular nur einen Teil der Seite auszutauschen statt sie komplett neu zu laden. Gesteuert wird das über HTML-Attribute, die HTMX im Markup auswertet: `hx-post` schickt das Formular, `hx-target` benennt das Element, das aktualisiert wird, `hx-swap` bestimmt, wie. Entscheidend ist, was der Server zurückliefert: ein HTML-Fragment, das an der richtigen Stelle eingesetzt wird, keine ganze Seite wie beim SSR und kein JSON wie bei einer SPA. Dieses Muster wird häufig als „HTML over the wire" bezeichnet und findet sich etwa auch bei Hotwire *[1]*; die Idee selbst ist älter. *Listing 3* zeigt das Sende-Formular und das zurückgelieferte Fragment. Eine einzige Antwort aktualisiert dabei drei Stellen im Dokument: Die Nachrichtenliste bekommt das Fragment, das `hx-target` benennt. Seitenleiste und Kopfzeile trägt dieselbe Antwort per `hx-swap-oob` (Out of Band) nach. Nötig ist das, weil die erste Nachricht den Titel der Konversation setzt und der an beiden Stellen steht.

Auch diese Variante vermeidet den Vollreload, bei der Seitenleiste sogar mit weniger Aufwand. In der Demo lädt die SSR+JS-Variante dafür die ganze Seite ein zweites Mal und schneidet sich den Abschnitt heraus. Bei HTMX genügt das `hx-swap-oob`-Attribut aus *Listing 3*, die Aktualisierung kommt in derselben Antwort mit. Die Demo kommt dabei ohne selbst geschriebenes JavaScript und ohne Client-Build aus: Die Bibliothek wird als einzelne Datei ausgeliefert. Serverseitige Logik bleibt beim Server, partielle Updates ersetzen den Vollreload, und die erste Seite kommt komplett vom Server.

HTMX wird inzwischen auch in der Spring-Dokumentation behandelt *[2]* und ist mit wenigen zusätzlichen Konzepten zugänglich.

So attraktiv das klingt: Komplexität wird verschoben, nicht beseitigt. HTMX reduziert den Bedarf an handgeschriebenem JavaScript und oft auch das Build-Tooling, dafür entstehen an anderer Stelle neue Dinge, die man beherrschen muss: Verträge über den Zuschnitt der HTML-Fragmente, Zielbeziehungen im DOM, Zustände für Historie, Fokus und Fehler. Kein Build-Tool zu brauchen heißt eben nicht, insgesamt weniger zu zahlen. Und es gibt eine Grenze dieses Modells: Sobald Client-Logik jenseits des Attributmodells nötig wird, fehlt HTMX ein Komponentenmodell oder eine andere festgelegte Struktur. Man weicht auf zusätzliches JavaScript aus oder muss sich selbst eine Struktur überlegen, die die Wartbarkeit mit wachsender Komplexität trägt. Genau hier setzt Inertia an: Es nutzt ein Frontend-Framework mit Komponentenmodell, lässt die Routendefinition aber beim Server.

Inertia: Routen beim Server, Rendering im Client

Einen Schritt weiter Richtung Client liegt Inertia, eine Bibliothek aus dem Laravel-Ökosystem, die dem Server das Routing belässt, während echte Frontend-Komponenten das Rendering übernehmen. Offiziell unterstützt werden dafür React, Vue und Svelte *[3]*. Inertia versteht sich als Kleber zwischen Server- und Frontend-Framework. Die Verantwortung verteilt sich dabei ungleich: Der Client rendert und navigiert mit echten Komponenten, die Autorität über die Routing-Definition bleibt beim Server.

```
<!-- htmx/chat.html -->
<form th:hx-post="@{/htmx/chat/{id}/messages(id=${conversation.id})}"
        hx-target="#message-list" hx-swap="afterbegin"
        hx-disabled-elt="find .send-button" hx-indicator="#thinking-indicator">
     <textarea name="content" required></textarea>
     <button type="submit" class="send-button">…</button>
</form>


<!-- htmx/fragments/messages.html — das zurückgelieferte Fragment -->
<!-- .message-list ist column-reverse, deshalb zuerst die Antwort, dann die Frage -->
<div class="message assistant"><div class="message-content" th:text="${assistantMessageContent}"></div></div>
<div class="message user"><div class="message-content" th:text="${userMessageContent}"></div></div>

<nav id="conversation-list" hx-swap-oob="true"> … </nav>       <!-- Out-of-Band: Seitenleiste -->
<header id="chat-header" hx-swap-oob="true" th:text="${conversation.title}"></header>
```

Listing 3: HTMX, `hx-*`-Attribute statt eigenem JavaScript. Oben das Formular: `hx-post` schickt es ab, `hx-target` und `hx-swap` bestimmen, wo die Antwort landet, `hx-disabled-elt` und `hx-indicator` übernehmen Button-Sperre und Ladeindikator. Unten die Antwort des Servers: ein HTML-Fragment statt einer ganzen Seite. Es füllt die Nachrichtenliste und trägt per `hx-swap-oob` zusätzlich Seitenleiste und Kopfzeile nach. (© Emil Pawlowski)

```
// InertiaConversationController.java
@GetMapping("/chat/{id}")
public ResponseEntity<String> chat(@PathVariable UUID id) {
     return chatService.getConversation(id)
                  .map(conversation -> inertia.render("Chat", Map.of(
                              "conversation", conversation,
                              "conversations", chatService.listConversations())))
                  .orElseGet(() -> inertia.redirect("/inertia/"));
}

@PostMapping("/chat/{id}/messages")
public ResponseEntity<String> sendMessage(@PathVariable UUID id, @RequestBody Map<String, String> body) {
     chatService.sendMessage(id, body.get("content"));
     return inertia.redirect("/inertia/chat/" + id);
}
```

Listing 4: Inertia: Der Controller rendert eine benannte Client-Komponente mit ihren Daten (`inertia.render`) und leitet nach dem Senden zurück (`inertia.redirect`). Das Routing bleibt serverseitig. (© Emil Pawlowski)

```
// api.js
export async function sendMessage(conversationId, content) {
  const res = await fetch(`/api/conversations/${conversationId}/messages`, {
     method: 'POST',
     headers: { 'Content-Type': 'application/json' },
     body: JSON.stringify({ content }),
  });
  return res.json();
}


// ChatPage.jsx
async function handleSend(content) {
  setPendingMessage(content);              // optimistisch
  setIsThinking(true);
  await sendMessage(id, content);
  const [conv, convs] = await Promise.all([
     fetchConversation(id), fetchConversations(),
  ]);
  setConversation(conv);
  setConversations(convs);                  // Zustand neu laden
  setPendingMessage(null);
  setIsThinking(false);
}
```

Listing 5: SPA, nur die Kommunikation mit dem Backend. Das Frontend spricht eine JSON-API (`/api/…`) an und pflegt seinen Zustand selbst: optimistisch setzen, senden, neu laden. Nicht zu sehen sind Router, Komponenten, Fehlerbehandlung und das Rendering der Oberfläche, die in dieser Variante ebenfalls im Client liegen. (© Emil Pawlowski)

Das zugrunde liegende Protokoll ist vergleichsweise simpel: Der erste Aufruf liefert HTML mit eingebettetem Page-Object. Folgenavigationen übertragen dieses als JSON; der Client aktualisiert daraus Komponente und Props. Auf der Serverseite sieht das wie gewohntes Controller-Handeln aus, nur dass man statt eines Templatenamens eine Komponente mit ihren Daten rendert. *Listing 4* zeigt das an den Endpunkten der Demo: `inertia.render(...)` liefert die Seite mit Daten, `inertia.redirect(...)` führt nach dem Senden zurück. Auf der Java-Seite arbeitet die Demo mit Inertia4J, im Client mit `@inertiajs/react`.

Inertia wirbt auf der eigenen Startseite damit, dass keine API nötig sei („No API required" *[3]*). Das heißt nicht, dass gar kein HTTP-Protokoll im Spiel wäre. Inertia hat ein spezifiziertes JSON-Protokoll, und der Client dieser Anwendung postet an `/inertia/…` statt an `/api/…`. Gemeint ist, dass keine zweite, eigens entworfene REST-Schnittstelle fürs Web-Frontend nötig ist. Ob dieser Vorteil trägt, hängt davon ab, ob ohnehin schon eine API für andere Clients existiert, etwa für eine mobile Anwendung.

Inertia spielt seine Stärke aus, wenn eine serverzentrierte Anwendung komponentenbasierte Frontend-Seiten braucht, ohne dafür eine separate Web-API und einen clientseitigen Router einzuführen. Zu bezahlen sind dafür eine stärkere Bindung an das Inertia-Protokoll und seine Adapter sowie eine im Vergleich kompliziertere Einrichtung. Das Ökosystem ist bei Laravel stark, im Java-Umfeld dünn: Die Java-Implementierung Inertia4J deckt das Kernprotokoll ab, neuere Bausteine stehen noch auf der Roadmap *[4]*.

SPA: Routing, State und Rendering im Client

Am äußersten rechten Ende des Spektrums steht die Single-Page-Application. In der hier betrachteten klassischen Client-SPA übernimmt das Frontend nach dem initialen Laden Rendering, Routing und UI-Zustand. Fachliche Daten kommen über eine JSON-API. Das erfordert einiges an manuellem Klebe-Code zwischen den beiden Seiten. Als Frontend-Framework kommt hier beispielhaft React 19 zum Einsatz, das clientseitige Routing übernimmt `react-router-dom` 7. *Listing 5* zeigt beide Hälften: den API-Aufruf und den Handler in der Komponente, der die Nachricht optimistisch setzt, sendet und

den Zustand anschließend neu lädt. Anders als bei der serverseitig gerenderten Variante übernimmt kein Standardverhalten des Browsers mehr das Absenden des Formulars, es ist selbst geschriebener Code.

Damit übernimmt der Client zusätzlich Routing, UI-Zustand und Rendering; bis das Bundle geladen und React initialisiert ist, enthält das ausgelieferte HTML in der Beispielimplementierung noch keinen Anwendungsinhalt, der Bildschirm bleibt leer. Hinzu kommen typischerweise Build-Tooling und eine explizite API.

Dafür ermöglicht eine SPA flüssige Navigation und große Freiheit bei komplexer Client-Interaktion. Für Anwendungen wie Google Maps oder Figma ist das eine begründete Wahl.

Wie viel Client braucht eine Anwendung?

Mehr Verantwortung im Client ermöglicht zusätzliche Interaktivität, bringt aber typischerweise weitere Komplexität mit sich. *Abbildung 3* stellt beides schematisch gegenüber. In der Demo liefern HTMX und Inertia vergleichsweise viel Interaktivität bei überschaubarer zusätzlicher Komplexität.



Abbildung 3: Qualitative Einordnung von Interaktivität und zusätzlicher Client-Komplexität in der Beispielanwendung (© Emil Pawlowski)

Für die eigene Anwendung gilt es, den passenden Trade-off zu finden.

Vor der Technologiewahl helfen fünf Fragen:

1. Wie viel dauerhafter, komplexer Zustand muss im Browser leben, und wie viel davon ausschließlich dort?
2. Wie häufig muss die Oberfläche unmittelbar reagieren, ohne zuerst serverseitigen Zustand oder Rendering abzuwarten?
3. Braucht die Anwendung Offline-Fähigkeit oder latenzkritische Interaktion?
4. Bleibt der Server die Autorität für Navigation, Validierung und Zustandsübergänge?
5. Gibt es ohnehin eine öffentliche oder mobile API, die dieselben Fähigkeiten braucht?

Die Antworten ordnen eine Anwendung schon recht gut ein. Gibt es kaum Interaktion, reicht statisches HTML. Bleibt die Autorität über Navigation und Zustandsübergänge weitgehend beim Server (Frage 4), ist SSR ein naheliegender Ausgangspunkt. Müssen einzelne Stellen ohne Reload reagieren (Frage 2), während Navigation und Validierung beim Server bleiben, liegt der Bereich von SSR+JS und HTMX. Lebt umfangreicher, browserexklusiver Zustand dauerhaft im Client und sind die Interaktionen latenzkritisch (Fragen 1 und 3), führt der Weg zur SPA. Existiert ohnehin eine öffentliche oder mobile API (Frage 5), sinkt der Zusatzaufwand einer SPA; Inertia verliert dagegen einen seiner Vorteile. Deshalb landet ein einfacher Support-Chat nach diesen fünf Fragen ganz anders als ein Werkzeug in der Größenordnung von Slack. Die Ansätze lassen sich auch mischen, sofern an Feature-Grenzen klar bleibt, wer Routing, Zustand und DOM besitzt.

Auch vorhandene Fähigkeiten und Infrastruktur zählen: Für ein erfahrenes React-Team mit bestehender API kann eine SPA einfacher sein als eine unbekannte SSR- oder Inertia-Lösung. Die simpelste Architektur ist relativ zum Team, nicht absolut.

Fazit

In der Praxis hilft eine Eskalationsleiter: mit der für Anforderungen und Team einfachsten tragfähigen Lösung beginnen und zusätzliche Client-Verantwortung erst einführen, wenn konkreter Bedarf sie rechtfertigt. Die Frage ist nicht, wie viel Frontend man bauen kann, sondern wie viel man wirklich bauen muss.

Quellen

[1] Hotwire: „HTML Over The Wire". *https://hotwired.dev*, abgerufen am 19.08.2026.

[2] Spring Framework Reference: „HTML Fragments". *https://docs.spring.io/spring-framework/reference/web/webmvc-view/mvc-fragments.html*, abgerufen am 19.08.2026.

[3] Inertia.js: „The Modern Monolith". *https://inertiajs.com*, abgerufen am 19.08.2026.

[4] Inertia4J. *https://github.com/Inertia4J/inertia4j*, abgerufen am 19.08.2026.



Emil Pawlowski
iteratec GmbH
emil.pawlowski@iteratec.com

Emil Pawlowski ist Senior Software Engineer bei iteratec und arbeitet an der Architektur und Entwicklung moderner Webanwendungen. Ein Schwerpunkt seiner Arbeit liegt auf KI-Agenten, sowohl als Werkzeug im Entwicklungsprozess als auch als Bestandteil von Kundenanwendungen. Seine Erfahrungen mit neuen Technologien gibt er in Vorträgen und Workshops weiter.

2026

DOAG

Konferenz + Ausstellung

Die ORACLE Anwenderkonferenz

Nürnberg | 17. – 19. Nov.

#DOAG2026



anwenderkonferenz.doag.org

Die neue Rolle des Architekten im KI-Zeitalter

Eldar Sultanow & Lars Santesson, Capgemini Deutschland GmbH



Künstliche Intelligenz verändert nicht nur Technologien, sondern auch die Art und Weise, wie Unternehmen organisiert, gesteuert und weiterentwickelt werden. Damit wandelt sich auch die Rolle des IT-Architekten grundlegend: Weg vom reinen Technologieexperten hin zu einem Integrator von Technologie, Menschen und Organisation. Der folgende Beitrag beleuchtet die zentralen Treiber dieses Wandels und zeigt auf, welche Kompetenzen Architekten künftig benötigen, um Unternehmen sicher durch das KI-Zeitalter zu führen.

Warum der Architekt zur entscheidenden Rolle im Unternehmen wird

Fähigkeiten, für die früher Jahre an Erfahrung nötig waren, sind heute in Sekunden verfügbar. Leistung wird demokratisiert. Und mit ihr steigt die Messlatte des Wettbewerbs dramatisch.

Künstliche Intelligenz ist eben nicht „bloß schon wieder ein weiterer Technologiewechsel". Sie definiert neu, wie Organisationen Wert schaffen, Entscheidungen treffen und konkurrieren. In dieser neuen Realität stößt klassische IT-Architektur an ihre Grenzen. Statische Systeme, starre Strukturen und isolierte Lösungen halten mit beschleunigtem Wandel, wachsender Ungewissheit und intelligenten Maschinen als aktiven Mitspielern der Wertschöpfung schlicht nicht mehr Schritt.

Zukunftsfähige Architekturen entstehen selten durch Technologie allein, sondern vielmehr durch die Fähigkeit, Zeit, Ungewissheit und Mensch-Maschine-Kollaboration zu einem stimmigen Ganzen zu integrieren.

Der Architekt wird zum Integrator zweier Extreme, nämlich zwischen tiefen technischen Systemen und menschlicher Komplexität.

Wer diesen Wandel versteht, entwirft nicht nur bessere Systeme. Er formt Organisationen, die anpassungsfähig, resilient und wettbewerbsfähig bleiben, und zwar in einer brutal neuen Welt, in der KI kein Werkzeug, sondern eine Grundschicht der Realität geworden ist.

Vom Umfeld zur neuen Rolle

Um die Transformation der Architektenrolle zu verstehen, brauchen wir drei Schritte in dieser Reihenfolge:

- **Erstens** verändert sich die IT-Landschaft selbst grundlegend, getrieben von KI, Echtzeitdaten und verteilten Systemen.
- **Zweitens** erzeugt diese neue Landschaft eine neue organisatorische Herausforderung: Wie bleiben Systeme transparent, nachhaltig, steuerbar und skalierbar, wenn Komplexität und Ungewissheit steigen?
- **Drittens** definieren genau diese Herausforderungen die Rolle des Architekten neu. Er wird nicht bloß zum technischen Designer, sondern zum Integrator, der Technologie, Organisation und Mensch-Maschine-Kollaboration zusammenführt.

Die IT-Landschaft der Zukunft

Die klassische IT-Landschaft „hier die Server, dort die Anwendungen" löst sich auf. Cloud, Edge und KI verschmelzen zu einem durchgehenden Gewebe. Die Datenströme sind nicht mehr statisch. Sie verhalten sich wie lebende Organismen: Sie fließen, verdichten sich, mutieren und verschwinden. Zukunftsfähige Architekturen entstehen dort, wo drei neue Dimensionen mitgedacht werden.

Zeit als Designfaktor

Jahrzehntelang war IT-Architektur räumlich gedacht: Rechenzentrum vs. Cloud, On-Prem versus Off-Prem. Künftig wird die Zeit zum entscheidenden Gestaltungsprinzip. Das heißt, Systeme müssen sowohl im Raum als auch in der Zeit resilient sein: anpassungsfähig auf Millisekunden-Ereignisse am Edge und robust über Jahrzehnte hinweg, um regulatorische Archivierungspflichten zu erfüllen.

Für große Architekturen entsteht eine neue Spannung: Echtzeitreaktion gegen Langzeitgedächtnis. Der Architekt muss für zeitliche Dualität entwerfen: für Systeme, die schnell vergessen und ewig erinnern. Das verlangt explizite Lifecycle-Strategien für Daten, Modelle und Entscheidungen. Zeit wird damit von einer äußeren Randbedingung zu einer steuerbaren Architekturdimension.

Uncertainty by Design

Früher bedeutete Architektur Kontrolle. Heute heißt es, Ungewissheit zu gestalten. Märkte, Gesetze und geopolitische Realitäten verändern sich in Zyklen, die kürzer sind als klassische IT-Investitionspläne. Künftige Architekturen müssen daher eine gewisse "Optionalität" einbauen, zum Beispiel in Form von Feature Toggles, modularen Verträgen oder umkehrbaren Datenflüsse. Wenn Architekten Systeme entwerfen, dann entwerfen sie auch Offenheit gegenüber der Zukunft.

Die eigentliche Stärke solcher Architekturen liegt in der Reversibilität. Entscheidungen sind nicht mehr endgültig, sondern vorläufig. Jedes Deployment, jede Integration, jeder Datenfluss muss so gestaltet sein, dass er rückgängig gemacht, umgeleitet oder ersetzt werden kann.

Das Architektur-Paradigma verschiebt sich von der Optimierung zur Überlebensfähigkeit. Gute Systeme sind mehr als nur effizient. Sie sind solche, die sich weiterentwickeln können, ohne zu kollabieren.

Maschinen als Stakeholder

IT wurde für Menschen gebaut. Künftig werden Maschinen selbst zu Akteuren: IoT-Geräte, autonome Systeme, KI-Agenten. Sie verhandeln, entscheiden und konsumieren Services. Architekten müssen Maschinen als vollwertige Stakeholder behandeln und Governance-Regeln etablieren, die Menschen und Maschinen gleichberechtigt einbinden.

Sobald Maschinen zu Stakeholdern werden, werden sie auch zu ökonomischen Akteuren. KI-Agenten verbrauchen Ressourcen, erzeugen Wert und konkurrieren um Aufmerksamkeit in digitalen

Ökosystemen. Architekten gestalten deshalb neben technischen Schnittstellen auch Anreizstrukturen. So entsteht eine neue Architekturschicht: **die Ökonomie der Maschinen**.

Die IT-Landschaft der Zukunft wird zum Nervensystem der Organisation: lernfähig wie ein Gehirn, sensibel wie ein Immunsystem, skalierbar wie ein Ökosystem. Architekten sind die Neurochirurgen dieser neuen Unternehmenslogik. Sie operieren an Verbindungen, nicht an einzelnen Komponenten.

Von der Black Box zum Enabler

Unternehmen bekommen KI-Systeme und -Werkzeuge heute oft übergestülpt. Externe Anbieter integrieren sie und CIOs sowie CDOs stehen vor einer neuen Frage: Wie bleibt das Ganze transparent, nachhaltig, steuerbar und skalierbar?

Genau hier wird die Rolle des Architekten kritisch. Architekten entmystifizieren KI, übersetzen sie in klare Strukturen und machen sie für das Business beherrschbar. Transparenz wird dabei zur strukturellen Anforderung. Es genügt nicht mehr, Entscheidungen im Nachhinein zu erklären. Wir müssen Architekturen so entwerfen, dass Entscheidungen jederzeit nachvollziehbar, einsehbar und prüfbar sind (Stichwort Data Lineage, Model Provenance und Decision Logs).

Architekturen entstehen aus äußeren Kräften

Externe Treiber, von geopolitischen Verschiebungen bis hin zu Cyberbedrohungen, zwingen Organisationen zu neuer Resilienz. Diese Kräfte formen zuerst den Architekten. Die neuen Architekturen folgen danach. Das dreht die alte Henne-Ei-Frage um: Architekt oder Architektur? Heute ist die Antwort eindeutig. Äußere Einflüsse formen den Architekten, der daraufhin die Architektur neu denkt.

Geopolitik und Cyberrisiken sind die heimlichen Architekten moderner IT.

Daraus folgt eine grundlegende Verschiebung: Architektur ist nicht mehr neutral. Jede Designentscheidung spiegelt Annahmen über Souveränität, Vertrauen und Kontrolle. Die Wahl eines Cloud-Anbieters, eines Modells oder eines Datenstandorts wird zum geopolitischen Statement. Architekten müssen deshalb über Technologie hinausdenken. Sie bewegen sich in einer Landschaft, die von Macht (nicht nur von Performance) geprägt ist.

Zwei Pole der Kompetenz

KI macht viel Spezialwissen obsolet. Architekten müssen deshalb zwei Extreme beherrschen *(siehe Abbildung 1)*:

- **Soft Skills:** transformative Wirkung auf Menschen, Kommunikation, Führung.
- **Deep Tech:** KI-Systeme verstehen, beherrschen und bauen.

Nur die Kombination beider Pole ermöglicht zukunftsfähige Architekturen.

Die Brücke zwischen den Extremen

Beide Pole zu beherrschen, bedeutet nicht Breite, sondern Integration. Der künftige Architekt besitzt nicht einfach Soft Skills und tiefe Technikkompetenz. Er arbeitet an der Schnittstelle, an der sich beide Domänen gegenseitig verstärken.

Auf der menschlichen Seite wird *Sensemaking unter Ungewissheit* zur Kerndisziplin. KI-Systeme erzeugen Antworten in großem Maßstab, aber definieren keine Relevanz. Architekten filtern, priorisieren und übersetzen Ergebnisse in Richtung. Das erfordert Urteilsvermögen und nicht nur Intelligenz sowie die Fähigkeit zu entscheiden, wann wir der Maschine vertrauen und wann wir sie überstimmen.

Ebenso entscheidend ist *emotionale Navigation*. KI beschleunigt Arbeitsabläufe und verstärkt zugleich Mehrdeutigkeit, Druck und Widerstand. Teams straucheln im ständigen Zustrom an Möglichkeiten ohne klare Grenzen. Der Architekt wird zur stabilisierenden Kraft und führt Menschen durch Veränderung, erhält Vertrauen und sorgt dafür, dass Geschwindigkeit nicht die Kohärenz zerstört.

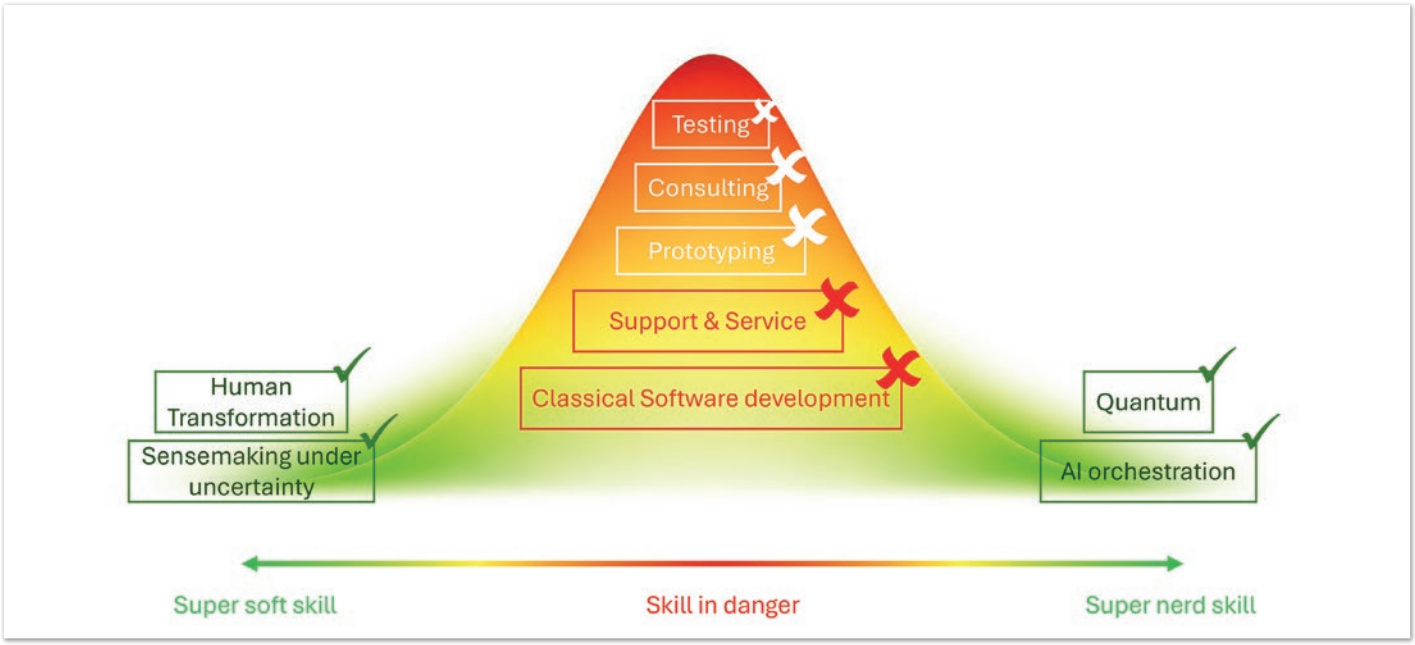


Abbildung 1: Zwei Kompetenz-Pole, die durch KI noch nicht einfach ablösbar sind (© Eldar Sultanow, Lars Santesson)

Auf der technischen Seite ersetzt *KI-System-Orchestrierung* das isolierte Lösungsdesign. Architekten hören auf, statische Systeme zu bauen und entwerfen dynamische Konstellationen aus Modellen, Agenten und Services. Sie müssen dabei die richtigen Modelle auswählen, Interaktionen strukturieren, Kontext managen und Zuverlässigkeit über verteilte Intelligenz hinweg sicherstellen. Die Herausforderung verschiebt sich vom Bauen von Komponenten zum Koordinieren von Verhalten. Der eigentliche Hebel entsteht erst, wenn beide Dimensionen zusammenkommen.

Ein Architekt, der Orchestrierung beherrscht, aber kein Gespür für Menschen hat, baut Systeme, die niemand annimmt. Ein Architekt, der stark in Führung, aber schwach in Technologie ist, kann das System selbst nicht gestalten.

Zukunftsfähige Architekturen verlangen deshalb eine neue Synthese, nämlich technische Systeme, entworfen mit menschlicher Absicht, und menschliche Systeme, geleitet von technischer Möglichkeit. Das ist der neue Standard. Nicht Tiefe oder Breite, sondern *Kohärenz über Extreme hinweg*. Der künftige Architekt ist kein Spezialist. Und kein Generalist. Der künftige Architekt ist ein Integrator der Extreme.

Warum Mittelmaß verschwindet

KI demokratisiert Leistung. Fähigkeiten, die früher jahrelanges Training, Eliteausbildung oder große Teams erforderten, sind heute über eine einzige Schnittstelle zugänglich. Das verändert Wettbewerb grundlegend. Der Vorteil liegt nicht mehr im Zugang zu Wissen, sondern in der Fähigkeit, es präzise und zielgerichtet anzuwenden. Die Messlatte steigt. Was einst als außergewöhnlich galt, wird zum Standard. Mittelmaß verschwindet. Dabei ist es wohl kaum so, dass alle exzellent werden. Dennoch wird Exzellenz zunehmend zur Mindesterwartung.

Architekten müssen Systeme und Organisationen entwerfen, die oberhalb dieser neuen Grundlinie operieren. Differenzierung entsteht dort nicht mehr aus Fähigkeit allein, sondern aus Kohärenz, Urteilskraft und Umsetzung.

Lernen verstehen

In einer Welt, in der KI selbst Wissen erzeugt, müssen Architekten verstehen, wie Lernen funktioniert, und zwar sowohl individuell als auch organisational und maschinell. Wer versteht, wie Menschen und Maschinen lernen, kann Strukturen entwerfen, die über die Zeit anpassungsfähig bleiben.

Wissen verliert an Wert. Lernen wird zur Kernkompetenz.

Wir brauchen also einen Wechsel von statischen Wissensmodellen zu kontinuierlichen Lernsystemen. Organisationen müssen als lernende Organismen gedacht werden, in denen Feedbackschleifen in jeden Prozess eingebettet sind. Daten werden nicht für Reporting gesammelt, sondern erfasst, um Verhalten anzupassen.

Vom Werkzeugkasten zum Mindset

Architekten sind keine reinen Technologie-Designer mehr. Sie entwickeln Denkweisen für Cyber-Resilienz, datengetriebenes Geschäft und KI-Ethik. Sie schaffen Architekturen, die nicht nur skalieren, sondern Vertrauen und Akzeptanz aufbauen. Mit anderen Worten wird Architektur zur ethischen Frage und Transparenz ist die neue Währung.

Dieses Mindset zeigt sich in konkreten Entscheidungen, und zwar wie wir Risiken verteilen, wie wir mit Fehlern umgehen und wie wir Verantwortung zwischen Mensch und Maschine teilen. Architektur wird zum Vehikel kulturellen Wandels und definiert, was Systeme tun und wie Organisationen denken.

AI-to-DNA: KI muss zur Kernsubstanz werden

Diesen Begriff „AI-to-DANN" übernehme ich von Christoph Magnussen *[1]*, denn von ihm stammt der Impuls sowie die zugrundeliegende Begriffsidee. Nimmt man dieses Mindset ernst, führt es zwangsläufig zu einer tieferen Transformation. KI darf kein Add-on sein. Sie muss Teil der DNA der Organisation werden. Ganz konkret muss das in Fähigkeiten, Entscheidungslogik und in alltäglichem Verhalten geschehen.

Unternehmen, die KI lediglich „nutzen", bleiben Nutzer. Unternehmen, die KI über Schlüsselpersonen und Rollen *verkörpern*, werden adaptiv. Weitergedacht: KI prägt Prozesse, Rollen und Kultur so selbstverständlich wie heute Strom oder Software. Entscheidungen sind datengetrieben. Lernen läuft kontinuierlich. Mensch und Maschine arbeiten nicht mehr nebeneinander, sondern eng verwoben. Es gibt einen fundamentalen Unterschied zwischen KI-Adoption und KI-Nativität. Adoption optimiert bestehende Prozesse. KI-native Organisationen gestalten sich rund um Lernen, Anpassung und Zusammenarbeit mit Maschinen selbst neu. Das ist kein inkrementeller Schritt. Das ist eine strukturelle Transformation.

Defense als Vorreiter

Im Verteidigungssektor ist der Rollenwandel bereits Realität. Soldaten tragen Exoskelette, nutzen Sensorfusion, KI-gestützte Lagebilder und Echtzeit-Entscheidungsunterstützung. KI sitzt dort nicht in irgendeinem Rechenzentrum, sondern erweitert den Menschen unmittelbar. Programme wie die der **DARPA** zeigen seit Jahren, wie Human-Machine-Teaming funktioniert: Der Mensch bleibt verantwortlich, die Maschine verstärkt Wahrnehmung, Reaktionsgeschwindigkeit und Ausdauer. Der Effekt ist unmissverständlich: Der Soldat *bedient* keine KI. Der Soldat *ist Teil eines KI-Systems*.

Was in der Verteidigung unter Druck entsteht, entsteht in Unternehmen unter Wettbewerb. Der Unterschied liegt allein in der Geschwindigkeit und nicht in der Richtung. Das Prinzip ist auf Unternehmen übertragbar:

- **Rollen verändern sich:** Fachkräfte entscheiden mit KI, nicht über KI.
- **Kompetenzen verschieben sich:** Prompting allein genügt nicht; Systemverständnis zählt.
- **Architektur wird biologisch:** Feedbackschleifen, Lernen, Echtzeit-Anpassung.

AI-to-DNA bedeutet auch, dass Onboarding KI-Kompetenz einschließt. Führung nutzt KI als Sparringspartner. Governance behandelt innerhalb klarer Leitplanken Maschinen als Mitentscheider.

Die Verantwortung des Architekten

Die Aufgabe des Architekten ist es, KI strukturell zu verankern. Architekten entwerfen Human-in-the-Loop-Systeme statt Werk-

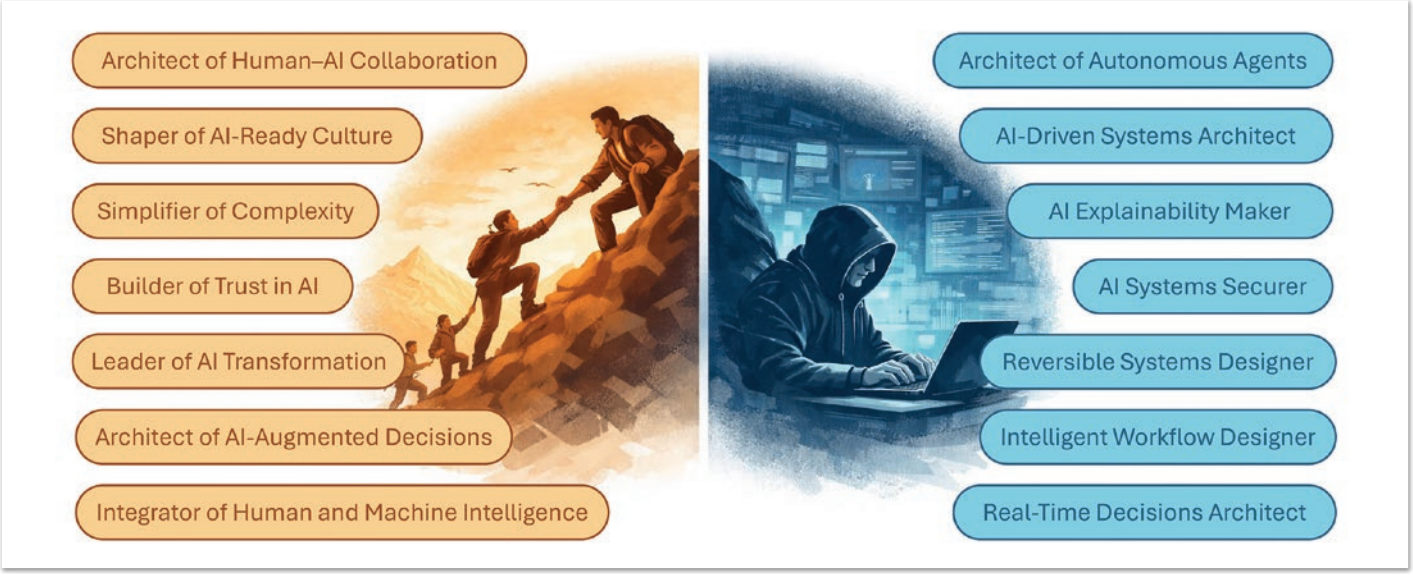


Abbildung 2: Rollen des Architekten im heutigen KI-Zeitalter (© Eldar Sultanow, Lars Santesson)

zeugketten. Sie betten Lernfähigkeit in Prozesse ein und sichern Rechenschaftspflicht trotz Automatisierung.

Wer KI nur integriert, optimiert die Gegenwart. Wer KI in die DNA schreibt, baut die Zukunft. Die Zukunft wird daher nicht von denen gebaut, die KI verstehen. Sie wird von denen gebaut, die Systeme entwerfen, in denen Mensch und KI untrennbar werden.

Wichtig: Damit ist nicht gemeint, dass der Architekt alle Fähigkeiten selbst besitzen muss. Der künftige Architekt ist kein Einzelexperte, sondern ein vernetzter Integrator. Er weiß, wie er die richtige Expertise zur richtigen Zeit mobilisiert, – Spezialisten verbindet, Perspektiven ausrichtet und Beiträge über organisatorische und disziplinäre Grenzen hinweg orchestriert. In diesem Sinne entwirft der Architekt neben Systemen auch ganze Kollaborationsstrukturen. Er schafft Umgebungen, in denen sich vielfältige Expertise wirksam kombinieren lässt, damit komplexe Herausforderungen durch kollektive Intelligenz statt durch individuelles Wissen gelöst werden.

Vertrauen gestalten: Kultur und Vorhersagbarkeit in der Mensch-Maschine-Kollaboration

Der künftige Architekt entwirft komplette Umgebungen, in denen Mensch und Maschine wirksam zusammenarbeiten können. Das verlangt mehr als technische Integration. Es verlangt die bewusste Schaffung eines kulturellen Raums, in dem Vertrauen in KI entstehen und dauerhaft wachsen kann.

Vertrauen entsteht nicht durch Leistung allein. Es entsteht durch Vorhersagbarkeit. Menschen müssen ein intuitives Verständnis dafür entwickeln, wie KI-Systeme sich verhalten, also wann sie zuverlässig sind, wo sie versagen und wie ihre Ergebnisse zu interpretieren sind. Ohne das bleiben selbst die stärksten Systeme ungenutzt oder, schlimmer, werden abgelehnt. Architekten verwandeln KI deshalb von einer Black Box in ein navigierbares System. Das heißt nicht, jedes technische Detail offenzulegen, sondern Verhalten verständlich, nachvollziehbar und konsistent zu machen. Muster müssen erkennbar, Entscheidungen erklärbar und Interaktionen verlässlich sein.

Zugleich prägen Architekten die kulturellen Bedingungen der KI-Nutzung. Sie definieren, wie Verantwortung geteilt wird, wie mit Fehlern umgegangen wird und wie Menschen die Kontrolle behalten, ohne das System auszubremsen. Dazu gehören Feedbackmechanismen, Eskalationspfade und Räume für menschliches Urteilsvermögen.

Das Ziel ist nicht Koexistenz, sondern Kollaboration. Mensch und Maschine dürfen nicht parallel arbeiten; sie müssen einander verstärken. Das erfordert Architekturen, die KI-Verhalten vorhersagbar genug machen, um ihm zu vertrauen, und flexibel genug, um sich anzupassen.

Skills und die Doppelnatur des neuen Architekten im KI-Zeitalter

Auf der einen Seite agiert er als **Leader**: Er führt Menschen, schafft Vertrauen und gestaltet ein Umfeld, in dem Mensch und KI wirksam zusammenarbeiten. Dazu gehört eine Kultur, die Ungewissheit annimmt, Lernen ermöglicht und Zuversicht in KI-gestützte Entscheidungen fördert. Praktisch zeigt sich das in Rollen wie *Architect of Human-AI Collaboration, Builder of Trust in AI und Shaper of AI-Ready Culture* (*Abbildung 2*, links). Der Architekt vereinfacht Komplexität, richtet Stakeholder aus und übersetzt zwischen Business und Technologie, damit KI nicht nur implementiert, sondern verstanden, akzeptiert und wirksam genutzt wird.

Auf der anderen Seite agiert er als **tiefer technischer Experte**: Er entwirft intelligente Systeme, orchestriert KI-Komponenten und stellt sicher, dass diese Systeme zuverlässig, beobachtbar und anpassungsfähig bleiben. Dazu gehören Rollen wie *Architect of AI-Driven Systems, Designer of Explainable AI und Architect of Observable AI Systems* (*Abbildung 2*, rechts). Eine Schlüsselfähigkeit ist das Entwerfen *reversibler Systeme*, also Systeme, die sich weiterentwickeln, anpassen und erholen können, ohne dass der Betrieb beeinträchtigt wird. Hinzu kommen Echtzeit-Entscheidungsfähigkeit, Sicherheit und Resilienz sowie modulare Plattformen, mit denen Organisationen KI skalieren, ohne die Kontrolle zu verlieren.

Der Wert entsteht an der Schnittstelle beider Dimensionen. Künftige Architekten wechseln nicht zwischen den Rollen, sie integrieren sie.

Am Ende wird Vertrauen zu einem Ergebnis von Architektur. Und der Architekt zu seinem Gestalter.

Kurz gesagt, vereint der künftige Architekt Führung und Technologie und macht KI zur organisatorischen Fähigkeit.

Fünf Handlungsempfehlungen

Organisationen müssen schließlich bewusst handeln, um in diesem neuen Paradigma wettbewerbsfähig zu bleiben:

1. **Modularität und Reversibilität verankern:** Systeme müssen sich weiterentwickeln können, ohne den Betrieb zu brechen.
2. **KI-Kompetenz in allen Rollen aufbauen:** nicht als technischen Skill, sondern als Kernfähigkeit für Entscheidungen und Zusammenarbeit.
3. **Governance-Frameworks implementieren**, die Transparenz, Rechenschaft und Vertrauen zwischen Mensch und Maschine sichern.
4. **Für kontinuierliches Lernen entwerfen:** Feedbackschleifen in Prozesse, Systeme und Organisationsstrukturen einbetten.
5. **Die Rolle des Architekten anheben:** vom technischen Experten zum strategischen Integrator, der Technologie, Menschen und Zweck ausrichtet.

Referenzen

[1] *https://www.linkedin.com/in/christophmagnussen*

Literatur

- AmnaBatool,DidarZowghi,MuneeraBano(2025).AIgovernance: a systematic literature review. *AI and Ethics*, 5(3), 3265–3279. *https://doi.org/10.1007/s43681-024-00653-w*
- DARPA (o. J.). *Exploratory Models of Human-AI Teams (EMHAT).* Defense Advanced Research Projects Agency. *https://www.darpa.mil/research/programs/exploratory-models-of-human-ai-teams*
- Alexander Ettinger (2025). *Enterprise Architecture as a Dynamic Capability for Scalable and Sustainable Generative AI adoption: Bridging Innovation and Governance in Large Organisations*. arXiv. *https://doi.org/10.48550/arXiv.2505.06326*
- Robyn Kozierok, John Aberdeen, Cheryl Clark, Christopher Garay, Bradley Goodman, Tonia Korves, Lynette Hirschman, Patricia L. McDermott, Matthew W. Peterson (2021). *Hallmarks of Human-Machine Collaboration: A framework for assessment in the DARPA Communicating with Computers Program*. arXiv. *https://doi.org/10.48550/arXiv.2102.04958*
- Bowen Long, Enjie Liu, Renxi Qiu, Yanqing Duan (2025). *Explainable AI: the Latest Advancements and New Trends*. arXiv:2505.07005. *https://arxiv.org/abs/2505.07005*
- Marcin Spoczynski, Marcela S. Melara, Sebastian Szyller (2025). *Atlas: A Framework for ML Lifecycle Provenance & Transparency*. arXiv:2502.19567. *https://arxiv.org/abs/2502.19567*
- Yanjun Wen, Jiale Wang, Xiaoxi Chen (2025). Trust and AI weight: human-AI collaboration in organizational management decision-making. *Frontiers in Organizational Psychology*, 3, 1419403. *https://doi.org/10.3389/forgp.2025.1419403*



Lars Santesso
Capgemini Deutschland GmbH
lars.santesson@capgemini.com

Lars Santesson ist Chief Technology Officer für den öffentlichen Sektor und einer der prägenden Enterprise-Architekten der deutschen Verwaltungsdigitalisierung. Seit mehr als zwanzig Jahren gestaltet er zentrale Transformationsvorhaben auf Bundes- und Landesebene, darunter das Onlinezugangsgesetz (OZG), die Registermodernisierung sowie aktuell die Initiative „G24 - Gemeinsam Schneller Gründen" zur Schaffung durchgängig digitaler Gründungsprozesse. Als Autor und Sprecher zu Themen wie Agentic AI, digitale Souveränität und moderne Verwaltungsarchitekturen verbindet er technologische Innovation mit der praktischen Umsetzung in Staat und Verwaltung.



Eldar Sultanow
Capgemini Deutschland GmbH
eldar.sultanow@capgemini.com

Dr. Eldar Sultanow ist IT-Stratege bei Capgemini, Lehrperson an der Universität Potsdam und Gutachter für das Deutsche Zentrum für Luft- und Raumfahrt (DLR). Der promovierte Wirtschaftsinformatiker blickt auf eine mehr als zwanzigjährige Erfahrung in der Softwareentwicklung zurück: von der Programmierung bis hin zum KI-Design.

Fairness not found: Wie neuronale Netze gesell-schaftliche Verzerrungen lernen

Dr. Sandra C. Signore





Neuronale Netze wirken objektiv, weil sie rechnen statt zu urteilen. Doch sie lernen aus Daten, Zielgrößen und Entscheidungen, die gesellschaftliche Ungleichheiten bereits enthalten können. Dieser Artikel zeigt anhand von bekannten und aktuellen Beispielen aus Medizin, Personalwesen und Gesichtserkennung, wie Bias entsteht, warum eine hohe Gesamtgenauigkeit darüber hinwegtäuschen kann sowie welche Verantwortung Entwicklerinnen und Entwickler im gesamten KI-Lebenszyklus tragen.

Wenn neutraler Code ungerechte Folgen hat

Der Java-Code kann sauber sein, die Tests grün leuchten und das Modell auf dem Testdatensatz eine beeindruckende Genauigkeit erreichen. Trotzdem kann das Gesamtsystem bestimmte Menschen systematisch benachteiligen. Genau darin liegt die Tücke von KI-Bias: Er sieht selten aus wie ein klassischer Programmierfehler. Es gibt keine Exception, keinen Stacktrace und häufig nicht einmal einen auffälligen Einbruch der durchschnittlichen Performance.

Bias bezeichnet zunächst eine systematische Verzerrung. Doch nicht jede Verzerrung ist automatisch diskriminierend: Jedes Modell simplifiziert die Wirklichkeit und bevorzugt bestimmte Muster. Problematisch wird Bias, wenn diese Verzerrung vorhersehbar zu schlechteren, unfairen oder gefährlichen Ergebnissen für bestimmte Gruppen führt. Das National Institute of Standards and Technology unterscheidet dabei systemische, statistisch-computationale und menschlich-kognitive Formen von Bias *[1]*. Diese Kategorien greifen ineinander. Gesellschaftliche Ungleichheiten prägen Daten, Daten prägen Modelle und Menschen interpretieren deren Ergebnisse wiederum durch ihre eigenen Erwartungen.

Die zentrale Botschaft lautet deshalb: Ein Modell ist nicht allein deshalb neutral, weil Alter, Geschlecht oder ethnische Zuordnung nicht als Eingabefeld vorkommen. Andere Merkmale können als Stellvertreter dienen. Postleitzahl, Berufslaufbahn, Wortwahl, Vorerkrankungen oder Nutzungskosten verraten oft indirekt genau jene Informationen, die man vermeintlich entfernt hat.

Neuronale Netze lernen Korrelationen, keine Gerechtigkeit

Neuronale Netze optimieren eine mathematische Zielfunktion. Sie suchen Muster, mit denen sich der Fehler auf den Trainingsdaten möglichst stark reduzieren lässt. Ob diese Muster medizinisch sinnvoll, gesellschaftlich erwünscht oder fair sind, ist in der Zielfunktion zunächst nicht enthalten.

Vor allem tiefe Netze können hochdimensionale Zusammenhänge entdecken, die Menschen nicht bewusst vorgegeben haben. Das ist ihre Stärke, aber zugleich ihr Risiko. Ein Bildklassifikator kann statt einer Krankheit unbemerkt Aufnahmeeinstellungen, Hauttöne, Kliniklogos oder typische Bildhintergründe nutzen. Ein Sprachmodell für Bewerbungen kann Stilmerkmale, Vereinsnamen oder Lücken im Lebenslauf als Signale verwerten. Das Netz erkennt nicht, dass eine Korrelation historisch oder sozial problematisch ist. Es erkennt nur, dass sie im Trainingsmaterial nützlich war.

Hinzu kommt der sogenannte Simplicity Bias: Tiefe Netze greifen häufig bevorzugt auf einfache, leicht verfügbare Merkmale zurück, selbst wenn robustere und generalisierbarere Zusammenhänge vorhanden wären *[2]*. Ein Modell, das auf dem Trainingsdatensatz hervorragend funktioniert, kann daher in einer anderen Klinik, Region oder Bevölkerungsgruppe plötzlich versagen. Nicht weil es plötzlich „bösartig" geworden ist, sondern weil die zuvor genutzte Abkürzung nicht mehr trägt.

Bias entsteht jedoch nicht ausschließlich während des Modelltrainings. *Abbildung 1* zeigt, wie gesellschaftliche Ungleichheiten, verzerrte Messverfahren, ungeeignete Zielgrößen und technische Entwicklungsentscheidungen über den gesamten KI-Lebenszyklus ineinandergreifen. Die Vorhersagen eines Modells können anschließend menschliche Entscheidungen und zukünftige Daten beeinflussen und bestehende Verzerrungen weiter verstärken. Ansatzpunkte für Gegenmaßnahmen bestehen deshalb vor dem Training, während Entwicklung und Evaluation sowie nach dem Deployment.

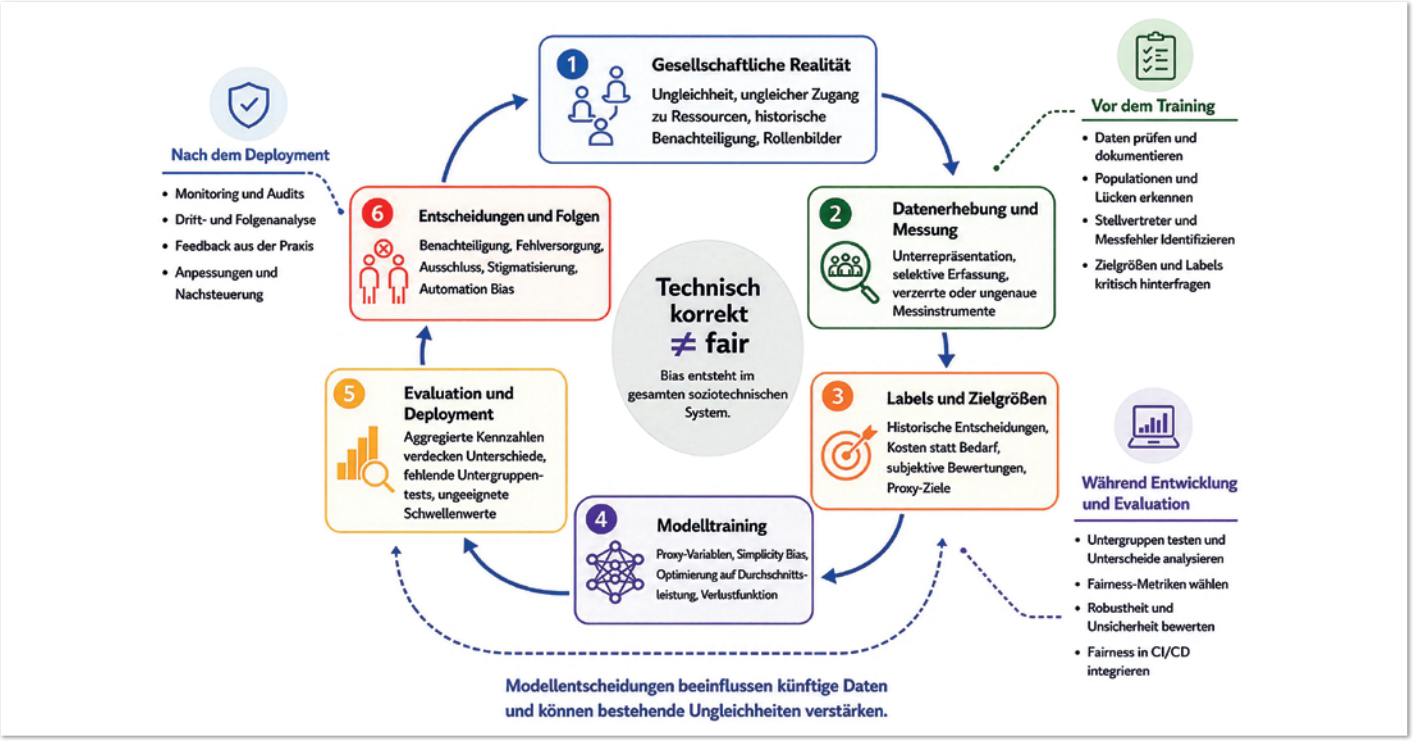


Abbildung 1: Bias entlang des KI-Lebenszyklus. Gesellschaftliche Ungleichheiten können über Datenerhebung, Zielgrößen, Modelltraining und Evaluation in automatisierte Entscheidungen eingehen und durch Rückkopplung weiter verstärkt werden. Gegenmaßnahmen sind daher vor, während und nach der Modellentwicklung erforderlich. Inhaltliches Konzept und redaktionelle Bearbeitung: Sandra C. Signore; Visualisierung mit Unterstützung generativer KI.

Fokus Personalwesen: Wenn historische Verzerrungen mitentscheiden

Ein bekanntes Beispiel stammt aus dem Personalwesen. Amazon entwickelte ab 2014 ein experimentelles System, das Bewerbungen für technische Stellen bewerten sollte. Trainiert wurde es mit Lebensläufen aus einem Zeitraum von rund zehn Jahren. Da in der Technologiebranche deutlich mehr Männer vertreten waren, enthielten auch die historischen Bewerbungen überwiegend männliche Profile.

Das System lernte daraus nicht lediglich, welche Qualifikationen historisch erfolgreich gewesen waren. Es lernte zugleich, dass männlich geprägte Lebensläufe dem historischen Ideal entsprachen. Nach Recherchen von Reuters wurden unter anderem Formulierungen mit dem Wort „women's", beispielsweise bei einem Frauen-Schachclub, abgewertet *[3]*. Einzelne Begriffe ließen sich zwar entfernen, das Grundproblem blieb jedoch bestehen: Ein komplexes Modell kann neue Stellvertreter finden, solange das Lernziel und die Daten dieselbe Vergangenheit abbilden.

Das Beispiel zeigt, warum „Wir haben das Geschlecht entfernt" keine ausreichende Fairness-Strategie ist. Das Modell kann Geschlecht aus anderen Merkmalen rekonstruieren oder Strukturen übernehmen, die mit Geschlecht zusammenhängen. Wer historische Einstellungsentscheidungen vorhersagt, automatisiert möglicherweise nicht die beste Auswahl, sondern die bisherige Auswahl.

Fokus Medizin: Wenn Kosten zum Maß für Krankheit werden

Besonders folgenreich wird Bias in der Medizin. Eine viel beachtete Untersuchung analysierte einen in den USA verbreiteten Algorithmus zur Identifikation von Patientinnen und Patienten mit erhöhtem Versorgungsbedarf. Das System nutzte Behandlungskosten als Stellvertreter für Krankheitsschwere. Diese Zielgröße wirkte zunächst plausibel: Diese Zielgröße wirkte zunächst plausibel: Wer schwerer erkrankt ist, verursacht häufig höhere Kosten.

Doch Gesundheitskosten spiegeln nicht nur Krankheit wider. Sie hängen auch davon ab, wer Zugang zu Versorgung erhält, wie früh Erkrankungen erkannt werden und welche Leistungen tatsächlich in Anspruch genommen werden können. Bei gleichem Risikoscore war die tatsächliche Krankheitslast bei als Schwarz kategorisierten Patientinnen und Patienten im Durchschnitt höher als bei als weiß kategorisierten Patientinnen und Patienten. Der Algorithmus unterschätzte ihren Bedarf, weil für sie historisch weniger Geld ausgegeben worden war *[4]*.

Hier lag der Bias nicht primär in einer einzelnen Modellschicht. Er steckte in der Definition dessen, was vorhergesagt werden sollte. Das Modell optimierte erfolgreich auf Kosten, aber Kosten waren nicht dasselbe wie medizinischer Bedarf. Ein technisch gutes Modell beantwortete äußerst präzise die falsche Frage.

Wenn der Messfehler zum Trainingssignal wird

Auch Eingangsdaten können bereits verzerrt sein, bevor ein neuronales Netz sie verarbeitet. Pulsoxymeter schätzen die Sauerstoffsättigung anhand der Lichtabsorption im Gewebe. Eine Studie zeigte, dass gefährlich niedrige arterielle Sauerstoffsättigungen bei der in den Patientendaten als „Black" erfassten Gruppe deutlich häufiger unentdeckt blieben als bei der als „White" erfassten Vergleichsgruppe *[5]*. Das Pulsoxymeter selbst ist zwar kein neuronales Netz. Das Beispiel verdeutlicht jedoch ein grundlegendes Problem: Wird ein verzerrtes Messsignal als Ground Truth, also als vermeintlich korrekter Referenzwert, oder als Feature verwendet, kann ein nachgelagertes KI-System diese Verzerrung übernehmen und weiter verstärken.

Ähnliches gilt für medizinische Bilddaten. Dermatologische KI erreicht bei der Klassifikation von Hautveränderungen beeindruckende Ergebnisse, doch viele Datensätze dokumentieren Hauttöne, Herkunft, Auswahlkriterien oder Aufnahmebedingungen jedoch nur unzureichend. Aktuelle Arbeiten berichten, dass Modelle auf dunkleren Hauttönen schlechter abschneiden können, wenn diese im Trainingsmaterial unterrepräsentiert sind *[6]*. Ein Convolutional Neural Network kann nur aus dem lernen, was man ihm zeigt. Eine selten vertretene Gruppe bleibt mathematisch selten – auch wenn ihre klinische Bedeutung keineswegs geringer ist.

Das führt zu einer unbequemen Wahrheit: Mehr Daten sind nicht automatisch bessere Daten. Eine Million nahezu gleicher Aufnahmen aus wenigen spezialisierten Zentren können weniger generalisierbar sein als ein kleinerer, sorgfältig dokumentierter und vielfältiger Datensatz.

Fokus Gesellschaft: Wenn Durchschnittswerte Diskriminierung verdecken

Gesichtserkennung illustriert, wie Durchschnittswerte Unterschiede verbergen. Die Studie „Gender Shades" untersuchte kommerzielle Systeme zur Geschlechtsklassifikation anhand von Gesichtern. Die Fehlerraten waren für hellhäutige Männer sehr niedrig, für dunkelhäutige Frauen jedoch erheblich höher *[7]*. Ein einziger Gesamtwert hätte diese Diskrepanz leicht kaschiert.

Das ist nicht lediglich ein kosmetisches Qualitätsproblem. Werden solche Systeme für Zugangskontrollen, Identitätsprüfungen oder polizeiliche Ermittlungen eingesetzt, verteilt sich das Risiko von Fehlentscheidungen ungleich. Eine falsche Klassifikation bedeutet dann nicht bloß ein unpassendes Label, sondern möglicherweise zusätzliche Kontrollen, verweigerten Zugang oder einen falschen Verdacht.

Fairness ist deshalb immer eine Frage des Anwendungskontexts. Derselbe Prozentpunkt Fehlerquote hat bei einer Foto-App andere Folgen als bei einer medizinischen Diagnose oder einer Entscheidung über Beschäftigung.

Gesamtgenauigkeit ist nicht gleich Fairness

Viele Teams bewerten Modelle zunächst über Accuracy, Precision, Recall oder den F1-Score. Diese Kennzahlen sind wichtig, aber ein aggregierter Wert beantwortet nicht, wie sich Fehler verteilen. Ein Modell kann insgesamt 92 % Genauigkeit erreichen und gleichzeitig für eine kleine Gruppe nur 65 %.

Deshalb sollten Metriken mindestens nach relevanten Untergruppen ausgewertet werden. Welche Gruppen relevant sind, ergibt sich nicht allein aus der Datenstruktur, sondern aus dem Einsatzgebiet und einer vorherigen Risikoanalyse. Neben Geschlecht, Alter oder Hauttyp können auch Klinikstandort, Gerätetyp, Sprache, sozioökonomischer Kontext oder Kombinationen mehrerer Merkmale entscheidend sein.

```
record CaseResult(String group, boolean expected, boolean predicted) {}

record GroupMetrics(long total, long truePositive, long falseNegative) {
     double recall() {
           long positives = truePositive + falseNegative;
           return positives == 0 ? Double.NaN
                                 : (double) truePositive / positives;
     }
}

Map<String, GroupMetrics> metricsByGroup =
     results.stream().collect(Collectors.groupingBy(
           CaseResult::group,
           Collectors.collectingAndThen(Collectors.toList(), cases -> {
                 long tp = cases.stream()
                       .filter(c -> c.expected() && c.predicted()).count();
                 long fn = cases.stream()
                       .filter(c -> c.expected() && !c.predicted()).count();
                 return new GroupMetrics(cases.size(), tp, fn);
           })
     ));

metricsByGroup.forEach((group, metrics) ->
     System.out.printf("%s: Recall %.2f%n", group, metrics.recall()));
```

Listing 1: Gruppenbezogene Auswertung des Recalls statt einer einzigen Gesamtkennzahl

Listing 1 zeigt bewusst einfach, wie ein Java-Service Ergebnisse gruppenspezifisch auswerten kann. Der Code löst Fairness zwar nicht, er verhindert aber, dass Unterschiede hinter einem freundlichen Durchschnitt verschwinden *(siehe Listing 1)*.

Wenn Fairness mehrere Wahrheiten hat

Schon die Frage „Ist das Modell fair?" bleibt ohne weitere Präzisierung unvollständig. Denn unterschiedliche Fairnessbegriffe setzen unterschiedliche Schwerpunkte und können sich sogar widersprechen. Demographic Parity verlangt beispielsweise, dass positive Entscheidungen in unterschiedlichen Gruppen ähnlich häufig vorkommen. Equal Opportunity betrachtet dagegen, ob tatsächlich positive Fälle in allen Gruppen vergleichbar zuverlässig erkannt werden. Predictive Parity fragt schließlich, ob ein positives Modellergebnis gruppenübergreifend dieselbe Aussagekraft besitzt.

Welche Definition angemessen ist, hängt vom Schaden ab, den man vermeiden möchte. In einem Screening kann es besonders wichtig sein, Erkrankte nicht zu übersehen. Bei einer knappen Ressource kann zusätzlich relevant sein, wie Fehlalarme verteilt sind. Entwicklerinnen und Entwickler sollten diese Entscheidung daher nicht allein treffen. Sie benötigen Fachwissen aus Medizin, Recht, Ethik, Produktentwicklung und von den betroffenen Gruppen selbst.

Fairness bedeutet außerdem nicht, jede Kennzahl zwanghaft identisch zu machen. Gruppen können unterschiedliche Ausgangsrisiken, Datenqualitäten oder Versorgungssituationen aufweisen. Eine mathematisch gleiche Behandlung kann bestehende Ungleichheit fortschreiben. Das NIST warnt deshalb davor, die Reduktion messbarer Verzerrungen bereits mit vollständiger Fairness gleichzusetzen *[1]*.

Bias reduzieren: An welcher Stelle greift man ein?

Gegenmaßnahmen lassen sich grob an drei Stellen ansetzen beziehungsweise implementieren. Vor dem Training können Datensätze ergänzt, neu gewichtet, dokumentiert oder gezielt nach problematischen Stellvertretern untersucht werden. Während des Trainings können Fairness-Ziele, robuste Optimierung oder angepasste Loss-Funktionen eingesetzt werden. Nach dem Training lassen sich Schwellenwerte, Kalibrierung, menschliche Freigaben oder eine Reject Option, bei der das Modell in unsicheren Fällen keine Entscheidung trifft, ergänzen.

Keine dieser Maßnahmen ist ein Zauberfilter. Rebalancing kann die Performance einzelner Gruppen verbessern, zugleich aber Unsicherheitsschätzungen verschlechtern. Eine aktuelle Untersuchung medizinischer Deep-Learning-Systeme zeigt genau diesen Zielkonflikt: Fairness-Maßnahmen reduzierten in einigen Aufgaben Performanceunterschiede, führten jedoch teilweise zu weniger verlässlichen Unsicherheitswerten *[8]*.

Ein System ist also nicht automatisch vertrauenswürdig, nur weil eine Fairness-Metrik besser aussieht. Es muss weiterhin wissen dürfen, wann eine Vorhersage unsicher ist. Besonders in der Medizin sollte ein Modell bei unbekannten oder schlecht repräsentierten Fällen nicht selbstbewusst raten, sondern die Entscheidung an einen Menschen zurückgeben.

Was Java-Teams tun können

Fairness beginnt lange vor dem ersten Model-Endpoint und endet nicht mit dem Deployment. Ein belastbarer Prozess umfasst mindestens folgende Schritte:

1. Zweck und Schaden definieren: Welche Entscheidung unterstützt das System? Wer trägt die Folgen eines Fehlers? Welche Fehlerart ist besonders kritisch?
2. Datenherkunft dokumentieren: Aus welchen Populationen, Zeiträumen, Geräten und Institutionen stammen die Daten? Wer fehlt? Welche Labels wurden von wem und nach welchen Regeln vergeben?
3. Stellvertreter prüfen: Kann das Modell geschützte oder sensible Eigenschaften indirekt aus anderen Features ableiten? Werden Kosten, Nutzung oder historische Entscheidungen fälschlich als objektive Wahrheit behandelt?
4. Untergruppen testen: Gesamtmetriken genügen nicht. Performance, Kalibrierung und Fehlertypen müssen für relevante Gruppen und sinnvolle Schnittmengen ausgewertet werden.

5. Unsicherheit berücksichtigen: Ist das Modell bei seinen Fehlern wenigstens unsicher? Gibt es Schwellen, bei denen es keine automatische Entscheidung trifft?
6. Menschen sinnvoll einbinden: „Human in the loop" darf nicht bedeuten, dass eine überlastete Person jede Modellentscheidung nur noch abnickt. Hinzu kommt der sogenannte Automation Bias: Menschen neigen dazu, Empfehlungen automatisierter Systeme übermäßig zu vertrauen und eigene Zweifel oder widersprechende Informationen zurückzustellen. Rollen, Eskalationswege und Entscheidungsspielräume müssen deshalb real und klar definiert sein.
7. Nach dem Go-live überwachen: Populationen, Geräte und Prozesse können sich im Laufe der Zeit verändern. Fairness ist keine Eigenschaft, die einmal bewertet, zertifiziert und anschließend vergessen werden kann.

Technisch lohnt es sich, Fairness-Checks in dieselbe CI/CD-Logik aufzunehmen wie Performance- und Sicherheitstests. Ein Deployment sollte scheitern können, wenn gruppenspezifische Mindestwerte unterschritten werden oder sich die Differenz zwischen Gruppen deutlich verschlechtert. Dabei müssen Schwellenwerte versioniert, fachlich begründet und im Monitoring sichtbar bleiben.

Warum das Thema jetzt besonders relevant ist

KI skaliert Entscheidungen. Genau das macht sie attraktiv, unter Umständen aber auch gefährlich. Ein einzelner Mensch kann voreingenommen entscheiden; ein automatisiertes System kann dieselbe Verzerrung potenziell tausendfach pro Stunde reproduzieren, dabei konsistent wirken und seine Ergebnisse mit einem präzisen Score versehen.

Zugleich werden KI-Systeme zunehmend in Bereiche eingebaut, in denen Gesundheit, Bildung, Arbeit und gesellschaftliche Teilhabe betroffen sind. Der europäische AI-Act stuft bestimmte KI-Anwendungen in der Medizin und im Beschäftigungsbereich als Hochrisiko-Systeme ein und verlangt unter anderem Risikomanagement, Datenqualität, Dokumentation, menschliche Aufsicht und Überwachung *[9]*. Fairness ist damit nicht mehr nur ein freiwilliges Ethik-Kapitel für die Projektpräsentation. Sie wird Teil professioneller Entwicklungs- und Governanceprozesse.

Für Entwicklerinnen und Entwickler bedeutet das keine persönliche Schuld an allen gesellschaftlichen Ungleichheiten. Es bedeutet jedoch Verantwortung für die Frage, welche Ungleichheiten ein System übernimmt, verstärkt oder unsichtbar macht. Der Satz „Das Modell hat so entschieden" ist keine neutrale Erklärung. Das Modell wurde mit einem Ziel, Daten, Schwellenwerten und einem konkreten Einsatzkontext versehen.

Fazit: Fairness muss entwickelt werden

Neuronale Netze besitzen kein eigenes Verständnis von Gerechtigkeit. Sie lernen, welche Muster in ihren Daten nützlich sind. Sind Vergangenheit, Messverfahren oder Zielgrößen verzerrt, kann ein Modell diese Verzerrungen mit beeindruckender Präzision automatisieren.

Bias lässt sich daher weder durch das Entfernen eines einzelnen Datenfeldes noch durch einen zusätzlichen Unit-Test beseitigen. Er entsteht im gesamten soziotechnischen System: in den Daten, im Modell, in der Software, in organisationalen Prozessen und in der Entscheidung, die am Ende auf Grundlage des Systems getroffen wird.

Die entscheidende Entwicklerfrage lautet deshalb nicht nur: „Wie genau ist unser Modell?" Sondern auch: „Für wen ist es genau – und wer trägt die Folgen seiner Fehler?" Erst wenn diese Frage Teil von Architektur, Testing und Monitoring wird, kann aus funktionierender KI verantwortbare KI werden.

Oder, in Java gesprochen: Fairness ist kein Default-Wert. Sie muss explizit modelliert, getestet und über den gesamten Lebenszyklus gepflegt werden.

Quellen

[1] National Institute of Standards and Technology (2023): Artificial Intelligence Risk Management Framework (AI RMF 1.0). NIST AI 100-1, Gaithersburg.

[2] Tiwari, R.; Shenoy, P. (2023): Overcoming Simplicity Bias in Deep Networks using a Feature Sieve. Proceedings of the 40th International Conference on Machine Learning, PMLR 202.

[3] Dastin, J. (2018): Amazon scraps secret AI recruiting tool that showed bias against women. Reuters, 10. Oktober 2018.

[4] Obermeyer, Z.; Powers, B.; Vogeli, C.; Mullainathan, S. (2019): Dissecting racial bias in an algorithm used to manage the health of populations. Science 366(6464), S. 447–453.

[5] Sjoding, M. W.; Dickson, R. P.; Iwashyna, T. J.; Gay, S. E.; Valley, T. S. (2020): Racial Bias in Pulse Oximetry Measurement. New England Journal of Medicine 383, S. 2477–2478.

[6] Li, Y. et al. (2025): Improving dataset transparency in dermatologic Artificial Intelligence using a dataset nutrition label. npj Digital Medicine 8, Artikel 641.

[7] Buolamwini, J.; Gebru, T. (2018): Gender Shades: Intersectional Accuracy Disparities in Commercial Gender Classification. Proceedings of Machine Learning Research 81, S. 77–91.

[8] Mehta, R.; Shui, C.; Arbel, T. (2024): Evaluating the Fairness of Deep Learning Uncertainty Estimates in Medical Image Analysis. Proceedings of Machine Learning Research 227, S. 1453–1492.

[9] Europäische Union (2024): Verordnung (EU) 2024/1689 zur Festlegung harmonisierter Vorschriften für künstliche Intelligenz (AI Act). Amtsblatt der Europäischen Union.



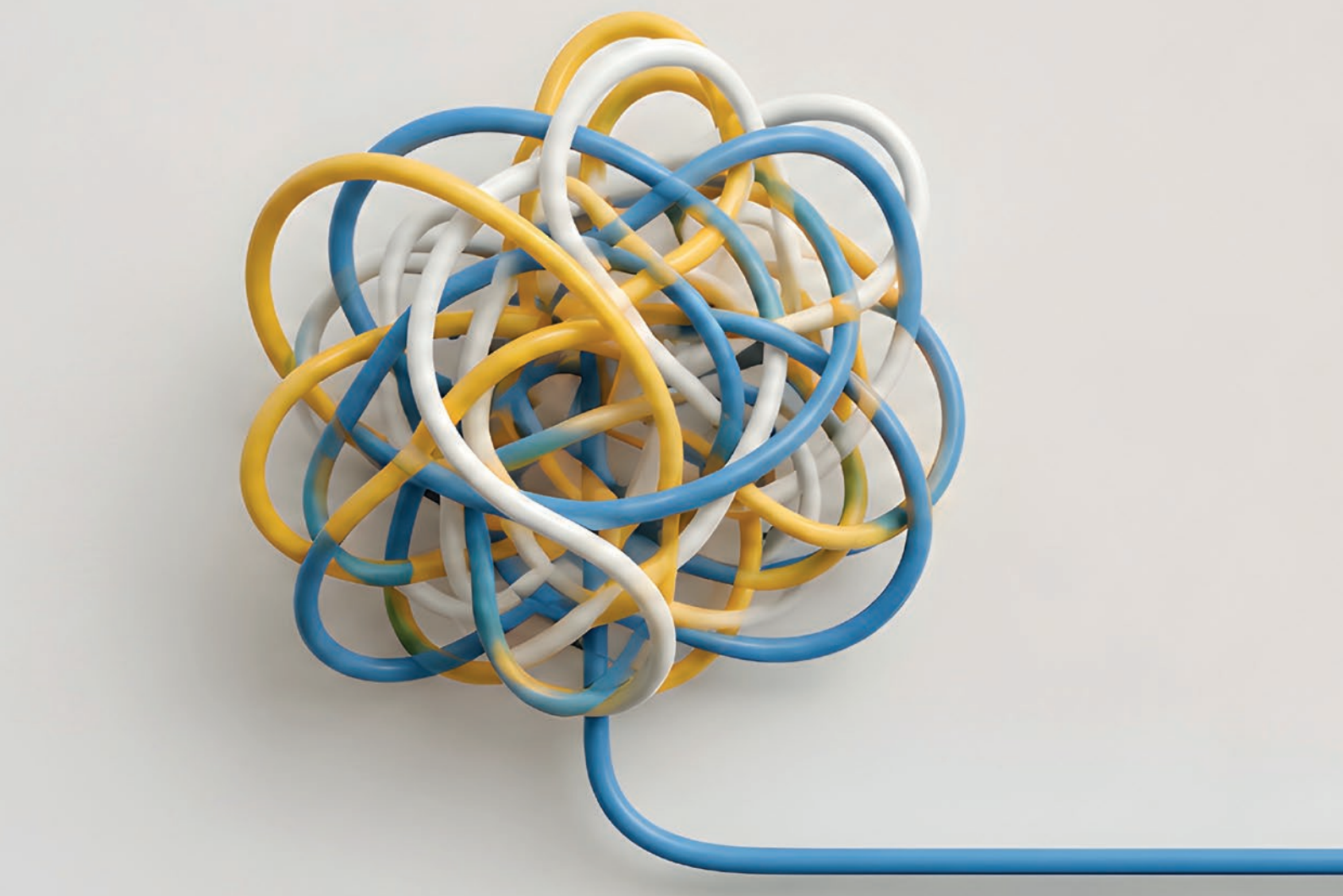
Dr. Sandra C. Signore

s.signore@online.de

Dr. Sandra C. Signore ist Neurologin und freiberufliche Beraterin für Medical Affairs und Marketing in der Healthcare-Branche. Sie arbeitet an der Schnittstelle von Medizin, Ethik und Technologie und beschäftigt sich intensiv mit Künstlicher Intelligenz und deren Anwendung in regulierten Bereichen. Ihr besonderes Interesse gilt dem Zusammenspiel von neuronalen Netzen, maschinellem Lernen und menschlicher Intuition. Sie setzt sich für verantwortungsvolle, erklärbare und empathisch gestaltete KI-Systeme ein, die Wissenschaft und Menschlichkeit verbindet.

Zwischen Frust und Verantwortung: Warum gute Entwickler sich ohnmächtig fühlen – und wie sie Wirksamkeit zurückgewinnen

Christian Seifert, gofore





Viele von uns kennen den Frust, der entsteht, wenn wir wissen, wie gute Software gebaut werden sollte, es im Projektalltag aber immer wieder nicht umsetzen können. Entscheidungen werden unter Zeitdruck getroffen, Qualität wird zur Verhandlungsmasse, und zunehmend entsteht das Gefühl, nur noch auszuführen statt zu gestalten. Dieser Text beschreibt, woher dieser Frust kommt, warum er nichts mit fehlender Kompetenz zu tun hat und wie sich Verantwortung schleichend aus der Entwicklung zurückzieht. Gleichzeitig zeigt er, warum gutes Handwerk allein nicht ausreicht und wie wir als Entwickler unsere Rolle neu begreifen können: weg von passiver Ohnmacht, hin zu aktiver Gestaltung, klarer Verantwortung und spürbarer Wirksamkeit im Zusammenspiel mit der Organisation und anderen Stakeholdern.

Wenn Frust zur Gewohnheit wird: Der Rückzug aus Verantwortung im Entwicklungsalltag

Viele von uns Softwareentwicklern tragen einen Gedanken mit sich, der selten offen ausgesprochen wird, aber umso schwerer wiegt: „Ich würde es gerne wenigstens einmal richtig machen."

Dieser Satz ist schmerzhafter, als er auf den ersten Blick klingt. Denn er impliziert bereits, dass „richtig machen" im aktuellen Arbeitsumfeld offenbar nicht möglich ist. Nicht, weil uns Wissen, Erfahrung oder handwerkliche Fähigkeiten fehlen würden – sondern weil die Art, wie Software entwickelt wird, unseren eigenen professionellen Anspruch immer wieder aushebelt. Der Schmerz liegt nicht im Wunsch nach Perfektion, sondern in der Erfahrung, dauerhaft unter den eigenen Standards zu arbeiten und dabei zusehen zu müssen, wie genau das langsam normal wird.

Dieser Frust entsteht selten durch ein einzelnes Ereignis. Er speist sich aus vielen kleinen Situationen, die sich über Zeit aufsummieren. Entscheidungen werden vor allem getroffen, um Termine zu halten. Technische Schulden werden bewusst akzeptiert, ohne dass ihre langfristigen Folgen wirklich Teil der Diskussion sind. Tests, Refactorings oder architektonische Leitplanken werden „für dieses eine Release" verschoben. Was ursprünglich als Ausnahme gedacht war, wird zur Routine. Irgendwann weiß man nicht mehr, wann zuletzt bewusst entschieden wurde – nur noch, dass man „keine Zeit dafür hatte".

Besonders zermürbend ist dabei ein wiederkehrendes Gefühl: Verantwortung tragen zu sollen, ohne wirklich gestalten zu dürfen. Wir erleben, dass wir für Stabilität, Wartbarkeit und Qualität verantwortlich gemacht werden, gleichzeitig aber kaum Einfluss auf die Entscheidungen haben, die genau diese Eigenschaften bestimmen. Wir sollen liefern – schnell, verlässlich, planbar – und uns im Zweifel später für Konsequenzen rechtfertigen, die wir fachlich kommen sehen, aber nicht verhindern konnten.

In so einem Umfeld beginnt oft ein schleichender Rückzug aus Verantwortung. Nicht aus Gleichgültigkeit oder mangelndem Anspruch, sondern als Anpassungsreaktion. Wer wiederholt erlebt, dass fachliche Argumente ins Leere laufen oder als „zu theoretisch" oder „zu langsam" wahrgenommen werden, konzentriert sich irgendwann auf das, was sich noch kontrollieren lässt. Das nächste Ticket. Der eigene Code. Die formale Umsetzung der Anforderungen. Fragen nach langfristigen Auswirkungen oder strukturellen Entscheidungen werden nicht mehr aktiv eingefordert – nicht, weil sie unwichtig wären, sondern weil man gelernt hat, dass sie im Prozess keinen echten Platz haben.

Damit entsteht ein Entscheidungsvakuum. Technische Fragen mit weitreichenden Folgen werden nicht mehr klar adressiert oder entschieden. Und wo Entscheidungen ausbleiben, entsteht Stillstand. Stillstand wiederum ist für Organisationen selten akzeptabel. Also passiert etwas sehr Menschliches – und meist nichts Böswilliges: Andere übernehmen die Entscheidung. Projektmanagement oder Führungskräfte treffen Festlegungen, nicht weil sie glauben, es fachlich besser zu wissen, sondern weil Entscheidungen getroffen werden müssen, damit es weitergeht. Die innere Logik ist nachvollziehbar:

„Ich bin nicht der Spezialist, aber bevor wir uns festfahren, entscheide ich lieber etwas, als gar nichts zu entscheiden."

Für uns fühlt sich das wie Fremdbestimmung an. Für das Management fühlt es sich wie notwendige Führung an. Beides kann gleichzeitig wahr sein – und genau daraus entsteht ein großer Teil der Spannung, die viele Entwicklungsprojekte prägt.

Wie weit diese Dynamik gehen kann, zeigt sich besonders deutlich bei konkreten Eingriffen ins Handwerk. Ein Beispiel, das ich in ähnlicher Form mehrfach erlebt habe: Ein Entwickler erklärte mir, er habe bewusst keine Tests für seine Implementierung geschrieben, weil aus dem Projektmanagement signalisiert worden sei, Tests seien aktuell „nicht so wichtig" und könnten – oder sollten sogar – weggelassen werden. Aus organisatorischer Sicht war das vermutlich als pragmatische Maßnahme gedacht, um Tempo zu machen oder einen Termin zu halten.

Dieses Beispiel macht zwei Dinge sehr deutlich. Zum einen ist es eine Grenzüberschreitung, wenn fachfremde Rollen so tief ins Handwerk eingreifen, dass implizit vorgegeben wird, auf Tests zu verzichten. Zum anderen – und das wiegt schwerer – zeigt es den Rückzug aus Verantwortung auf Entwickerseite. Denn nur wir können seriös beurteilen, welche Tests notwendig sind, welches Risiko ein Verzicht bedeutet und welche Folgen daraus entstehen. Wird eine solche Entscheidung stillschweigend akzeptiert, wird Verantwortung nicht nur abgegeben, sondern aktiv aus der eigenen Rolle herausgelöst.

Genau hier schließt sich der Kreis. Management entscheidet nicht, weil Entwicklung falsche Entscheidungen trifft, sondern weil Ent-

wicklung Entscheidungen nicht sichtbar trifft oder nicht nachvollziehbar vertritt. Diese Entscheidungen wirken dann wie „von oben verordnet", obwohl sie aus einem Vakuum heraus entstanden sind. Für uns verstärkt sich das Gefühl, fremdbestimmt zu sein. Für die Organisation bestätigt sich der Eindruck, dass Entwicklung nicht entscheidungsfähig ist. Der Frust wächst auf beiden Seiten.

All das ist kein individuelles Versagen und keine persönliche Schwäche. Es beschreibt ein Muster, das in vielen professionellen Entwicklungsumgebungen leicht entsteht. Ein Muster, in dem sich Frust, Rückzug und Fremdentscheidung gegenseitig verstärken – und das den Satz „Ich würde es gerne wenigstens einmal richtig machen" zu einem bitteren Kommentar über den eigenen Arbeitsalltag werden lässt.

Warum professionelle Softwareentwicklung so oft missverstanden wird

Der Zustand, den wir im ersten Kapitel beschrieben haben, ist kein Zufall. Er ist auch kein individuelles Problem einzelner Teams oder Personen. Er ist die logische Folge davon, dass professionelle Softwareentwicklung in vielen Organisationen – und nicht selten auch von uns selbst – falsch verstanden wird. Dieses Missverständnis wirkt leise, aber nachhaltig. Es erklärt, warum Frust entsteht, warum Verantwortung wandert und warum Entscheidungen am Ende dort getroffen werden, wo sie fachlich eigentlich nicht hingehören.

Ein zentrales Missverständnis betrifft unsere Rolle im Gesamtprozess. Softwareentwicklung wird häufig als Ausführungsdisziplin gesehen: Ziele werden definiert, Anforderungen priorisiert, Termine gesetzt – und dann soll Entwicklung liefern. Qualität gilt dabei als etwas, das „gute Entwickler automatisch mitbringen". Technische Entscheidungen werden als Detailfragen betrachtet: wichtig, ja – aber bitte nicht auf der Ebene, auf der über Budget, Zeit oder Richtung entschieden wird.

In diesem Bild sind wir verantwortlich für das Wie, nicht für das Ob oder das Warum. Verantwortung für Abwägungen, Zielkonflikte und Prioritäten liegt vermeintlich woanders. Dass genau diese Abwägungen tief in technische Entscheidungen hineinreichen, bleibt oft unsichtbar. Architektur, Tests oder technische Schulden werden getrennt von den Zielen diskutiert, die sie eigentlich absichern sollen.

Dieses Bild wird jedoch nicht nur von Organisationen getragen. Auch wir Entwickler tragen unseren Teil dazu bei. Viele von uns definieren ihre Professionalität primär über ihr Handwerk: sauberer Code, durchdachte Architektur, solide Tests. Das ist ein zentraler Teil guter Softwareentwicklung – aber eben nicht der ganze Job. Problematisch wird es dort, wo sich dieses Selbstverständnis ausschließlich auf das Technische verengt. Wer sich selbst nur als technischer Spezialist begreift, überlässt Entscheidungen über Prioritäten, Zielkonflikte und Risiken anderen Rollen. Oft begleitet von dem stillen Gedanken: „Das kann und wird so nicht funktionieren."

Damit entsteht ein paradoxer Zustand. Wir ärgern uns über Entscheidungen, an deren Zustandekommen wir selbst nicht aktiv beteiligt waren. Verantwortung wird eingefordert, aber nicht konsequent übernommen. Nach außen halten wir uns raus, innerlich wissen wir es besser. Genau aus dieser Spannung speist sich ein großer Teil des Frusts.

Verschärft wird dieses Missverständnis durch die Art, wie wir kommunizieren. Technische Notwendigkeiten werden häufig nicht als Entscheidungen mit klaren Konsequenzen formuliert, sondern als vage Forderungen. Sätze wie „Das müssen wir sauber machen", „Dafür brauchen wir mehr Zeit" oder „Das ist technisch notwendig" bleiben ohne Kontext. Für andere Stakeholder ist nicht erkennbar, welches Risiko besteht, welche Alternativen es gibt oder welche Folgen bestimmte Entscheidungen haben. Was für uns offensichtlich ist, wirkt nach außen schnell wie eine persönliche Präferenz oder ein übertriebener Qualitätsanspruch.

Wo Entscheidungen nicht nachvollziehbar begründet werden, verlieren sie an Gewicht. Und wo Gewicht fehlt, entsteht ein Vakuum. Dieses Vakuum bleibt in Projekten selten leer. Es wird gefüllt – nicht aus Machtstreben, sondern aus Handlungsdruck. Management trifft dann Entscheidungen nicht, weil es das Handwerk besser beurteilen könnte, sondern weil Entscheidungen getroffen werden müssen, damit es weitergeht. Aus organisatorischer Sicht ist das rational. Aus Entwicklersicht fühlt es sich wie ein Eingriff an.

Besonders deutlich wird dieses Missverständnis dort, wo handwerkliche Grundlagen plötzlich verhandelbar werden. Wenn darüber diskutiert wird, ob Tests „in diesem Fall" verzichtbar seien oder ob technische Schulden „vorübergehend" akzeptiert werden können, ohne klar zu benennen, welches Risiko damit eingegangen wird. Was für uns eine fachliche Basis ist, erscheint anderen als optionaler Luxus. Nicht, weil Qualität unwichtig wäre, sondern weil ihre Bedeutung nicht in eine gemeinsame Sprache übersetzt wurde.

Hinzu kommt ein weiterer Punkt, der oft unterschätzt wird: die Vermischung von Hobby- und Berufsperspektive. Viele von uns sind aus Begeisterung für Technik in diesen Beruf gekommen. In eigenen Projekten entscheiden wir selbst über Zeit, Scope und Qualität. Diese Erfahrung prägt unseren Anspruch – kollidiert im professionellen Umfeld jedoch mit anderen Realitäten. Dort ist Software kein Selbstzweck, sondern dient einem übergeordneten Zweck. Zeit, Budget und Prioritäten sind begrenzt, und jede Entscheidung ist eine Abwägung zwischen konkurrierenden Zielen.

Wer diesen Unterschied nicht bewusst reflektiert, landet zwangsläufig im Frust. Organisationen erleben Entwicklung als langsam oder unflexibel. Entwickler erleben Organisationen als kurzsichtig oder qualitätsfeindlich. Beide Perspektiven greifen zu kurz. Das eigentliche Problem ist selten mangelnder Wille zur Qualität oder fehlendes Verständnis für Zwänge – sondern ein fehlendes gemeinsames Verständnis von Verantwortung.

An dieser Stelle wird der Kern des Missverständnisses sichtbar. Verantwortung wird entweder zu eng verstanden – rein technisch – oder zu weit – rein organisatorisch. Beides führt dazu, dass Entscheidungen dort getroffen werden, wo sie nicht fundiert getroffen werden können.

Wir sind nicht Opfer der Umstände – aber wir müssen akzeptieren, dass Professionalität mehr verlangt als gutes Handwerk.

Dieser Gedanke markiert den Übergang von Analyse zu Gestaltung. Erst wenn wir bereit sind, Verantwortung auch jenseits des rein

Technischen anzunehmen, entsteht die Grundlage dafür, den beschriebenen Frust tatsächlich zu reduzieren.

Zusammenarbeit auf Augenhöhe: Ein Zielbild jenseits von Ausführung und Kontrolle

Wenn der Frust vieler Entwickler nicht aus mangelnder Kompetenz entsteht, sondern aus einem Missverständnis von Rolle und Verantwortung, dann stellt sich zwangsläufig die Frage nach dem Zielbild. Wie sieht Zusammenarbeit aus, wenn wir nicht nur ausführen, aber auch nicht in eine Haltung verfallen, in der wir alles kontrollieren wollen? Wie können wir wirksam sein, ohne uns entweder zurückzuziehen oder ständig in Abwehrkämpfe zu geraten?

Zusammenarbeit auf Augenhöhe beginnt mit einer klaren Verschiebung des eigenen Selbstverständnisses. Wir sind weder reine Umsetzer vorgegebener Anforderungen noch die alleinige Instanz für die Entscheidung darüber, was „technisch richtig" ist. Unsere Rolle liegt dazwischen: als professionelle Partner im Entwicklungsprozess. Partner, die technische Expertise einbringen, Konsequenzen sichtbar machen und gemeinsam mit anderen Stakeholdern Entscheidungen treffen, die tragfähig sind – fachlich wie organisatorisch.

Dieses Zielbild setzt voraus, dass unterschiedliche Rollen unterschiedliche Perspektiven einbringen dürfen und sollen. Projektmanagement, Produktverantwortliche und Führungskräfte tragen Verantwortung für Markt, Zeit, Budget und Risiko. Wir als Entwickler tragen Verantwortung für Machbarkeit, Nachhaltigkeit, Wartbarkeit und technische Qualität. Keine dieser Perspektiven ist verzichtbar. Keine ist per se wichtiger als die andere. Augenhöhe entsteht nicht dadurch, dass alle alles entscheiden, sondern dadurch, dass alle relevanten Perspektiven in Entscheidungen einfließen.

Ein zentraler Baustein dafür ist eine gemeinsame Sprache. Technische Entscheidungen entfalten ihre Wirkung nicht allein dadurch, dass sie fachlich korrekt sind, sondern dadurch, dass ihre Auswirkungen verstanden werden. Wer erklären kann, welches Risiko ein Verzicht auf Tests bedeutet, welche Kosten technische Schulden langfristig verursachen oder welche Alternativen es gibt, verschiebt Diskussionen weg von Geschmacksfragen hin zu echten Abwägungen. Umgekehrt müssen fachliche und organisatorische Ziele so formuliert sein, dass wir sie technisch sinnvoll einordnen können. Augenhöhe entsteht dort, wo diese Übersetzungsleistung gelingt.

In einer solchen Zusammenarbeit verändern sich auch Entscheidungsprozesse. Zielkonflikte zwischen Qualität, Zeit und Kosten werden nicht mehr stillschweigend entschieden oder einfach „nach unten" durchgereicht. Sie werden explizit gemacht. Wenn entschieden wird, Tests zu reduzieren, Refactorings zu verschieben oder technische Schulden bewusst einzugehen, dann als gemeinsame Entscheidung – mit bekanntem Risiko und geteilter Verantwortung. Wir sind in diesem Modell nicht Bittsteller, sondern Mitentscheider. Nicht, weil wir Macht einfordern, sondern weil ohne unsere Perspektive keine fundierten Entscheidungen möglich sind.

Dieses Zielbild verlangt von allen Beteiligten ein Umdenken. Für Organisationen bedeutet es, Entwicklung nicht nur als Lieferant zu betrachten, sondern als Gestaltungsdisziplin. Für uns bedeutet es, Verantwortung nicht an der Grenze unseres eigenen Codes enden zu lassen. Es heißt, unbequeme Fragen zu stellen, Alternativen aufzuzeigen und Entscheidungen auch dann mitzutragen, wenn sie nicht dem idealen technischen Zustand entsprechen – solange sie bewusst, transparent und gemeinsam getroffen wurden.

Zusammenarbeit auf Augenhöhe ist kein harmonisches Ideal und kein konfliktfreier Zustand. Sie ist ein anspruchsvolles Arbeitsmodell. Sie verlangt Klarheit, Dialog und die Bereitschaft, Verantwortung nicht abzugeben, sondern zu teilen. Der Gewinn ist jedoch spürbar: weniger implizite Konflikte, nachvollziehbarere Entscheidungen und ein Arbeitsumfeld, in dem wir unsere Expertise tatsächlich wirksam einbringen können – statt zwischen Ausführung und Ohnmacht zu pendeln.

Professionell handeln heißt gestalten: Verantwortung zwischen Handwerk und Organisation

Das Zielbild aus dem vorherigen Kapitel ist attraktiv – und für viele von uns gleichzeitig unbequem. Denn es bedeutet, die eigene Rolle nicht nur anders zu sehen, sondern auch anders auszufüllen. Wenn wir den Frust, den wir im Entwicklungsalltag erleben, wirklich reduzieren wollen, reicht es nicht, bessere Rahmenbedingungen einzufordern oder auf Einsicht von außen zu hoffen. Wir müssen bereit sein, unseren Job neu zu begreifen – und ihn aktiv zu gestalten.

Der erste Schritt ist dabei überraschend nüchtern: die Realität professioneller Softwareentwicklung anzuerkennen. Software entsteht nicht im luftleeren Raum, sondern in Organisationen mit klaren Zielen, wirtschaftlichem Druck, Zeitvorgaben und politischen Zwängen. Diese Rahmenbedingungen lassen sich nicht einfach wegargumentieren. Wer so tut, als dürften sie keine Rolle spielen, landet zwangsläufig im Dauerfrust. Professionell zu handeln heißt, diese Realität zu akzeptieren – ohne sich ihr unterzuordnen.

Gleichzeitig dürfen wir unser Handwerk nicht kleinreden. Softwareentwicklung ist ein Handwerk mit klaren Grundlagen. Tests, saubere Architekturentscheidungen, verständliche Strukturen und Wartbarkeit sind keine Geschmacksfragen, sondern Voraussetzungen dafür, dass Systeme langfristig tragfähig bleiben. Diese Grundlagen sind nicht beliebig verhandelbar. Sie zu kennen, zu vertreten und im Zweifel auch zu verteidigen, gehört zu unserer Verantwortung. Nicht aus Prinzip, sondern weil wir die Folgen schlechter Entscheidungen fachlich abschätzen können. Wer hier dauerhaft nachgibt, verliert nicht nur Qualität, sondern langfristig auch Einfluss.

Der entscheidende Hebel liegt darin, Verantwortung sichtbar zu machen. Verantwortung endet nicht beim eigenen Code. Sie beginnt dort, wo Entscheidungen mit Konsequenzen anstehen. Das bedeutet, Risiken klar zu benennen, Alternativen aufzuzeigen und deren Auswirkungen verständlich zu erklären – nicht in technischen Details, sondern im Kontext der Ziele der Organisation. Wer diese Übersetzungsarbeit leistet, macht es anderen überhaupt erst möglich, fundierte Entscheidungen zu treffen. Das ist anstrengend. Aber es ist genau der Punkt, an dem Wirksamkeit entsteht.

Dazu gehört auch, Initiative zu ergreifen. In vielen Projekten scheitert gute Entwicklung nicht an falschen Entscheidungen, sondern an fehlenden Entscheidungen. Dinge bleiben liegen, weil niemand den ersten Schritt macht. Zuständigkeiten sind unklar, Konflikte werden vermieden, Diskussionen vertagt. Aktiv zu werden heißt hier nicht,

fremde Rollen an sich zu reißen oder alles kontrollieren zu wollen. Es heißt, Verantwortung dort zu übernehmen, wo sie sonst niemand wahrnimmt. Wer Gestaltung will, muss bereit sein, sichtbar zu werden und Haltung zu zeigen.

All das erfordert Entwicklung – nicht nur fachlich, sondern auch persönlich. Kommunikation, Konfliktfähigkeit, das Aushalten von Spannungen und das Verlassen der eigenen Komfortzone sind keine weichen Zusatzqualifikationen. Sie sind zentrale Bestandteile professioneller Softwareentwicklung. Sie entscheiden darüber, ob wir unsere Expertise wirksam einbringen oder ob sie im Frust versandet.

Der entscheidende Punkt ist: Wir sind nicht auf Gedeih und Verderb den Umständen ausgeliefert. Dieses Gefühl mag sich im Alltag einstellen, aber es ist nicht die ganze Wahrheit. Wir haben mehr Einfluss, als es sich oft anfühlt – vorausgesetzt, wir sind bereit, ihn zu nutzen. Nicht durch Konfrontation, nicht durch Rückzug, sondern durch aktives, professionelles Handeln.

Der Auftrag ist damit klar. Wenn wir wollen, dass sich unsere Arbeit besser anfühlt, wenn wir weniger Frust und mehr Wirksamkeit erleben wollen, dann dürfen wir nicht darauf warten, dass sich die Organisation zuerst bewegt. Wir müssen selbst aktiv werden. Verantwortung übernehmen. Gestalten.



Christian Seifert
gofore
perdian@perdian.de

Christian Seifert ist Brückenbauer zwischen Code, Köpfen und Komplexität. Als Principal Software Architect bei gofore unterstützt er Unternehmen dabei, ihre Softwarelandschaften zukunftssicher zu gestalten – technisch fundiert, nachhaltig und mit Fokus auf starke Teams. Seit fast 25 Jahren gestaltet und verantwortet er Softwareprojekte in unterschiedlichen Branchen und hat dabei gelernt, dass erfolgreiche Architektur nicht nur aus technischen Details besteht, sondern vor allem aus Menschen, die gemeinsam an guten Lösungen arbeiten.

Was macht ein Java-Entwickler in seiner Freizeit? Oder: 10 Jahre IT-Workshops für Kinder

Fin Labusch




Kids4IT
Zauberschule Informatik
CODE WEEK 2019
1. Zahlenzauber
2. Gedankenzauber
3. Bilderzauber im Internet

Vor zehn Jahren ist meine Tochter eingeschult worden, und ich habe mich gefragt, wie ich bei ihr das Interesse für Informatik wecken könnte. Zur gleichen Zeit wurde von LEGO® ein neues, programmierbares Robotik-Set für Grundschüler vorgestellt. In nur einer halben Stunde können Kinder damit einen kleinen Roboter bauen und ihn programmieren. Sie können so sehr schnell, leicht und spielerisch einen Einstieg in die Welt der Algorithmen finden. Jetzt, 10 Jahre später, bin ich Mitgesellschafter der Kids4IT und führe etwa alle zwei Monate einen Workshop für Kinder durch. Neben dem oben erwähnten Robotik-Set sind ein Scratch-Kurs, die „Zauberschule Informatik", und die wirbeligen Ozobots zu meinem Portfolio hinzugekommen. In diesem Artikel berichte ich von meinen Erfahrungen bei der Arbeit mit Kindern und wie ihr selbst aktiv Kurse für Kinder (mit-)gestalten könnt.

Also, steigt mit mir in die Zeitmaschine ein und wir reisen zurück in das Jahr 2016. In diesem Jahr wurde meine Tochter eingeschult, die allererste Code Week fand in Hamburg statt und mit LEGO Education WeDo 2.0 wurde ein bis heute sehr beliebter, programmierbarer Bausatz vorgestellt.

Eine kurze Geschichte der Zeit

Sowohl meine Frau als auch ich haben Informatik studiert und wir sind in der IT-Branche tätig. Schon früh haben wir uns überlegt, wie wir bei unserer Tochter das Interesse für Informatik wecken könnten. 2016 brachte LEGO dann mit WeDo 2.0 *[1]* einen neuen Bausatz heraus, der sich ganz leicht mit einer graphischen Programmiersprache auf einem Tablet programmieren ließ. Ich besorgte mir ein Exemplar dieses Sets und brachte es zur Code Week in Hamburg *[2]* mit, wo Kids4IT *[3]* einen Workshop anbot, bei dem LEGO Mindstorms zum Einsatz kam.

Mindstorms richtet sich an Kinder ab zirka 12 Jahre und man muss erst einmal etwa eine gute Stunde bauen, bevor das Modell programmiert werden kann. Gerade jüngere Kinder sind dafür zu ungeduldig und der Zusammenbau von Technik-Bricks ist zudem herausfordernder. Mit dem viel einfacher gehaltenen WeDo-Bausatz kommen Grundschulkinder schneller und frustfreier ans Ziel. Die erste Code Week verließ ich mit zwei Zielen: Ich entschloss mich, an der Grundschule meiner Tochter einen wöchentlichen Kurs in der Nachmittagsbetreuung anzubieten und die nächste Code Week mit einem eigenen Workshop aktiv zu unterstützen. Mit fünf weiteren Sets und Tablets begann damit ein neues „Hobby": Kindern spielerisch einen Einstieg in die Welt der Informatik anzubieten.

Die zweite Code Week fand 2017 in den viel größeren Räumlichkeiten der Hamburger Zentralbibliothek der Öffentlichen Bücherhallen statt und wurde auch aktiv von den Medien beworben *(siehe Abbildung 1)*. Der Andrang war groß und mit dem Beginn der Auftaktveranstaltung um 10 Uhr morgens versammelten sich bereits dutzende Kinder mit ihren Eltern vor dem Veranstaltungsraum. Zwei Mentor:innen, die mich an diesem Tag unterstützen sollten, meldeten sich beide unabhängig voneinander am Morgen krank. Ich weiß bis heute nicht, wie ich es in den gut vier Stunden geschafft habe, 80 bis 100 Kindern zu zeigen, wie sich mit LEGO ein Roboter bauen und



Abbildung 1a: Kinder bauen und programmieren kleine LEGO-Roboter während der Code Week 2017 (© Körber Stiftung/Sandra Schink)



Abbildung 1b (Alternative): Kinder bauen und programmieren kleine LEGO-Roboter während der Code Week 2017 (© Fin Labusch)

programmieren lässt *(siehe Abbildung 1)*. Wieder zu Hause angekommen, fiel ich nur noch ins Bett und wachte erst am späten Vormittag des nächsten Tages wieder auf.

Besonders eine Begegnung ist mir von diesem Tag in Erinnerung geblieben und ist, neben dem unbezahlbaren Lachen der Kinder, der Grund, weshalb ich das Hobby bis heute fortführe. Ein kurz vor dem Abitur stehendes Mädchen war sich unsicher, ob sie Informatik studieren sollte oder nicht. Mein Kurs hat sie darin bestärkt, diesen Weg zu gehen, worüber sich auch ihre Eltern sichtbar gefreut haben. Ich hoffe, dass sie diese Entscheidung nicht bereut hat – vielleicht liest du ja gerade diesen Artikel.

Ein paar Jahre später präsentierte mir YouTube ein Video mit dem Titel „Computer Science Unplugged – The Show" *[4]*. In einer einstündigen Show haben zwei Moderatoren grundlegende Algorithmen der Informatik den Kindern spielerisch im Publikum präsentiert – alles ganz ohne Computer! Das fand ich sehr spannend und die Suchmaschine meiner Wahl führte mich schnell zu den Seiten der Zauberschule Informatik der RWTH Aachen *[5]* und dem Verzeichnis Computer Science Unplugged *[6]*. Aus diesen Quellen bastelte ich mir ein Programm für eine eigene „Zauberschule Informatik", die auf der Code Week 2019 Premiere hatte.

2020 hatte meine Tochter die Grundschule beendet und ich hatte während der Zeit des Lockdowns ausreichend Zeit zu überlegen, ob und wie ich mein Hobby weiterführen konnte. Ich kontaktierte die Bücherhalle in meinem Stadtteil, in der man von der Idee begeistert war. Auf anfängliche Werbung für den Kurs konnte schnell verzichtet werden und so biete ich den Kurs dort noch heute etwa alle zwei Monate an. Neben der jährlich im Herbst stattfindenden Code Week kam auch noch die JavaLand4Kids *[7]* und das JavaFormumNord-4Kids (JFN4Kids) *[8]* hinzu.



Abbildung 2. Konferenzteilnehmer der JavaLand bauen den Duke aus Klemmbausteinen (© Fin Labusch)

2021 unterstützte ich eine Crowdfunding-Aktion für das alternative Klemmbausteinset Apitor Robot X *[9]*. Das Set richtet sich ähnlich wie Mindstorms an ältere Kinder. Ein Kurs in meiner Bücherhalle wurde von lediglich zwei Kindern besucht. Auf die Gründe für diesen „überschaubaren" Erfolg gehe ich später genauer ein.

Immer wieder habe ich während meiner Kurse mitbekommen, wie gerne auch Erwachsene noch einmal Kind sein würden, um mit Klemmbausteinen oder kleinen Robotern zu spielen. Mit der JavaLand4Kidults besteht dieses Angebot seit 2023 im Rahmen der JavaLand-Konferenz. Zuletzt konnten die Konferenzteilnehmer:innen den Duke nachbauen *(siehe Abbildung 2)*. Die Einnahmen wurden unter anderem dem Verein Trauerland *[10]* gespendet. Es sind noch Bausätze vorhanden, schickt mir gerne eine E-Mail, wenn Duke auch euer Heim schmücken soll.

Bei Kindern sehr beliebt ist die am MIT entwickelte grafische Programmiersprache Scratch *[11]*. Tausende von Kindern haben damit weltweit die tollsten Spiele und Programme mit einfachen Klicks zusammengestellt. Kids4IT bietet hier regelmäßig Kurse an. Scratch kann mit Hardware wie LEGO WeDo kombiniert werden und dies eröffnet beiden Welten neue Möglichkeiten. Seit 2025 biete ich einen Kurs an, bei dem die Kids eine aufregende 3D-Ballonjagd mit Scratch programmieren und den Steuerknüppel aus LEGO bauen, um damit einen Doppeldecker aus dem Spiel zu steuern *[12]*.

Unsere Zeitreise beenden wir im aktuellen Jahr mit den Ozobots *[13]*. Das sind etwa walnussgroße Roboter, vollgestopft mit LEDs, Sensoren und Motoren. Jüngere Kinder malen schwarze Linien, denen der Ozobot folgt. Farbcodes in den Linien lassen ihn bestimmte Aktionen ausführen, zum Beispiel Tanzen oder an der nächsten Kreuzung links abbiegen. Ältere Kinder können mit einer Scratch-ähnlichen Programmieroberfläche den Roboter jedoch frei programmieren und ihn so beispielsweise zum Ausgang eines Labyrinths führen.

In den folgenden Abschnitten stelle ich kurz die Inhalte und den Aufbau meiner Kurse vor.

LEGO Education WeDo 2.0

Einer der beliebtesten Workshops widmet sich dem Bau und der Programmierung kleiner Roboter mit LEGO Education WeDo 2.0. Der dänische Spielwarenhersteller hat mit diesem Baukasten fast alles richtig gemacht und das macht auch den Erfolg aus. Das Set wurde von 2016 bis 2021 hergestellt, besteht aus 280 Teilen und kommt mit einem stabilen, stapelbaren Sortierkasten. Das sorgt für Ordnung und 280 Teile sind auch ruckzuck wieder einsortiert – ein nicht zu unterschätzender Punkt, insbesondere für den Kursleiter. Jeder, der schon mal ein größeres Mindstorm-Set mit über 1000 Teilen sortieren musste, weiß, wovon ich spreche.

Aber selbst mit nur 280 Teilen lassen sich viele verschiedene Modelle bauen, das Set selbst bietet 32 Modelle vom Roboter über Rennautos, Schiffe, Kräne bis hin zu Spinnen und Kaulquappen oder Erdbebensimulatoren *(siehe Abbildung 3)*. Je nach Modell beträgt die Bauzeit zwischen 5 und 25 Minuten und noch einmal zirka 5 Minuten zur Programmierung, das heißt, nach einer halben Stunde steigt der Lärmpegel merklich, wenn die Kinder mit dem Modell experimentieren. Die zugehörige App kommt fast ohne Text aus und ist weitestgehend selbsterklärend. Sie läuft sehr stabil und Verbindungsabbrüche zwischen App und Roboter treten fast

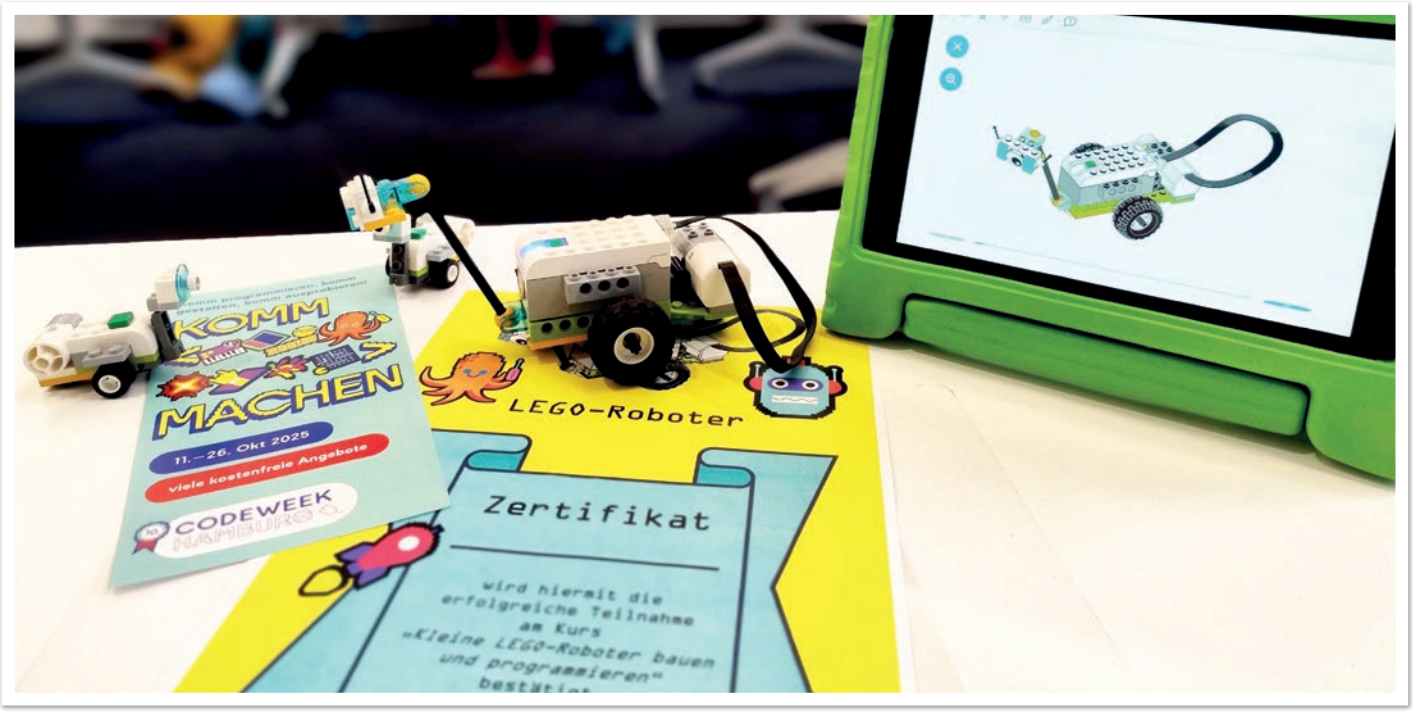


Abbildung 3: Kinder bauen mit LEGO WeDo 2.0 einen kleinen Roboter (© Fin Labusch)

nie auf. Es gibt andere MINT-Sets, bei denen die App nicht richtig funktioniert oder Verbindungen immer mal wieder abbrechen. So etwas ist extrem frustrierend für Kinder – und schweißtreibend für den Kursleiter.

Ein Negativbeispiel hierfür ist zum Beispiel die App zum bereits erwähnten alternativen Klemmbausteinset Apitor. Die App lief anfangs nur auf Smartphones und nicht auf Tablets. Das Zusammenzuschieben von Anweisungsblöcken ist auf dem kleinen Bildschirm herausfordernd. Leider sind mit den vorhandenen Blöcken, auch keine guten Programme zu schreiben. Möchte man zum Beispiel die Geschwindigkeit in Abhängigkeit vom Abstandsensor ändern, ist für jeden Sensorwert eine eigene if-Anweisung erforderlich, um die Geschwindigkeit anzupassen, da der Sensorwert nicht direkt als Eingabewert für die Geschwindigkeit genutzt werden kann. Die App läuft inzwischen auf Tablets und ein Sortierkasten für die mehr als 600 Teile ist jetzt verfügbar. An den Programmblöcken hat sich jedoch nichts geändert, sodass ich in einem Freizeitprojekt *[14]* kürzlich eine auf Blockly *[15]* basierende Webanwendung gebaut habe, bei der der Webbrowser direkt per Bluetooth mit dem Smarthub kommunizieren kann *[16]*.

Das 2021 erschienene Nachfolgeset, LEGO Education SPIKE Essential war deutlich teurer, größer und hat sich als Thema ausschließlich auf das Szenario Jahrmarkt konzentriert. Ich habe es nie ausprobiert und es gibt auch keine Nachbauten aus China. Dieses Set wird dieses Jahr durch LEGO Education Computer Science & AI abgelöst, nochmals teurer und der Fokus liegt – wie könnte es heutzutage nicht anders sein – auf Künstlicher Intelligenz (KI).

Anfangs hatte ich meinen LEGO-Kurs, der auf 3 Stunden ausgelegt ist, streng durchgeplant: Mit Vorgaben der Modelle und zu lösenden Aufgabenzetteln. Im Schulunterricht mag das funktionieren, nicht aber wenn die Kids einen spaßigen Vormittag verbringen sollen. Heute baue ich mit den Kids nur noch das Einstiegsmodell und danach haben sie gut zwei Stunden Zeit selbst zu wählen, welches Modell sie bauen möchten – oder sie können selbst kreativ werden und gänzlich eigene Konstruktionen zum Leben erwecken. Es ist immer wieder erstaunlich, was für großartige Ideen die Kinder haben. Um das Verständnis der Programmblöcke noch etwas zu verbessern, müssen die Kinder zu Beginn des Kurses mich programmieren: Sie müssen ein Programm schreiben, mit dem ich einmal durch den Raum gehe und danach wieder zurück zur Ausgangsposition. Das finden die Kinder als Einstieg und zur Lockerung der Atmosphäre nicht nur



Abbildung 4: Kinder haben in der Zauberschule Informatik das Sortiernetzwerk erfolgreich durchlaufen (© Fin Labusch)

spaßig, sondern die von ihnen geschriebenen Programme sind im Anschluss auch effizienter.

Bis zu zehn Kinder kann ein Kursleiter noch allein betreuen, danach ist eine weitere Person hilfreich. Der Kurs ist überschaubar in der Vorbereitung und recht entspannend durchzuführen – es müssen aber alle oben genannten Faktoren optimal zusammenspielen.

Zauberschule Informatik

Deutlich mehr Vorbereitung und Materialeinsatz benötigt die „Zauberschule Informatik" und ohne mindestens einer Assistenz für den Zauberer wird es schwierig. Dafür – und das ist auch das Besondere an diesem Kurs – kommt er völlig ohne Computer, Tablet oder Smartphone aus. Die Kinder lernen grundlegende Prinzipen und Algorithmen kennen: Zum Beispiel das Binärsystem, Datenübertragung im Internet, Sortier- und Suchalgorithmen, Algorithmen zur Fehlererkennung, Komprimierung oder gar Markov-Ketten. Alles ist dabei in kleine Zauberkunststücke verpackt und die Kinder müssen bei jedem Zaubertrick aktiv mitmachen. Natürlich wird jede Zauberei aufgelöst, sodass die Kinder abends auch ihre Eltern verblüffen können – oder selbst gestandene Informatiker:innen.

Mein Kurs beginnt mit einem Kennenlern-Spiel: Auf einer 5x5 Meter großen Plane ist ein Sortiernetzwerk abgebildet *(siehe Abbildung 4)*. Die Kinder gehen oder hüpfen durch dieses Netzwerk, vergleichen ihre Namen oder Körpergrößen und am Ausgang stehen sie alle sortiert nebeneinander. Sie haben eben ihren ersten – parallel ausgeführten (!) – Algorithmus zum Sortieren kennengelernt.

Weiter geht es mit einer kleinen Zeitreise zu den Pionier:innen der Informatik: Charles Babbage, Ada Lovelace, Konrad Zuse, Grace Hopper, Alan Turing, Vinton Cerf, Christiane Floyd oder Stephanie Wehner und wie sich Computer in dieser Zeit von raumfüllenden Monstren zum Supercomputer am Handgelenk entwickelt haben.

In der ersten Zauberei lernen die Kinder wie ein Computer anhand von Binärkarten zählen. Dass sich die Wertigkeit der Karten jedes Mal verdoppelt (1, 2, 4, 8, 16, 32) bekommen die Kids sehr schnell mit. Und darin liegt auch der Trick der zweiten Zauberei, dem Gedankenlesen, bei den der Zauberer eine gedachte Zahl ganz schnell erraten kann. Anschließend werden Texte (ASCII-Code) und Bilder verzaubert. Die Bilder werden dabei über ein Springseil (das „Internet") übertragen. An beiden Enden des Seils steht ein Modem/Kind, das 0 und 1 in Schwingungen versetzt beziehungsweise diese in 0 und 1 dekodiert. Ein Digitizer an einem Ende wandelt ein Pixelbild in 0 und 1 und ein Monitor am anderen Ende malt aus 0 und 1 wieder ein Pixelbild. Ein fünftes Kind als Taktgeber synchronisiert die beiden Modems.

Wenn es dabei zu Datenübertragungsfehlern kommt, hilft der Kartenzaubertrick weiter: Auf einem (magnetischen) Whiteboard kann ein Kind rot-grüne Karten beliebig auf einem 4x4 oder 6x6 Raster anordnen. Der Zauberer dreht sich dann weg und das Kind kann eine beliebige Karte umdrehen – zuvor macht die Assistenz die Aufgabe für den Zauberer jedoch noch schwieriger, in dem er/sie noch eine weitere Reihe und Spalte an Karten hinzufügt. Der Zauberer dreht sich wieder um und kann treffsicher die umgedrehte Karte identifizieren. Wie bei Zaubereien üblich, liegt die Lösung in der Ablenkung, denn die Assistenz hat die zusätzlichen Karten nach einem zuvor abgestimmten Prüfziffernalgorithmus abgelegt *(siehe Abbildung 5)*.

Es folgen weitere Zaubertricks und zum Abschluss gibt es zum einen die Geschichte vom Weihnachtsmann, die auf einer binären Suche aufbaut und damit den Kreis zum Sortieralgorithmus vom Anfang schließt. Als abschließendes Schmankerl bekommt jedes Kind noch ein Stückchen Kuchen – von einem Miniküchlein. Ganz nach Informatiker-Manier wird der Kuchen geteilt und das Problem, jedem Kind ein Stück Kuchen zu geben, so mit jedem Schnitt kleiner.

Auch dieser Kurs ist auf zirka 3 Stunden ausgelegt. Durch die Vielzahl der möglichen Zaubereien – Computer Science Unplugged bietet hier noch viele Ideen – lässt sich der Kurs auch auf eine 45-minütige Show reduzieren, wie ich sie im Rahmen der Code Week schon mehrmals aufgeführt habe.

Alter, Geschlecht und Corona

Die von mir angebotenen Kurse richten sich an Grundschulkinder. Kinder in diesem Alter erforschen die Welt noch völlig unvoreingenommen und lassen sich noch mehr begeistern, wenn Wissen spielerisch verpackt wird. Es gibt immer mal wieder Kinder, die anfangs noch etwas schüchtern sind, am Ende sind bisher aber immer alle mit großer Begeisterung dabei gewesen oder würden gar noch länger bleiben – und drei Stunden durchzuhalten ist schon

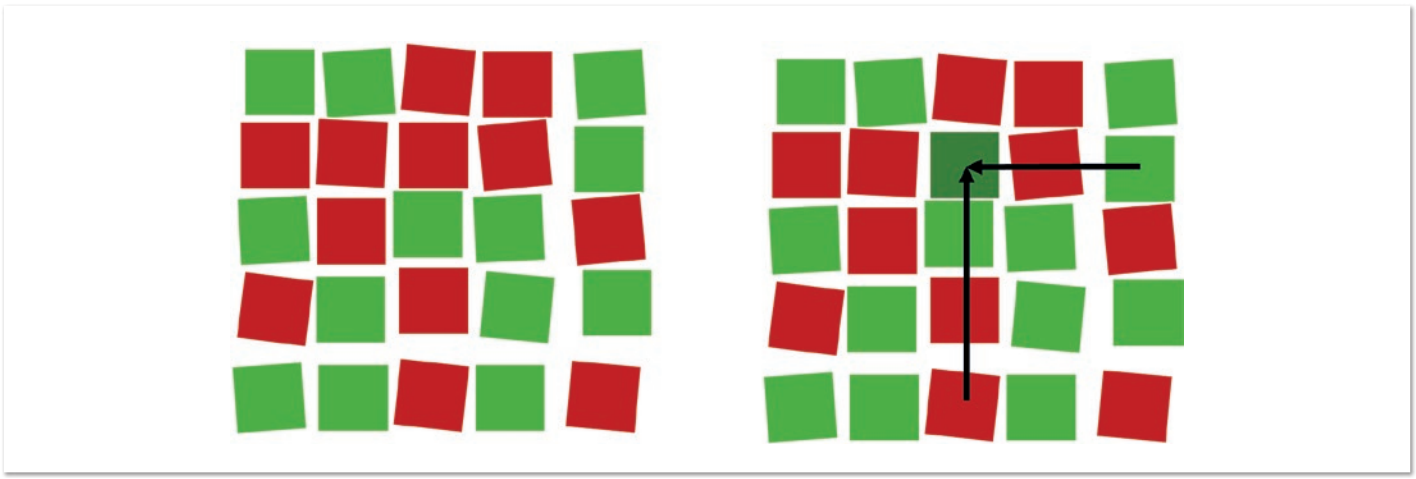


Abbildung 5: Kartentrick. Sobald eine Karte umgedreht wird, stimmt die „Prüfziffer" in der letzten Spalte und Zeile nicht mehr. (© Fin Labusch)

eine beachtliche Leistung! Für jüngere Kinder sind drei Stunden definitiv zu viel. Hier sind 60 bis 90 Minuten mehr als ausreichend und die Ozobots sind für diese Zielgruppe gut geeignet. Es gibt sehr wohl MINT-Sets für ein noch jüngeres Publikum. Der Markt dafür ist riesig, was man merkt, wenn man sich erstmal näher damit beschäftigt. Ab einem Alter von 12 Jahren wird es deutlich schwieriger: Jugendliche habe ihre eigenen Interessen und die Teilnehmer:innen beschäftigen sich dann auch zuhause mit dem Thema. Das kann für Mentor:innen herausfordernd werden, wenn die Teilnehmer:innen die Tools besser kennen als man selbst…

Im Grundschulalter gibt es noch keine extrem ausgeprägten geschlechterspezifischen Klischees. Jungen und Mädchen lassen sich gleichermaßen für die Thematik begeistern. Erste Anzeichen sind jedoch erkennbar, denn das Rennauto ist bei Jungs meistens die erste Wahl während Mädchen häufiger den Hubschrauber aus LEGO-Steinen bauen – er rettet ein Pandabärchen vor dem Hochwasser. Vor diesem Hintergrund haben wir in meiner Bücherhalle im letzten Jahr zum ersten Mal einen reinen Mädchenkurs angeboten. Die Entscheidung dafür war schwierig: Kann oder darf ein Mann einen Kurs nur für Mädchen anbieten? Wir haben uns für eine offene Kommunikation gegenüber den Eltern entschieden. Im ersten Kurs waren auch Mädchen dabei gewesen, die zuvor schon einen gemischten Kurs besucht haben. Zwei von drei Mädchen fanden den reinen Mädchenkurs angenehmer, sodass wir das Angebot dauerhaft aufgenommen haben. Kurse mit einem Mädchenanteil von mehr als 30 % sind übrigens zu Beginn deutlich ruhiger – bis das erste Modell fertig gebaut ist. Begeisterung lässt sich in Dezibel messen.

Die Corona-Pandemie hat jedoch auch Auswirkungen auf die Psyche/Mentalität der Kids gehabt: Ich habe festgestellt, dass etwa zwei Jahre nach der Pandemie die Aufmerksamkeit und Motivation – insbesondere bei Jungen – deutlich nachgelassen hat und einige nach zwei, zweieinhalb Stunden kein Interesse mehr am Bau von Legomodellen hatten. Ich hatte schon überlegt, die Kursdauer entsprechend zu verkürzen, doch in diesem Jahr waren drei Stunden wieder kein Problem und die Kids haben die gleiche Begeisterung wie vor der Pandemie gezeigt.

Umgang mit Störungen

Zunächst einmal muss ich herausstellen, dass ich – bis auf einmal – kein Problem mit Störungen hatte. Im genannten Fall war es ein Kurs in der Praktikumswoche an der Grundschule meiner Tochter und die Schulleitung hatte mich im Vorfeld aufmerksam gemacht und wie die Situation zu handhaben war. Ich bin studierter Informatiker und habe keine spezielle Ausbildung zum Umgang mit schwierigen Situationen mit Kindern. Allerdings ist die Situation auch eine andere: Alle Kids besuchen die Kurse freiwillig – also eine ganz andere Ausgangsbasis als gegenüber Pflichtkursen wie zum Beispiel in der Schule.

Dennoch empfiehlt es sich einen Kurs nie allein durchzuführen oder in einer Umgebung, in der notfalls geschultes Personal zur Verfügung steht. Eine zweite oder dritte Person ist schon deshalb sinnvoll, weil es immer Fragen gibt oder die Kids bei einer Problemstellung Hilfe benötigen – selbst bei so gut durchdachten Sets wie dem WeDo. Und ihr könnt euch das Lachen und die Begeisterung der Kids teilen – ein unbezahlbares Honorar.

Wie kann ich mitmachen?

Es gibt in Deutschland eine Vielzahl von Organisationen und Vereinen, die MINT-Kurse für Kinder und Jugendliche anbieten. Die MINT-vernetzt Plattform führt in ihrer Community über 1000 Vereine und Organisationen auf *[17]*. Viele arbeiten professionell, andere wie beispielsweise Kids4IT, bei der ich seit 2023 Mitgesellschafter bin, auf ehrenamtlicher Basis. Wenn du selbst Kinder hast, besuche mit deinem Kind einfach mal einen Kurs und nimm Kontakt mit dem Veranstalter auf. Wir freuen uns immer über neue Mentor:innen, die uns bei unseren Kursen unterstützen – oder gar selbst einen Kurs organisieren.

Eine andere Möglichkeit sind große Events: Die Code Week findet europaweit jedes Jahr im Herbst statt, mit hunderten Veranstaltungen alleine in Deutschland. Im Sommer gibt es den Digitaltag *[18]* und lokal findet man auch Veranstaltungen, zum Beispiel den Hamburger MINT-Tag *[19]*. Und wenn du gerne Konferenzen besuchst, auch da gibt es Möglichkeiten: zum Beispiel bei der JavaLand4Kids oder der JFN4Kids.

Fazit

Ein Zuhörer meines Vortrags auf der JavaLand hatte am Ende die Bemerkung, dass ich im Jahr ziemlich viele Kurse anbiete – etwa 12 Stück. Es kommt mir gar nicht so viel vor, was vor allem daran liegt, dass es mir viel Spaß bereitet, Kinder für ein paar Stunden aus dem Alltag herauszuholen und ihnen die Zauberwelt der Informatik zu zeigen. Jedes Lachen motiviert mich wahnsinnig dazu, den nächsten Kurs zu planen.

Bleibt noch die Frage eines anderen Zuhörers: Konnte ich meine Tochter für Informatik begeistern? Die Antwort ist ernüchternd: Nein. Papa ist dennoch stolz auf sie, denn sie hat sich seinerzeit für Chinesisch als Wahlpflichtfach entschieden und spricht nun auch fließend Englisch und Spanisch. Digitale Medien gehören für sie zum Alltag, sie zaubert mit Adobe After Effects kurze Videos oder nutzt KI, um ihr Abi-Wissen abfragen zu lassen.

Wer meinen Vortrag auf der JavaLand verpasst hat: Ich habe kurz vor Redaktionsschluss die Zusage für die IT-Tage 2026 im Dezember in Frankfurt bekommen *[20]*.

Quellen

[1] LEGO Education WeDo 2.0: *https://education.lego.com/de-de/products/lego-education-wedo-2-0-set/45300/*
[2] Code Week Hamburg: *https://hamburg.codeweek.de/*
[3] Kids4IT gUG: *https://www.kids4it.de/*
[4] Computer Science Unplugged – The Show: *https://youtu.be/VpDDPWVn5-Q?si=PodbaFMuIP-89jm0*
[5] RWTH Aachen – Zauberschule Informatik: *https://www.info-sphere.rwth-aachen.de/cms/infosphere/angebote/infosphere-workshops/~bhbiud/workshop-zauberschule-informatik/*
[6] Computer Science Unplugged: *https://www.csunplugged.org/de/*
[7] JavaLand: *https://www.javaland.eu*
[8] Java Forum Nord: *https://javaforumnord.de*
[9] Crowdfunding Apitor Robot X: *https://www.startnext.com/en/apitor-robot-x/more-info*
[10] Trauerland – Zentrum für trauernde Kinder und Jugendliche eV: *https://www.trauerland.org/*
[11] Scratch: *https://scratch.mit.edu/*

[12] Ballonjagd mit Scratch und Lego WeDo: *https://github.com/kids4it/lego-wedo-scratch*
[13] Ozobot Deutschland: *https://ozobot-deutschland.de/*
[14] Freizeitprojek Blocktronic: *https://futzl.de/blocktronic/*
[15] Blockly: *https://developers.google.com/blockly?hl=de*
[16] Web Bluetooth API: *https://developer.mozilla.org/de/docs/Web/API/Web_Bluetooth_API*
[17] MINTvernetzt: *https://www.mint-vernetzt.de/*
[18] Digitaltag: *https://digitaltag.eu/*
[19] Hamburger MINT-Tag: *https://mintforum.de/*
[20] IT-Tage 2026: *https://www.ittage.informatik-aktuell.de/index.html*



Fin Labusch
kontakt@labusch.de

Fin Labusch hat in Hamburg Informatik studiert und entwickelt mit Java seit über 25 Jahren Fullstack-Webanwendungen für namhafte Kunden in den Branchen Energie, Versicherung und Finanzen. In den letzten Jahren war er freiberuflich unterwegs und ist aktuell dabei, sich beruflich neu zu orientieren. Er ist Mitgesellschafter der Kids4IT gUG, gibt in seiner Freizeit IT-Kurse für Kinder und entwickelt privat ein Energiemanagement-system und eine auf Blockly basierende Browseranwendung zur Steuerung alternativer Klemmbausteinroboter.

JavaLand



Mit dem **On-demand-Ticket**
bestes Java-Wissen nach Hause holen
und **Vortragsaufzeichnungen** und
-unterlagen für mindestens
ein Jahr genießen!

#JavaLand | www.javaland.eu

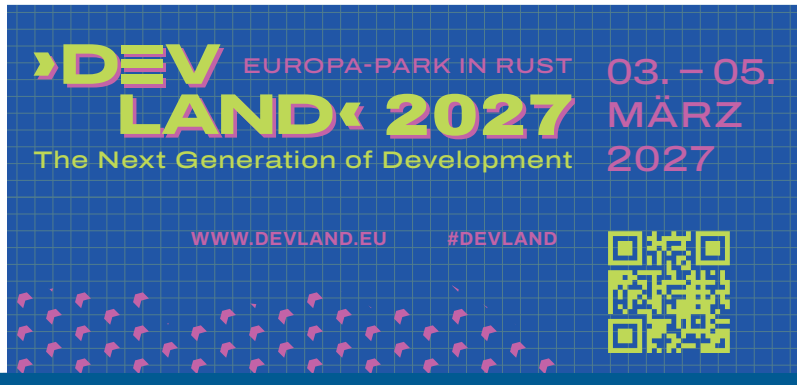
Präsentiert von: iJUG Verbund · heise medien · DOAG
Veranstalter: JavaLand

EVENT KALENDER









Impressum

Java aktuell wird von der Interessensgemeinschaft aus DOAG e.V., JavaLand GmbH und DevLand GmbH (Tempelhofer Weg 64, 12347 Berlin, java-aktuell.eu) herausgegeben.
Es ist das User-Magazin rund um die Programmiersprache Java im Raum Deutschland, Österreich und Schweiz. Es ist unabhängig von Oracle und vertritt weder direkt noch indirekt deren wirtschaftliche Interessen. Als herstellerunabhängiger Verein bietet die DOAG Software-Anwenderinnen und -Anwendern sowie Entwicklerinnen und Entwicklern eine Plattform für den Wissens- und Erfahrungsaustausch.

Java aktuell wird verlegt von der DOAG Dienstleistungen GmbH, Tempelhofer Weg 64, 12347 Berlin, Deutschland, gesetzlich vertreten durch den Geschäftsführer Fried Saacke, deren Unternehmensgegenstand Vereinsmanagement, Veranstaltungsorganisation und Publishing ist.

DOAG e.V. hält 100 Prozent der Stammeinlage der DOAG Dienstleistungen GmbH. DOAG e.V. wird gesetzlich durch den Vorstand vertreten; Vorsitzender: Jonas Janz. DOAG e.V. informiert kompetent über alle Oracle-Themen, setzt sich für die Interessen der Mitglieder ein und führen einen konstruktiv-kritischen Dialog mit Oracle.

Redaktion:
Sitz: DOAG Dienstleistungen GmbH
ViSdP: Fried Saacke
Redaktionsleitung: Lisa Damerow
Kontakt: redaktion@java-aktuell.eu

Redaktionsbeirat:
Andreas Badelt, Marcus Fihlon, Markus Karg, Manuel Mauky, Bernd Müller, Benjamin Nothdurft, Daniel van Ross, Bennet Schulz

Titel, Gestaltung und Satz:
Alexander Kermas,
DOAG Dienstleistungen GmbH

Bildnachweis:
Titel: Bild © Bild erstellt mit ChatGPT von OpenAI
https://chatgpt.com
S. 10 + 11: pch.vector
https://freepik.com
S. 14 + 15: Bild erstellt mit ChatGPT von OpenAI
https://chatgpt.com
S. 22 + 23: Bild © Suger Bloom
https://stock.adobe.com
S. 32: Bild © pitinan
https://123rf.com
S. 38 + 39: Bild © lamin
https://stock.adobe.com
S. 44 + 45: Bild © naufal
https://stock.adobe.com
S. 52: Bild © rinrada
https://stock.adobe.com
S. 58 + 59: Bild © yakobali
https://123rf.com
S. 64 + 65: Bild © Elite Studio
https://stock.adobe.com
S. 70 + 71: Bild © Fin Labusch

Anzeigen:
DOAG Dienstleistungen GmbH
Kontakt: sponsoring@doag.org
Mediadaten und Preise:
www.doag.org/go/mediadaten

Druck:
WIRmachenDRUCK GmbH
www.wir-machen-druck.de

Alle Rechte vorbehalten. Jegliche Vervielfältigung oder Weiterverbreitung in jedem Medium als Ganzes oder in Teilen bedarf der schriftlichen Zustimmung des Verlags.

Die Informationen und Angaben in dieser Publikation wurden nach bestem Wissen und Gewissen recherchiert. Die Nutzung dieser Informationen und Angaben geschieht allein auf eigene Verantwortung. Eine Haftung für die Richtigkeit der Informationen und Angaben, insbesondere für die Anwendbarkeit im Einzelfall, wird nicht übernommen. Meinungen stellen die Ansichten der jeweiligen Autoren dar und geben nicht notwendigerweise die Ansicht der Herausgeber wieder.

Inserentenverzeichnis

DevLand GmbH	U 2, S. 9
DOAG e. V.	S. 51, U 3
Java aktuell	S. 21, S. 31,
JavaLand GmbH	S. 13, S. 77, U 4

DOAG

APEX connect

CINEDOM KÖLN



5.–7.
APRIL
2027

apex.doag.org

JavaLand

JAVA COMMUNITY KONFERENZ



2.–4. MÄRZ

Europa-Park in Rust

Mehr Infos!

www.javaland.eu

Veranstalter ist die Javaland GmbH
im Auftrag des DOAG e.V.

Eventpartner: DOAG | iJUG Verbund | heise medien